MATERIAL AUTORAL

Projeto Integrador – Capstone Project

Disciplina 11 • Engenharia de Software em IA Aplicada

Autoria: Prof. Alvaro Camillo Neto

Edição didática completa • Conteúdo integral das aulas • GitHub • práticas • diagramas • checkpoints

Como usar este material

Este material foi elaborado a partir do conteúdo integral das aulas da disciplina. Os conceitos, exemplos e afirmações seguem a transcrição fornecida; a edição reorganiza o conteúdo de forma didática e acrescenta destaques, recursos visuais, prática orientada e navegação para facilitar o estudo. Os prompts reproduzidos nas aulas foram extraídos dos roteiros.

PRINCÍPIO DE FIDELIDADE

- Todo o conteúdo vem das aulas: nenhum conceito, exemplo ou dado foi acrescentado na edição.
- Nenhum gráfico numérico foi inventado: quando há gráfico, os valores aparecem explicitamente nas aulas.
- Diagramas, fluxos e destaques são recursos editoriais para organizar conceitos já apresentados nas aulas.
- Os prompts enviados ao Claude Code e ao Claude Design aparecem na íntegra, como constam nos roteiros das aulas.
- Os links do GitHub apontam para o repositório oficial e para os projetos relacionados à disciplina.

REPOSITÓRIO OFICIAL DA DISCIPLINA

Capstone Project AI Researcher no GitHub

Use o repositório como parte do estudo: leia a explicação, reproduza o código e depois adapte o exemplo para outro problema. Para construir o projeto passo a passo, o ponto de partida do estudante é o AI Researcher Seed.

COMO ESTUDAR CADA AULA

1. **Entenda:** leia os objetivos e a visão geral antes do conteúdo.
2. **Acompanhe:** avance pelas seções na ordem e use os destaques para fixar conceitos, exemplos e pontos de atenção.
3. **Pratique:** abra o projeto relacionado no GitHub quando houver.
4. **Valide:** responda ao checkpoint sem consultar o texto.
5. **Adapte:** modifique o exemplo e aplique o conceito em um cenário diferente.

Introdução do material

Esta disciplina reúne o que foi estudado nas outras matérias em um projeto construído do começo ao fim, do primeiro commit à entrega em produção na nuvem. O projeto é o AI Researcher, um chatbot especializado em investimentos de bolsa: ele busca em uma base vetorial com dados públicos da CVM, julga se o contexto encontrado é suficiente, recorre à web quando não é e responde citando a fonte, sem nunca recomendar compra ou venda.

O percurso tem cinco unidades. A primeira apresenta o projeto, configura o agente de código, percorre a arquitetura, coleta o corpus da CVM, provisiona a infraestrutura e coloca o primeiro deploy no ar. A segunda constrói o cérebro do agente: o banco vetorial, os nós do grafo, a busca na web, o guardrail e a resposta por streaming. A terceira trata de testes, resiliência, erros padronizados e da exposição do agente por MCP. A quarta constrói o frontend, do protótipo ao teste de ponta a ponta. A quinta cuida de observabilidade, controle de custo, documentação e encerramento do ambiente.

Escrevi o código com o Claude Code, prompt a prompt, e por isso os prompts aparecem reproduzidos no material. A ferramenta, porém, é o que menos importa: ferramentas vão e vêm. Ao final, você saberá construir, testar, publicar, observar e encerrar uma aplicação agêntica, e poderá refazer esse caminho com outro agente, outra nuvem ou outra linguagem. O que sustenta tudo isso são os fundamentos: padrões de projeto, segurança, testes, resiliência e a responsabilidade de ler e entender o código que o agente escreve.

Público

Mapa da disciplina

Clique em uma unidade ou aula para navegar diretamente pelo documento.

Unidade 1: Ideação, Setup e Deploy Inicial
- Aula 1: Bem-vindo ao Capstone
- Aula 2: CLAUDE.md e AI Dev Skills
- Aula 3: Tour pelo Google Cloud
- Aula 4: GitHub e GitHub Actions
- Aula 5: Corpus CVM
- Aula 6: Fundação GCP
- Aula 7: Walking Skeleton e Deploy Zero

Unidade 2: Orquestração Cognitiva e APIs
- Aula 8: pgvector, seed
- Aula 9: Os nós retrieve e grade
- Aula 10: Serper
- Aula 11: Fallback pra busca web, upsert retroativo
- Aula 12: O grafo real: aresta condicional e o guard do loop
- Aula 13: O nó answer, citação obrigatória, guardrail por código e streaming SSE

Unidade 3: Qualidade, Resiliência e MCP
- Aula 14: Testando o grafo sem rede
- Aula 15: Circuit breaker, retry e degradação graciosa
- Aula 16: Erros padronizados
- Aula 17: O servidor MCP

Unidade 4: Frontend
- Aula 18: Do protótipo no Claude Design à skill frontend-design
- Aula 19: Chat com streaming e citações clicáveis
- Aula 20: Painel de estado do grafo em tempo real
- Aula 21: Disclaimer, guardrail visual e o E2E no CI

Unidade 5: Observabilidade e produção
- Aula 22: Langfuse: tour e instrumentação por nó
- Aula 23: Métricas, budget de tokens e rate limiting
- Aula 24: README e desligando o ambiente pelo Console do GCP

Público

Unidade 1: Ideação, Setup e Deploy Inicial

Do panorama do projeto ao primeiro deploy em produção, ainda sem IA na resposta.

NESTA UNIDADE

- Aula 1: Bem-vindo ao Capstone
- Aula 2: CLAUDE.md e AI Dev Skills
- Aula 3: Tour pelo Google Cloud
- Aula 4: GitHub e GitHub Actions
- Aula 5: Corpus CVM
- Aula 6: Fundação GCP
- Aula 7: Walking Skeleton e Deploy Zero

Da ideia ao deploy zero



Fluxo conceitual criado para apoiar a leitura da unidade.

GITHUB E PRÁTICA

- Projeto completo: Capstone Project AI Researcher
- Ponto de partida do estudante: AI Researcher Seed

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público

Aula 1: Bem-vindo ao Capstone

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Reconhecer o que a disciplina constrói: o AI Researcher, um chatbot especializado em investimentos de bolsa, do primeiro commit à produção.
- Explicar, em alto nível, a regra de negócio do agente: buscar na base vetorial, julgar o contexto e recorrer à web quando ele não basta.
- Justificar por que a restrição de não recomendar investimentos é garantida no código, e não apenas no system prompt.

Visão geral

Esta disciplina aplica o que foi aprendido nas demais matérias em um único projeto, construído do começo ao fim: do primeiro commit até a entrega em produção, na nuvem. O projeto é o AI Researcher, um chatbot especializado em investimentos de bolsa.

Esta primeira aula apresenta o produto pronto e a sua regra de negócio em alto nível. Ao final, você saberá dizer o que o sistema faz com uma pergunta, por que ele nunca recomenda compra ou venda e o que, além do agente, faz parte do projeto.

O que a disciplina constrói: do primeiro commit à produção

O projeto percorre todas as etapas de um sistema real. Além do código da aplicação, ele inclui:

- observabilidade, para acompanhar o sistema em funcionamento;
- FinOps, inclusive aplicado a sistemas que consomem LLMs;
- deploy automatizado, com o GitHub Actions publicando o projeto na nuvem.

Em todo o projeto há observabilidade. Uso o Grafana para verificar as requisições e a latência, e outra ferramenta de observabilidade é apresentada mais adiante, quando o projeto chega à produção.

O AI Researcher: um chatbot de investimentos

O AI Researcher é um chatbot especializado em investimentos de bolsa. Escolhi esse exemplo porque ele exercita as duas frentes da disciplina ao mesmo tempo: é um projeto em que agentes são usados para desenvolver o código e é, ele próprio, um sistema que consome LLMs. O agente que pesquisa e dá a resposta também é construído ao longo das aulas.

A interface tem um recurso que ajuda a entender o que acontece por dentro: o pipeline de raciocínio. Ele mostra cada etapa que o sistema percorre para responder, inclusive os erros, caso ocorram.

A regra de negócio: buscar na base, julgar o contexto e recorrer à web

A regra de negócio do agente tem três movimentos. Quando o usuário faz uma pergunta, o sistema busca na base vetorial, a base de RAG montada para ele. Se o contexto encontrado é suficiente, a resposta é dada com a citação da base. Se não é suficiente, e é o próprio sistema que faz esse julgamento, ele busca na internet.

Público

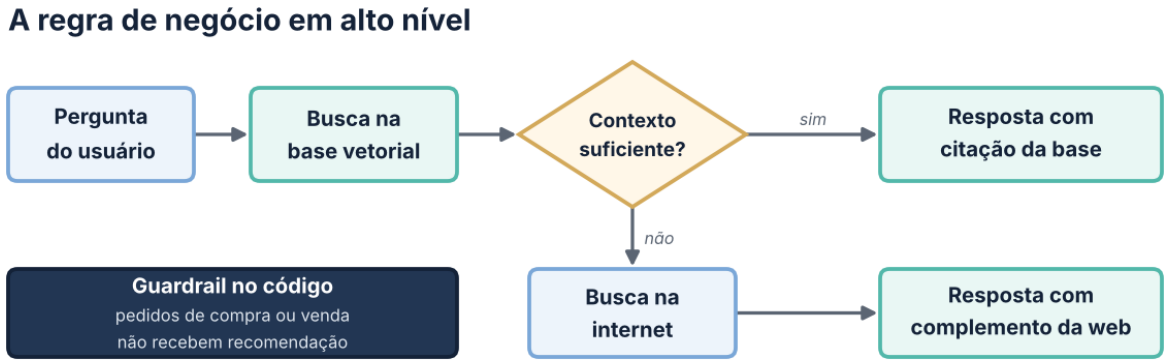


Diagrama editorial com o fluxo de decisão descrito na aula.

> **EXEMPLO · Pergunta respondida pela base**
> À pergunta "Qual é a receita líquida da Vale no último semestre?", o sistema responde com o que encontrou na base de RAG. O pipeline de raciocínio mostra que ele não inventou a informação e também não foi buscá-la na internet.

> **EXEMPLO · Pergunta que exige a web**
> À pergunta "Qual foi o último fato relevante da Vale?", o sistema apresenta o fato relevante e avisa: "Contexto local insuficiente neste turno — a etapa Search buscou complemento na web". Ele fez a busca na base e, em seguida, a pesquisa na web.

Informar, nunca recomendar: a restrição regulatória

O chat exibe um aviso permanente: "Este assistente apresenta informações públicas, não recomendação de investimento". Esse aviso reflete uma regra do domínio, e não uma escolha de estilo.

> **ATENÇÃO · Recomendar investimento é atividade regulada**
> No Brasil, para dar recomendações de investimento, é preciso ter as certificações e ser registrado na CVM. Os youtubers sérios nunca recomendam: comentam informações, forecast e as ações. Para dizer "compre" ou "venda" uma ação, é preciso estar registrado junto à Bolsa de Valores e à CVM.

Por isso, as regras de negócio precisam estar também no chat. Um guardrail impede o sistema de responder a pedidos de compra ou venda. O ponto central é onde ele fica: o guardrail está no código. Mais do que no system prompt, coloquei essa proteção no próprio código, como a Aula 13 mostra em detalhe.

> **EXEMPLO · Pergunta que força uma recomendação**
> À pergunta "Eu devo comprar a Vale no preço atual?", o sistema faz a busca, mas bloqueia a recomendação. A resposta é "Não recomendamos decisões de compra, venda ou manutenção", com o aviso de que o contexto disponível pode estar incompleto para responder com certeza.

A ferramenta de desenvolvimento e o que realmente importa

O desenvolvimento do projeto é feito com o Claude, da Anthropic, uma ferramenta muito utilizada para desenvolvimento de software. Em outro curso, usei bastante as ferramentas do Google, e aqui a escolha foi outra para variar.

Você não precisa ficar preso ao Claude. O mesmo trabalho pode ser feito com o Antigravity, o Codex ou o Kimi. Ferramentas vão e vêm: quando você estiver estudando este material, pode haver outra ferramenta da moda.

> **EM SÍNTESE**
> O que permanece é o fundamento. Quem entende o processo e o fundamento consegue aplicá-los com qualquer ferramenta.

Público

Leitura visual: os três comportamentos do sistema

Pergunta	Como o sistema se comporta
"Qual é a receita líquida da Vale no último semestre?"	Busca na base de RAG e responde com esse contexto, sem inventar a informação e sem buscar na internet.
"Qual foi o último fato relevante da Vale?"	Indica que o contexto local é insuficiente, e a etapa Search busca complemento na web.
"Eu devo comprar a Vale no preço atual?"	Faz a busca, mas o guardrail implementado no código bloqueia a recomendação de compra, venda ou manutenção.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que é o AI Researcher e por que ele foi escolhido como projeto da disciplina sem consultar o material?
- Você consegue explicar o que o sistema faz quando o contexto encontrado na base vetorial não é suficiente sem consultar o material?
- Você consegue explicar por que a proteção contra recomendação de investimento fica no código, além do system prompt, sem consultar o material?

PRÁTICA RECOMENDADA

- Explique os pontos centrais desta aula com suas palavras, por escrito ou a um colega, sem consultar o material. Depois, confira o que ficou de fora.

RESUMO DA AULA

- A disciplina constrói um projeto do primeiro commit até a produção na nuvem, com observabilidade, FinOps e deploy automatizado pelo GitHub Actions.
- O AI Researcher busca primeiro na base de RAG; se o contexto não for suficiente, complementa com busca na web e mostra cada etapa no pipeline de raciocínio.
- Recomendar compra ou venda de ações exige certificação e registro na CVM, por isso o chat informa e não recomenda, com um guardrail colocado no código.
- O desenvolvimento usa o Claude, mas o que importa é o processo e o fundamento, que valem para qualquer ferramenta.

Público

Aula 2: CLAUDE.md e AI Dev Skills

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Preparar o ponto de partida do projeto: fazer o fork do repositório, clonar e trabalhar na branch `starter`.
- Explicar o papel do `CLAUDE.md` como arquivo de contexto e de regras de trabalho que o agente carrega em toda interação.
- Diferenciar o `CLAUDE.md` das skills e aplicar a regra de precedência entre a skill oficial de um framework e a skill do próprio projeto.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Um agente de código não lembra, de uma interação para a outra, como o projeto funciona nem quais regras deve seguir. Esta aula mostra como resolver isso com dois recursos: o arquivo `CLAUDE.md`, carregado em toda interação, e as skills, carregadas conforme a tarefa.

Antes disso, a aula prepara o ponto de partida do projeto. Ao final, você terá o repositório clonado na branch certa e saberá o que colocar no `CLAUDE.md`, o que deixar para as skills e qual skill vale quando duas tratam do mesmo assunto.

O ponto de partida: fork, clone e a branch starter

O projeto é feito de ponta a ponta, inclusive com a esteira de DevOps que entrega o código no Google Cloud. Por isso, você usa as suas próprias contas, tanto do GitHub quanto do Google Cloud, e precisa de uma cópia do repositório na sua conta.

1. Pegue o link do repositório do Capstone no material complementar da aula. O endereço pode mudar, e por isso o material complementar é a referência. O repositório é o Capstone Project AI Researcher.
2. Faça um fork do projeto, para tê-lo na sua conta do GitHub.
3. Clone o projeto a partir do seu fork, com `git clone`.
4. Entre na pasta do projeto e abra-a no seu editor. Uso o VS Code, mas qualquer ferramenta de desenvolvimento serve.

O repositório contém o projeto de fim a fim, separado em várias branches, para que o desenvolvimento possa ser consultado passo a passo. O trabalho acontece na branch `starter`, que tem o mínimo já configurado, para que o estudo se concentre nas partes mais importantes e mais teóricas do projeto.

> **CONCEITO-CHAVE · Projeto seed**
> É comum chamar de projeto seed um ponto de partida que traz só o início do que é necessário. A branch `starter` cumpre esse papel. A Aula 7 apresenta um repositório separado com esse mesmo conteúdo.

Muita coisa nesse repositório já está adiantada, e as aulas destacam as partes mais importantes. Como você tem acesso ao código completo, pode examinar cada detalhe ou pedir ao Claude que explique cada um deles.

Por que o agente precisa de um arquivo de contexto

O modelo com o qual um agente de codificação conversa é stateless: ele não lembra o contexto. Cada vez que você fala com ele, é como se fosse uma requisição nova aos modelos, no caso, os da Anthropic.

Público

A solução é um arquivo em que se configura como o agente deve trabalhar no projeto e quais são as suas diretrizes. No Claude, esse arquivo é o `CLAUDE.md`, que fica na pasta `.claude`. O Gemini e o Codex têm o equivalente: todas as ferramentas têm.

> **CONCEITO-CHAVE · CLAUDE.md**
> Arquivo que o Claude carrega em toda interação para saber onde está, qual é o contexto do projeto e quais regras deve seguir. É fundamental para quem trabalha com o Claude.

O que o CLAUDE.md deste projeto define

O arquivo começa pelo contexto: o repositório é um demonstrativo do módulo Capstone, e o código é didático. Ele descreve as partes do projeto e guarda também o estado atual do repositório. Depois vêm as regras de trabalho, divididas em dois grupos.

O primeiro grupo lista o que o agente deve perguntar antes de fazer:

- instalar qualquer dependência nova;
- criar uma abstração ou interface;
- alterar esquemas do banco de dados;
- mudar configurações de deploy, de workflow ou de infraestrutura;
- reorganizar pastas ou mover arquivos.

O segundo grupo lista o que o agente nunca deve fazer:

- escrever segredos, chaves ou connection strings em arquivo versionado: tudo vem de variável de ambiente e, em produção, do Secret Manager, o que coloca segurança e desenvolvimento seguro no projeto desde o zero;
- usar LLM para gerar os parágrafos de resumo de cada ação, na carga do RAG com os CSVs da CVM;
- adicionar `any` sem um comentário justificando: no TypeScript, quando não se sabe ou não se tem certeza do tipo, como no retorno de uma API, o preferível é `unknown` com narrowing;
- escrever teste que dependa de rede: usam-se fixtures ou fakes.

A regra dos testes tem um complemento: em todo desenvolvimento são criados testes automatizados, testes unitários, para validar o que está acontecendo. O arquivo termina com as convenções do projeto.

> **EXEMPLO · Por que a carga do RAG aparece nas regras**
> Os CSVs da CVM são carregados na base de dados que serve de primeira fonte do agente. Ele busca primeiro no RAG, que é uma informação muito mais rápida e mais barata do ponto de vista de tokens, e só depois, caso não encontre, vai para a internet.

CLAUDE.md global e CLAUDE.md do projeto

Segundo a documentação do Claude, existem dois níveis de `CLAUDE.md`: o da máquina, global ao ambiente, e o do projeto. Hoje, praticamente todos os agentes convergiram para esse modelo.

O que manda é o arquivo do projeto. Quando o projeto não tem alguma definição, o agente recorre ao arquivo global. Por isso, em uma equipe, é comum versionar o `CLAUDE.md` do projeto e compartilhá-lo, para que o agente siga sempre as mesmas regras.

Skills: instruções carregadas conforme o contexto

Skills não têm nada de mágico. São arquivos Markdown, com texto, que trazem instruções para o agente trabalhar segundo um padrão. O formato é parecido com o do `CLAUDE.md`, separado por estruturas, porque o Markdown é um dos jeitos mais fáceis de uma LLM trabalhar com regras.

> **CONCEITO-CHAVE · Skill**
> Arquivo Markdown com instruções para o agente trabalhar segundo um padrão. Ao contrário do `CLAUDE.md`, que entra sempre no contexto, a skill é carregada conforme o contexto da tarefa.

Público

Seria possível colocar no `CLAUDE.md` tudo sobre como programar o frontend em Angular, o backend e várias outras coisas. Isso não é feito porque o contexto inicial ficaria muito grande, o que gera custos de tokens mais altos e demora de processamento.

Com a skill, o contexto existe, mas só é carregado quando necessário. Em uma tarefa de backend, o Claude não carrega as skills do Angular, porque não há necessidade.



Diagrama editorial que organiza o carregamento do CLAUDE.md e das skills.

> **EXEMPLO · A skill do Fastify**
> O projeto tem uma skill do Fastify, o framework usado no backend. Ela só entra no contexto quando a tarefa envolve o backend.

Skills oficiais e a regra de precedência

O outro uso das skills é consumir as que foram criadas pelo próprio time de um framework, para que o agente de desenvolvimento, seja ele qual for, siga sempre a melhor prática. Muitos frameworks têm skills oficiais, e o próprio Claude tem várias.

O time do Angular criou skills oficiais, e fiz questão de usar o Angular neste projeto por causa disso. O projeto poderia ser feito com React sem problema nenhum.

Às vezes a comunidade consegue criar coisas melhores do que o próprio time do framework. É raro, mas acontece. Via de regra, quem conhece melhor o framework é a equipe que o desenvolve, e uma skill criada por ela quase sempre será melhor do que uma skill própria.

O projeto também tem uma skill sua, a `angular-component`, que descreve como os componentes são feitos aqui. Para o caso de as duas tratarem do mesmo assunto, o `CLAUDE.md` define uma precedência: se houver ambiguidade entre a skill do projeto e a oficial, a oficial sempre vence, porque foi feita pelo pessoal do próprio framework. É possível inverter a regra e dizer que a skill do projeto vence, mas a minha recomendação é que a oficial vença.

> **ATENÇÃO · Escreva as regras, mesmo as que parecem óbvias**
> Vale registrar as regras no arquivo de configuração mesmo quando a documentação do agente diz que ele já as segue. Assim, se a regra de negócio mudar, o agente sempre encontra a regra atual na configuração.

Público

Leitura visual: CLAUDE.md e skills

Critério	CLAUDE.md	Skills
O que é	Arquivo Markdown com o contexto do projeto, as regras de trabalho e as convenções.	Arquivos Markdown com instruções para o agente trabalhar segundo um padrão.
Quando entra no contexto	Sempre, em toda interação com o agente.	Conforme o contexto da tarefa: no backend, as skills do Angular não são carregadas.
Por que existe	O modelo é stateless e não lembra o contexto entre as interações.	Evitar um contexto inicial muito grande, com custo de tokens mais alto e processamento mais lento.
Origem	Versionado no projeto e compartilhado com a equipe; há também um arquivo global da máquina.	Do próprio projeto ou oficiais, criadas pelo time do framework; em caso de ambiguidade, a oficial vence.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar por que é preciso fazer um fork do repositório e em qual branch o trabalho acontece sem consultar o material?
- Você consegue explicar por que o agente precisa do `CLAUDE.md` e quais tipos de regra ele traz sem consultar o material?
- Você consegue explicar a diferença entre o `CLAUDE.md` e uma skill, e qual skill vence quando há ambiguidade, sem consultar o material?

PRÁTICA RECOMENDADA

- Faça um fork do repositório do curso na sua conta do GitHub, clone o projeto e abra a branch `starter`. Depois, leia o `CLAUDE.md` e as skills do repositório e, se quiser, peça ao Claude para explicar cada detalhe.

RESUMO DA AULA

- O trabalho começa por um fork do repositório do curso, clonado e aberto na branch `starter`, que traz o mínimo já configurado.
- O `CLAUDE.md` é carregado em toda interação e guarda o contexto, o que o agente deve perguntar antes de fazer e o que ele nunca deve fazer.
- Skills são arquivos Markdown carregados conforme o contexto, o que evita inflar o contexto inicial e o custo de tokens.
- Quando existe uma skill oficial do framework, ela vence a skill do projeto em caso de ambiguidade.

Público

Aula 3: Tour pelo Google Cloud

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Identificar as peças do Google Cloud que sustentam o projeto e o papel de cada uma na arquitetura.
- Explicar por que o projeto usa um backend serverless no Cloud Run e o Postgres com a extensão pgvector no Cloud SQL.
- Relacionar cada serviço do GCP ao equivalente que existe em outros provedores de nuvem.

Visão geral

Antes de pedir a um agente que provisione qualquer peça da arquitetura, é preciso entender o que será provisionado. Esta aula percorre, sem terminal e sem commit, os serviços do Google Cloud que o projeto usa e explica o papel de cada um.

A escolha do Google Cloud não é uma obrigação. No seu projeto, você pode usar qualquer solução de nuvem, ou até montar uma VM com essas peças. Todas elas existem em outros provedores, como AWS e Azure. A exceção é o Vertex AI, que é um serviço bem específico do Google, mas os outros provedores também têm o seu garden de modelos, talvez com outros modelos, não necessariamente com o Gemini.

O mapa da arquitetura

Cada peça da arquitetura tem um papel definido:

- **GitHub Actions**: a estrutura de DevOps do GitHub. Todas as ações na infraestrutura são disparadas por ele. A esteira passa pelas guardas de qualidade do código e, no final, faz o deploy no Google Cloud.
- **Cloud Run**: a peça principal, onde é executada toda a regra de negócio, a API, o backend. É o serviço de backend serverless do Google.
- **Firebase Hosting**: armazena a interface e os arquivos estáticos, como a interface feita em Angular.
- **Artifact Registry**: armazena a imagem Docker que o Cloud Run executa, e que também pode ser executada localmente no desenvolvimento.
- **Cloud SQL**: o banco de dados Postgres, com a extensão pgvector, onde ficam os dados, principalmente os do RAG.
- **Vertex AI**: o serviço de API em que o modelo é consumido.
- **Secret Manager**: o cofre de secrets e senhas.

O modelo cumpre dois papéis. Ele faz o embedding, transformando o texto do RAG em embeddings para serem guardados como vetores no Cloud SQL, e é também a LLM que o agente utiliza para fazer a busca e toda a orquestração.

Público

Mapa da arquitetura no Google Cloud

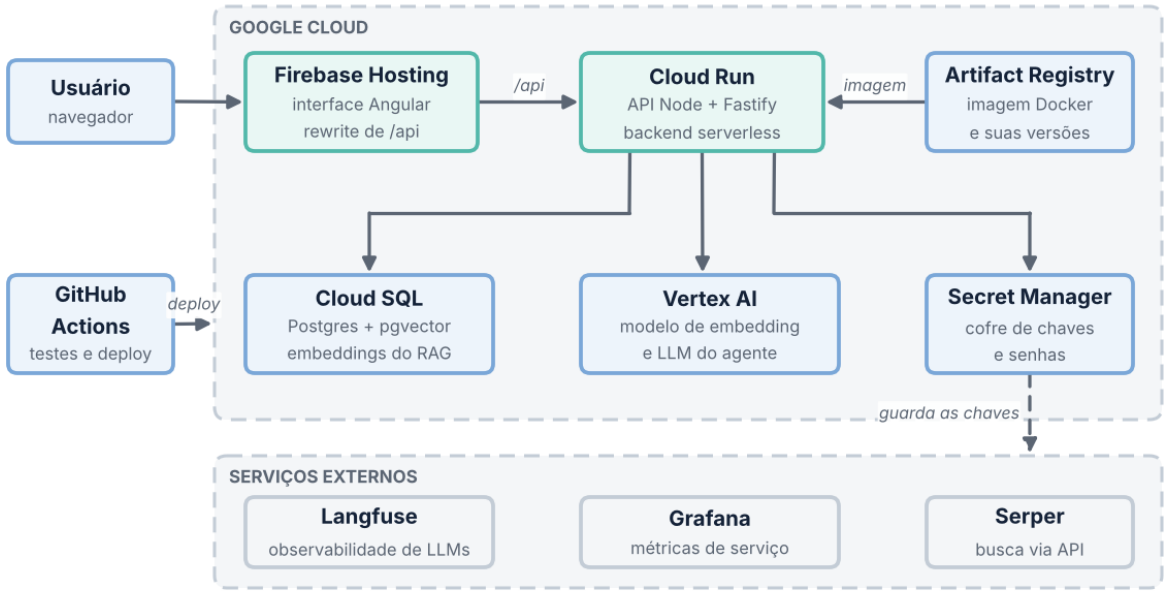


Diagrama editorial com as peças e as ligações descritas na aula.

Cloud Run: o backend serverless

O serviço que roda no Cloud Run é a API do AI Researcher, um backend em Node com o framework Fastify, escolhido porque se trata de uma API relativamente simples.

> **CONCEITO-CHAVE · Serverless**
> O próprio GCP sobe os containers, serviços dedicados exclusivamente a atender às requisições, e faz isso sob demanda, sem nenhuma configuração manual. Enquanto ninguém acessa, não há nenhum servidor de backend ligado. O serviço é executado somente quando necessário e pode escalar infinitamente.

Na AWS, o equivalente são as Lambda Functions. A Azure também tem um serviço serverless equivalente.

Escolhi essa peça por dois motivos: quis manter o projeto o mais barato possível, e ela pode ser útil no seu dia a dia. O Cloud Run, como os outros serviços do tipo Lambda, é extremamente barato, porque é cobrado por requisição. Na tabela de preços, o valor por gigabyte de memória por segundo tem vários zeros depois da vírgula.

Se a aplicação fica no ar 24 horas por dia, 7 dias por semana, com muita gente fazendo requisições, às vezes vale a pena criar um servidor dedicado e deixá-lo ligado. Em regras de negócio com consultas esporádicas, o serverless deixa o processamento basicamente gratuito.

Cloud SQL: o banco de dados e o seu custo

Os dados ficam no Cloud SQL. O Google tem serviços de banco de dados mais robustos, como o Spanner, mas o projeto só precisa de um pequeno banco Postgres. Ele foi criado na menor instância possível, com 10 GB de armazenamento.

Quem nunca teve conta no GCP ganha créditos ao criá-la, em torno de 300 dólares, dependendo da época. Com eles, toda essa infraestrutura fica basicamente gratuita. No meu caso, como tenho conta há bastante tempo, não tenho crédito em alguns desses serviços. Mesmo assim, com a aplicação inteira rodando por mais de um mês, o gasto foi de R$ 30.

> **ATENÇÃO · O Cloud SQL cobra mesmo sem uso**
> De todos os serviços do projeto, o que continua cobrando é o Cloud SQL, porque a instância é cobrada por tamanho. Usando ou não o banco, os 10 GB estão reservados. Por isso, a infraestrutura deve ser desmontada quando o projeto termina, para não ficar consumindo recursos.

Público

O Cloud SQL permite subir Postgres, MySQL e, ao que parece, também Oracle e SQL Server. O projeto usa o Postgres por causa das extensões. Ele é um banco relacional normal, com tabelas, colunas e índices, mas a comunidade criou várias extensões que permitem usá-lo em diversas aplicações.

pgvector: o Postgres como banco de dados vetorial

A extensão usada no projeto é a pgvector. Ela transforma o Postgres em um banco de dados vetorial e permite fazer buscas vetoriais com a linguagem SQL.

> **CONCEITO-CHAVE · Banco de dados vetorial**
> Banco que guarda embeddings, isto é, o texto em forma numérica, do mesmo jeito que uma LLM processa informação: matrizes de números. Com eles, é possível buscar por semântica, por proximidade de contexto.

> **EXEMPLO · "Casa" e "lar"**
> Em uma busca por palavra, "lar" e "casa" não são a mesma coisa. Semanticamente, porém, são termos muito próximos. Uma busca por "eu quero ver minha casa" pode devolver "este é o seu lar".

Existem bancos de dados vetoriais específicos, nativos, oferecidos como serviço. O Postgres, porém, é extremamente versátil e pode ser uma opção válida na sua arquitetura.

Uma aplicação que já roda Postgres e precisa de um RAG, por exemplo para um chatbot de atendimento, não precisa de outro banco de dados: pode usar o próprio Postgres. Dá para criar outra instância, mas o ganho da extensão é aproveitar todo o conhecimento que a equipe já tem do Postgres.

Secret Manager: o cofre de segredos

Tudo o que foi descrito precisa de secrets e senhas. Por segurança, tanto no ambiente local quanto na nuvem, eles não ficam em nenhum local estático do código. Na nuvem, ficam no Secret Manager, um cofre seguro: toda vez que um serviço precisa de uma senha, ele a busca ali e a consome. No ambiente local, as informações sensíveis ficam em variáveis de ambiente.

O cofre guarda as chaves dos serviços internos, como a do banco de dados, e as dos serviços externos ao GCP:

- o Langfuse, serviço de observabilidade de LLMs, que precisa de API keys e de secret;
- o exportador do OpenTelemetry, usado para enviar ao Grafana as métricas de serviço, a observabilidade de serviço;
- o Serper, serviço de busca de informações via API.

O termo vault, cofre, aparece com frequência para esse tipo de serviço. É o jeito seguro de guardar secrets, e pensar em segurança da informação é parte do projeto.

Artifact Registry: as imagens Docker

O Artifact Registry guarda a imagem Docker criada para o serviço de backend. Ele guarda também as versões: a cada imagem nova, a anterior é versionada. O Artifact Registry só armazena as imagens. Quem as executa é o Cloud Run.

Firebase Hosting: o frontend e a regra de rewrite

Todo o frontend é servido pelo Firebase Hosting. Seria possível usar um Cloud Run com a imagem de um nginx, um servidor web, configurar o firewall para expor o endpoint à internet e colocar um DNS na frente. O Firebase simplifica tudo isso: basta subir o código Angular para ter um servidor web escalável, já preparado.

O Firebase faz outras coisas, mas é muito bom para servir páginas web. Ele também fornece um DNS. No projeto, o nome é genérico, mas é possível comprar um endereço web e configurá-lo ali com facilidade.

Em outras nuvens, o papel é cumprido por um bucket S3 exposto, na AWS, ou pelo Blob Storage, na Azure. Cada serviço tem a sua facilidade, mas todos fazem basicamente a mesma coisa.

Público

CONCEITO-CHAVE · Rewrite

Configuração do Firebase que redireciona automaticamente para o Cloud Run toda requisição da interface cujo caminho tem `/api`. O que não é `/api` é servido pela interface Angular. Com isso, não é preciso configurar regras de CORS no backend e no frontend.

Vertex AI e o Model Garden

O Vertex AI é o serviço em que se executam diversos modelos de LLM, não somente os do Gemini. No Model Garden, é possível testar vários tipos de modelo e ver o custo de cada um:

- modelos do Google, como o Flash, o Flash-Lite e o Gemini Pro, além do Gemma;
- modelos de parceiros, como os da Anthropic;
- modelos abertos, como o Kimi, o Llama, da Meta, e os do Hugging Face. Todos os modelos abertos que estão no Hugging Face podem ser executados ali.

No Vertex AI, você configura o modelo que quiser usar na sua aplicação. Na AWS, o Bedrock faz basicamente a mesma coisa, e quem tem a infraestrutura na AWS tem mais facilidade em usá-lo. A Azure também parece ter algo equivalente.

EM SÍNTESE

O AI Researcher roda no GCP com esses serviços, mas o GCP não é o único provedor nem necessariamente o melhor: depende do caso. Azure, AWS e Oracle têm equivalentes. O que importa são os conceitos.

Leitura visual: as peças e os seus equivalentes

Peça	Papel no projeto	Equivalente citado na aula
GitHub Actions	Esteira de DevOps: passa pelas guardas de qualidade e faz o deploy no Google Cloud.	—
Cloud Run	Backend serverless que executa a API e a regra de negócio, sob demanda.	Lambda Functions (AWS); serviço serverless da Azure
Firebase Hosting	Serve a interface Angular e redireciona `/api` para o Cloud Run com a regra de rewrite.	Bucket S3 (AWS); Blob Storage (Azure)
Artifact Registry	Guarda e versiona a imagem Docker do backend.	—
Cloud SQL com pgvector	Banco Postgres que guarda os embeddings do RAG e faz busca vetorial com SQL.	—
Vertex AI	Executa o modelo de embedding e a LLM usada pelo agente.	Bedrock (AWS)
Secret Manager	Cofre das chaves dos serviços internos e externos (banco, Langfuse, Grafana, Serper).	—

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o papel de cada peça do Google Cloud na arquitetura do projeto sem consultar o material?
- Você consegue explicar por que o Cloud Run é barato para este projeto e o que a extensão pgvector acrescenta ao Postgres sem consultar o material?
- Você consegue explicar quais serviços de outras nuvens fazem o papel do Cloud Run, do Firebase Hosting e do Vertex AI sem consultar o material?

PRÁTICA RECOMENDADA

- Explique os pontos centrais desta aula com suas palavras, por escrito ou a um colega, sem consultar o material. Depois, confira o que ficou de fora.

RESUMO DA AULA

- A arquitetura usa Cloud Run para a API, Firebase Hosting para o Angular, Artifact Registry para a imagem Docker, Cloud SQL com pgvector para os dados, Vertex AI para os modelos e Secret Manager para os segredos.
- O Cloud Run é serverless e cobrado por requisição; o serviço que continua cobrando mesmo sem uso é o Cloud SQL, pela instância reservada.
- A pgvector transforma o Postgres em banco de dados vetorial e permite aproveitar o conhecimento que a equipe já tem do Postgres.
- Todas essas peças têm equivalentes em outras nuvens: o que importa são os conceitos.

Público

Aula 4: GitHub e GitHub Actions

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Explicar o papel do GitHub Actions como a peça que faz build, teste e deploy a cada push.
- Comparar a autenticação por chave estática guardada em secret com a autenticação por workload identity pool, de chaves efêmeras.
- Reconhecer a estrutura de um arquivo YAML do GitHub Actions: gatilho, jobs, máquina de execução e steps.

Visão geral

Falta uma peça para fechar o mapa da arquitetura: a que faz o build do código, executa os testes automatizados, passa por todos os quality gates e envia o código aos serviços do Google Cloud, sem comandos manuais a cada push. Essa peça é o GitHub Actions.

Esta aula explica como uma esteira do GitHub Actions é organizada e como ela se autentica na nuvem. Ao final, você saberá ler um workflow e explicar por que o projeto não guarda nenhuma chave de acesso no repositório.

Dois jeitos de dar à esteira acesso à nuvem

As primeiras etapas da esteira, o build e os testes, não precisam de acesso ao Google Cloud. O deploy precisa. Há dois jeitos de dar esse acesso.

O primeiro é o modelo padrão: criar uma chave no Google Cloud, sempre com as permissões mínimas necessárias para o deploy, e guardar esse JSON na área de secrets do repositório no GitHub. A cada execução, a esteira usa essa chave.

> **ATENÇÃO · Nunca versione um secret**
> Um secret nunca vai para o código versionado. O GitHub tem uma área própria para os secrets do repositório.

Esse modelo funciona, mas tem um risco. Se o repositório for comprometido de alguma forma, o secret pode ser usado de maneira mal-intencionada. Mesmo com a boa prática de dar o mínimo de permissão, o risco continua existindo.

O segundo jeito, adotado no projeto como opção de segurança, é a Workload Identity Federation (WIF), com um workload identity pool.

Workload identity pool: chaves que existem só durante o deploy

Com o workload identity pool, a esteira não guarda chave nenhuma. A autenticação acontece a cada execução, em quatro passos:

1. O runner do GitHub Actions, ao executar, faz um login no próprio GitHub, baseado em quem está fazendo o push, e recebe um token OIDC de curtíssima duração. A cada execução, um novo token é gerado.
2. Com esse token, ele faz o login no GCP, previamente configurado. O login confirma duas coisas: que quem chama é alguém que a nuvem conhece e que a chamada vem de um repositório que o GCP conhece.
3. Nesse momento é criado um token impersonado, que representa uma service account de deploy. Ele é criado no ato e faz toda a execução necessária.
4. Quando o deploy acaba, tudo isso é apagado.

Público

Autenticação com chaves efêmeras



Fluxo editorial que organiza as etapas descritas na aula.

> **EM SÍNTESE**
> Todas as chaves são efêmeras: fazem o seu papel e somem. Isso é mais seguro do que manter um JSON estático no secret.

Nas configurações gerais de Actions do GitHub existe, por padrão, o `GITHUB_TOKEN`. Esse token não é da nuvem, é do próprio GitHub. Ele identifica quem está executando aquela run, conforme a permissão que a pessoa tem no repositório.

A tabela a seguir compara os dois modelos de autenticação:

Critério	Chave JSON em secret	Workload identity pool
Como autentica	Uma chave criada no Google Cloud fica guardada na área de secrets do GitHub.	O runner troca um token OIDC de curtíssima duração por um token impersonado da service account de deploy.
Duração da credencial	Estática: fica guardada enquanto não for removida.	Efêmera: é criada no ato e apagada quando o deploy termina.
Risco	Se o repositório for comprometido, o secret pode ser usado de maneira mal-intencionada.	Não sobra nenhuma chave, e o acesso é condicionado a um repositório específico.

A anatomia de um workflow

Toda esteira do GitHub Actions é um arquivo YAML com a mesma estrutura:

- **nome**: o nome da ação;
- **on**: o gatilho, que define quando a ação é executada. Pode ser um push, a criação de um PR, de uma issue ou de um fork. Várias ações dentro do ecossistema do GitHub podem disparar uma esteira;
- **jobs**: o que é executado a cada gatilho. Pode haver um ou mais jobs. A esteira do projeto tem o job de teste e o de deploy;
- **steps**: os comandos que cada job executa. O job de deploy tem vários.

Anatomia de um workflow do GitHub Actions



Diagrama editorial com os elementos de um workflow descritos na aula.

Público

Onde a esteira é executada

O GitHub sobe uma máquina, na verdade um container, e executa toda a esteira nesse container. Quando a run acaba, o container é desmontado, e o GitHub espera a próxima execução.

A imagem da máquina é definida no próprio YAML. Por padrão, é uma máquina Linux, a `ubuntu-latest`. Também há máquinas Windows e Mac, mas normalmente são pagas. De graça, é basicamente só a Linux.

O sistema operacional é definido por job. Como essa chave fica abaixo do job, um job pode executar em Linux e outro, por exemplo o de teste, em Windows.

> **EXEMPLO · Jobs em máquinas diferentes**
> Em aplicações para o ecossistema da Apple, como as de iPhone, é normal fazer o build em uma máquina Mac e os testes em uma máquina Linux. Neste projeto, que é uma aplicação web, tudo roda em Linux.

Steps e actions prontas

Cada step executa um comando. Como a máquina é Linux, um step pode rodar comandos do próprio sistema, como um `echo` que aparece na linha de comando. Ele também pode usar ações já criadas pela equipe do GitHub ou por terceiros.

A action mais usada é a `checkout`, que faz o checkout do código-fonte. Ela é necessária porque a máquina nasce nova a cada execução e precisa receber o fonte. Daria para escrever um step com o `git clone` e todo o resto, mas as actions da equipe do GitHub e as da comunidade facilitam o trabalho e deixam o YAML mais simples.

O lado do GCP: o pool de identidade

No GCP, o pool fica na parte de IAM, em federação de identidade. O console em português traduz workload pool como pool de identidade. No projeto, o pool se chama GitHub Actions, e é ele que guarda as definições usadas na autenticação.

A tela do pool traz três informações úteis:

- quando o pool foi usado: cada uso é registrado para auditoria, e o próprio Google Cloud avisa se houver algum tipo de abuso;
- as asserções usadas na autenticação;
- a condição: o acesso só vale para um repositório específico.

> **ATENÇÃO · A condição aponta para o seu repositório**
> Quem trabalha em um fork do repositório do curso precisa que a condição do pool aponte para o seu próprio fork.

Na Aula 6, esse pool é criado com o apoio do agente de código, e não só gerando comandos: usando, conferindo, verificando e testando cada etapa.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que o GitHub Actions faz a cada push e por que só a etapa de deploy precisa de acesso ao Google Cloud sem consultar o material?
- Você consegue explicar por que a autenticação com workload identity pool é mais segura do que guardar uma chave JSON nos secrets sem consultar o material?
- Você consegue explicar o que são o gatilho, os jobs, a máquina de execução e os steps de um workflow sem consultar o material?

PRÁTICA RECOMENDADA

- Escolha um conceito desta aula e aplique-o em um exemplo diferente do apresentado. Registre o que mudou, o que permaneceu igual e qual foi o resultado.

RESUMO DA AULA

- O GitHub Actions faz build, testes, quality gates e deploy no Google Cloud a cada push, sem comandos manuais.
- Em vez de uma chave estática nos secrets, o projeto usa workload identity pool: um token OIDC curto é trocado por um token impersonado da service account de deploy, e tudo some ao final.
- Um workflow é um arquivo YAML com um gatilho (`on`), um ou mais jobs, a máquina em que cada job roda e os steps, que podem usar actions prontas como a `checkout`.
- No GCP, o pool de identidade registra cada uso para auditoria e condiciona o acesso a um repositório específico, que no caso do aluno é o seu fork.

Público

Aula 5: Corpus CVM

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Explicar por que os dados do sistema vêm da CVM e por que os parágrafos do corpus nunca são gerados por LLM.
- Descrever o que o script de coleta faz: baixar os arquivos da CVM, normalizar os valores e gerar um JSON por empresa.
- Reconhecer como uma situação não prevista na fonte de dados é tratada com um novo prompt e uma nova execução.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Antes de embedding, de busca vetorial e de qualquer LLM, o sistema precisa de dados. A fonte usada no projeto é a CVM, a Comissão de Valores Mobiliários, o órgão regulador do mercado de capitais.

Esta aula constrói o script que baixa as informações do site da CVM, normaliza o que estiver fora de padrão e gera um JSON limpo, pronto para ser carregado no banco de dados vetorial e usado nas buscas do RAG. Ao final, você saberá o que esse script faz, por que ele não usa LLM e como tratar um erro que vem da fonte de dados, e não do código.

A fonte: os dados abertos da CVM

O Portal de Dados Abertos da CVM disponibiliza diversos arquivos de todas as companhias de capital aberto. Os dados vêm em arquivos ZIP, e cada ZIP contém vários arquivos, um para cada tipo de informação financeira publicada pelas empresas. Cada arquivo é um CSV, ou seja, um formato padrão de dados.

Para alimentar o RAG, esses arquivos precisam ser lidos e baixados automaticamente, sem download manual a cada vez, depois limpos e convertidos em JSON.

Por que os parágrafos do corpus não são gerados por LLM

Uma das regras mais importantes do `CLAUDE.md`, na parte do "nunca", é nunca usar LLM para gerar os parágrafos do corpus, ou seja, os parágrafos que descrevem cada empresa. O agente não pode inventar nem alucinar números: tudo precisa ser baseado em dados.

Por isso, o texto de cada trimestre é montado por um template. Todos os parágrafos, de todas as empresas, são gerados da mesma forma, concatenando strings: o template faz as comparações entre períodos e coloca no texto o percentual correspondente, com uma cláusula para cada tipo de comparação.

> **CONCEITO-CHAVE · Template determinístico**
> Parágrafo montado por regras fixas a partir dos dados. Todos são gerados da mesma forma, sem LLM, para não haver risco de alucinação.

Seria possível pedir a uma LLM que gerasse o texto a partir dos dados. Há dois motivos para não fazer isso:

- **Custo**: o script precisaria de uma LLM a cada execução, o que o encareceria. Com o template, ele pode ser executado um milhão de vezes sem nenhuma chamada de LLM.
- **Alucinação**: mesmo recebendo todos os valores certos, uma LLM pode alucinar. O template fixo, de comparações e regras, evita esse risco.

Público

> **EM SÍNTESE**
> É preciso saber dosar. Onde o dado não pode ter risco de alucinação, o caminho é o código determinístico, e não a LLM.

O prompt que cria o pacote de coleta

O pacote de coleta é criado com um único prompt:

Prompt enviado ao Claude Code

```
Crie o pacote packages/corpus. Ele lê a lista de 20 empresas da B3 em data/empresas.csv (ticker, razão social, código CVM, setor), baixa os arquivos ITR e DFP mais recentes direto de dados.cvm.gov.br, e extrai de cada um dos últimos 4 trimestres: receita líquida, lucro bruto, lucro líquido, ativo total, patrimônio líquido e caixa operacional. A CVM publica em milhares de reais — normalize para reais. Gere um arquivo JSON por empresa em data/documentos/{TICKER}.json, um item por trimestre, cada um com um parágrafo de texto gerado por template determinístico — nunca por LLM. Cada documento precisa ter ticker, company_name, setor, period, period_end, source_url e fetched_at.
```

O prompt define cinco coisas:

- **A lista de empresas**: 20 empresas da B3, com ticker, razão social, código CVM e setor.
- **A fonte**: os arquivos ITR e DFP, que são os arquivos de informações financeiras, baixados direto do site de dados da CVM, com os dados dos quatro últimos trimestres.
- **A normalização**: a CVM publica os valores em milhares de reais, e o prompt pede a conversão para reais.
- **A saída**: um arquivo JSON por empresa, com um item por trimestre, cada um com um parágrafo gerado por template determinístico, nunca por LLM.
- **Os metadados**: ticker, nome da companhia, setor, período, data final do período, URL da fonte e data em que o dado foi baixado.

> **EXEMPLO · O que significa "milhares de reais"**
> Nos arquivos da CVM, um valor aparece como 100 ou 200. Não são 200 mil reais: são 200 mil milhares, ou seja, 200 milhões. Guardar o valor bruto, em reais, permite fazer contas com ele depois.

Na branch `starter`, o pacote já existe. Ao receber o prompt, o agente confirma as 20 empresas e roda os testes para ver se passam. Isso acontece porque o `CLAUDE.md` trata de qualidade de código: para todo requisito, deve ser criado um teste que comprove a regra de negócio.

O que o código precisou resolver

A coleta é determinística, mas, para escrever esse código, o agente leu os arquivos baixados e encontrou inconsistências. Na teoria, os dados estariam normalizados no CSV. Na prática, não estão.

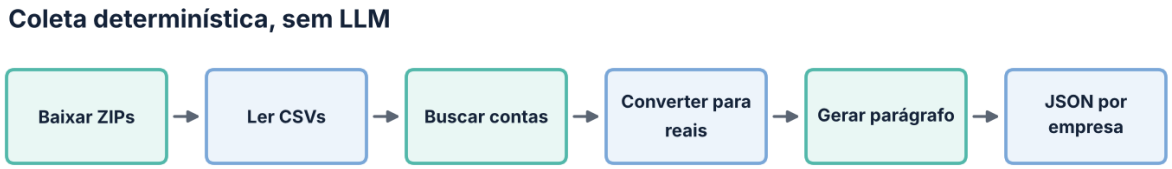
> **ATENÇÃO · O código de conta não é estável entre os setores**
> Os bancos usam uma árvore de DRE e de balanço patrimonial diferente da árvore da indústria, o que foi confirmado com Vale, Itaú, Bradesco e Banco do Brasil. E o próprio Itaú tem menos linhas antes do lucro líquido do que o Bradesco: dentro do mesmo setor, o padrão de contas não bate.

O arquivo `select-quarters.ts` trata disso. A busca das contas é feita pelo nome, e não pela quantidade de linhas, porque a coluna de descrição bate entre as empresas e o código da conta não. Na função de busca da conta, o filtro usa o código de conta definido no início do arquivo.

Outros dois pontos do código merecem atenção:

- **Conversão para reais**: uma função verifica em que escala estão os valores e, no caso da CVM, multiplica por 1.000, para que o JSON guarde o valor cheio, e não o abreviado.
- **Template do parágrafo**: o arquivo de template gera o parágrafo determinístico de cada trimestre, com as comparações descritas acima.

Público



Fluxo editorial que organiza as etapas do script descritas na aula.

Executando a coleta e tratando um erro não previsto

A coleta é executada com o comando `pnpm collect`. Ele vai à CVM, baixa os dados, descompacta os arquivos e faz todo o tratamento.

Na demonstração, a coleta falhou: erro 404 ao baixar os dados das companhias abertas. O arquivo de 2025 não estava no site. A CVM tinha os arquivos de 2013 a 2024 e o de 2026, mas não o de 2025.

> **ATENÇÃO · Nem todo erro é erro do código**
> O script estava correto. Quando ele foi criado, o site da CVM tinha todos os arquivos. A falha veio de uma situação não prevista na fonte de dados, e o tratamento para ela não tinha sido pensado.

A correção foi pedida ao agente com um novo prompt, informando que o site da CVM não tinha o dado de 2025, que isso causou um erro no script de coleta e que ele deveria ignorar o arquivo quando ele não existisse e seguir com os dados disponíveis. Optei pela correção mais simples: ignorar o que falta e trabalhar com o que existe. Com o ajuste feito, a segunda execução gerou os arquivos.

O JSON gerado

O resultado fica na pasta `data` do pacote `packages/corpus`, em um formato adequado para a importação no banco de dados. Cada documento traz:

- os valores inteiros, ao contrário das demonstrações, que cortam os zeros;
- a data em que a informação foi baixada;
- o parágrafo, sempre no mesmo modelo: no trimestre tal, a companhia reportou receita líquida de tanto, e assim por diante.

O parágrafo não foi gerado com LLM. Ele foi feito de maneira determinística, já com o tratamento dos dados aplicado. Para ler o que foi coletado, o pacote tem também um script de leitura, executado com o `--filter` do pnpm.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar por que a CVM é a fonte dos dados e por que os parágrafos do corpus não podem ser gerados por LLM sem consultar o material?
- Você consegue explicar o que o script de coleta faz com os arquivos da CVM, da busca das contas à normalização para reais, sem consultar o material?
- Você consegue explicar como o erro 404 do arquivo de 2025 foi tratado sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O corpus do RAG vem dos dados abertos da CVM, baixados e tratados por um script, sem nenhuma chamada de LLM.
- O código de conta da CVM não é estável entre os setores, por isso o script precisou de tratamento próprio para localizar as contas.
- Os valores publicados em milhares de reais são convertidos para reais, e o parágrafo de cada trimestre é gerado por um template fixo, o que evita custo e alucinação.
- Quando o arquivo de 2025 não foi encontrado na CVM, um novo prompt pediu para ignorar arquivos inexistentes e seguir com os dados disponíveis.

Público

Aula 6: Fundação GCP

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Provisionar a fundação do projeto no Google Cloud com prompts ao Claude Code: projeto, APIs, Artifact Registry, Cloud SQL, Secret Manager, Workload Identity Federation e Firebase Hosting.
- Explicar os conceitos de conta de faturamento, projeto e mínima permissão no GCP.
- Aplicar os cuidados com secrets: nunca gravá-los em arquivo, gerar senhas aleatórias e invalidar imediatamente qualquer credencial vazada.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Esta aula cria a infraestrutura do projeto no Google Cloud usando o Claude Code e o SDK do GCP. O agente propõe e executa os comandos, um por vez, e você aprova cada um.

O mesmo caminho serve para AWS ou Azure. Talvez algum prompt precise mudar, mas o jeito é muito parecido, porque todos os grandes provedores de nuvem têm o seu SDK, e isso existe muito antes das IAs.

O SDK é uma parte fundamental do que se chama IaC, Infrastructure as Code.

Ao final, você terá provisionado a fundação do projeto e saberá explicar a organização do GCP em contas de faturamento e projetos, o princípio da mínima permissão e os cuidados com secrets.

Como o GCP se organiza: conta de faturamento e projetos

Em alto nível, o GCP trabalha com dois conceitos: a billing account, ou conta de faturamento, no console em português, e os projetos.

> **CONCEITO-CHAVE · Conta de faturamento e projeto**
> Uma conta de faturamento pode ter N projetos. Cada projeto reúne uma infraestrutura, que fica isolada da infraestrutura dos outros projetos.

Uma conta pessoal normalmente tem uma só conta de faturamento. Empresas muito grandes podem ter mais de uma, conforme a organização.

O isolamento entre projetos é quase total. Alguns recursos são compartilhados, mas a maioria não: por padrão, o banco de dados de um projeto não enxerga o de outro. É possível ligar projetos por rede, mas o efeito é o de duas instâncias separadas de nuvem.

Mínima permissão: as APIs começam desligadas

Por segurança, o GCP trabalha com a mínima permissão necessária. Um projeto recém-criado nasce sem permissão para nenhum SDK fazer absolutamente nada. As APIs são habilitadas conforme a necessidade do projeto.

Por isso, o primeiro prompt do provisionamento habilita as APIs, e só as que o projeto vai utilizar.

Antes de começar: dois logins e o tempo do banco

Antes de abrir o agente de código, é preciso instalar o `gcloud`, que é o SDK, o CLI do Google Cloud, e fazer dois logins com a sua conta:

1. O login do CLI.

Público

2. O login do SDK, feito com outro comando. Depois dele, o terminal informa que as bibliotecas do Google já podem usar as credenciais.

Ter dois logins é uma característica do Google Cloud. Em AWS, Azure ou Oracle, talvez seja um só.

> **ATENÇÃO · O banco de dados é o passo mais demorado**
> O provisionamento do Cloud SQL leva de 20 minutos a meia hora. De tudo o que é criado aqui, é de longe o que mais demora. O resto é rápido.

O provisionamento, prompt a prompt

O provisionamento segue sete passos, cada um com um prompt. Em todos, o agente lê o `infra/README.md`, que descreve a arquitetura, e usa o `gcloud` para interagir com o Google. Em outra nuvem, seria o Azure CLI ou o AWS CLI.



Fluxo editorial com a ordem em que os recursos são criados.

1. Crie o projeto, vincule a conta de faturamento e habilite as APIs:

Prompt enviado ao Claude Code

```
Seguindo o infra/README.md, crie o projeto GCP ai-researcher-capstone, vincule à billing account e habilite as APIs necessárias: Cloud Run, Cloud SQL Admin, Artifact Registry, Secret Manager e IAM Credentials. Rode um comando por vez e espere minha aprovação antes de cada execução.
```

Ao criar o projeto, o agente mostra o número dele. Cada projeto tem o seu número. Essa informação é específica do projeto, mas não é sensível: não é um secret.

2. Crie o repositório de imagens Docker:

Prompt enviado ao Claude Code

```
Crie o repositório Docker do Artifact Registry pra este projeto, seguindo o infra/README.md.
```

O repositório recebe as imagens Docker do projeto. A região usada é `us-central1`, escolhida por custo: é a mais barata. Você pode alterar a sua.

3. Provisione o banco de dados:

Prompt enviado ao Claude Code

```
Provisione o Cloud SQL Postgres 16 com pgvector, seguindo o infra/README.md: instância ai-researcher-db, tier db-f1-micro, região us-central1. Depois crie o banco ai_researcher, habilite a extensão vector, e crie o usuário app_user com senha gerada de forma segura.
```

O prompt informa o nome da instância, o tier e a região. A ordem é: criar a instância, criar o banco de dados, habilitar a extensão `vector` e criar o usuário, com a senha gerada de forma segura.

Enquanto a instância é criada, o agente avisa que vai ficar acompanhando. Você também pode acompanhar pelo console: em bancos de dados aparecem o Cloud SQL e os outros produtos do Google, como AlloyDB, Spanner, Bigtable, Firestore e Memorystore.

Público

4. Guarde a connection string no Secret Manager:

Prompt enviado ao Claude Code

```
Monte a connection string do app_user no formato de socket Unix que o Cloud Run usa em produção, e salve no Secret Manager como database-url — sem escrever em nenhum arquivo local.
```

Esse passo prepara o backend para se conectar ao banco. A string de conexão vai direto para o Secret Manager, sem passar por nenhum arquivo local. O `CLAUDE.md` já proíbe escrever secret em arquivo, e mesmo assim reforcei a regra no prompt.

Se o secret já existir, o agente avisa que a senha antiga continua funcionando até o código em produção reler o secret novo e pergunta se deve resetar ou cancelar, isto é, manter o que já está lá.

5. Crie a Workload Identity Federation:

Prompt enviado ao Claude Code

```
Crie a Workload Identity Federation pra este repositório GitHub, seguindo o infra/README.md: pool, provider restrito por attribute.repository a unipds-engenharia-de-ia-aplicada/capstone-project-ai-researcher, a service account github-actions-deployer com as roles listadas no README, e o binding que autoriza o WIF a se passar por ela.
```

É a conexão explicada na Aula 4. O agente cria o pool para o repositório do GitHub, as roles e toda a configuração necessária.

> **ATENÇÃO · Troque o repositório pelo seu**
> O prompt aponta para o repositório do curso. Você consegue cloná-lo, mas, como não é o dono dele, o mecanismo de autenticação da WIF não funciona, e o deploy na sua nuvem dá erro. Coloque no prompt o nome do seu repositório: o seu usuário, barra, o nome do Capstone, se for um fork direto, ou o nome do repositório que você criou.

Criar esse pool na mão não é simples. Há vários detalhes e permissões para acertar, e isso dá trabalho. Vale usar o agente, com um bom modelo, para essa etapa.

6. Tire os identificadores do código e leve-os para variáveis:

Prompt enviado ao Claude Code

```
O projeto, a Workload Identity Federation e a service account de deploy existem agora. Configure o que falta pra nenhuma dessas informações ficar fixa em código: exporte GOOGLE_CLOUD_PROJECT_ID no meu ~/.zshrc, e crie as variáveis GCP_PROJECT_ID, GCP_WIF_PROVIDER e GCP_DEPLOY_SERVICE_ACCOUNT no repositório do GitHub, seguindo o infra/README.md.
```

Essas variáveis não existem porque os dados são sensíveis. Eles não são. Existem porque o ID do projeto e a conta de cada pessoa são diferentes, e com as variáveis isso fica flexível no código. O número do projeto de um aluno não existe na conta de outro, e manter esses valores fora do código evita erros.

> **ATENÇÃO · Cuidado com o que o agente mostra na tela**
> Ao mexer nas variáveis de ambiente, o agente pode exibir outras informações do ambiente no console. Se algo sensível aparecer, apague e troque as senhas correspondentes.

7. Inicialize o Firebase Hosting:

Prompt enviado ao Claude Code

```
Inicialize o Firebase Hosting neste projeto, seguindo o infra/README.md.
```

Esse passo é mais simples e leva em torno de cinco minutos. O Firebase tem o seu próprio CLI, e não usa o `gcloud`: o agente roda essa ferramenta pontualmente com o `npx`. O nome do site e os demais detalhes podem ser alterados.

Público

Quanto custa manter essa infraestrutura

Com tudo online, a cobrança ficou em R$ 50 no mês. Na prática, só dois serviços são cobrados: o Cloud SQL, o banco de dados, e o Vertex AI, pelo consumo de modelo. Os demais ficam gratuitos, porque só começam a cobrar a partir de milhares de imagens ou de downloads de imagens de um registro.

Uma conta nova do Google Cloud recebe em torno de 300 dólares de crédito por seis meses, o que cobre o projeto. E a nuvem pode ser outra, à sua escolha.

Segurança de secrets: o que fazer antes e depois de um vazamento

No desenvolvimento agêntico, ou AI-native, a segurança é de longe o que mais exige atenção. Performance é importante e pode ser otimizada, e o mesmo cuidado vale para a segurança. Há skills que ajudam nisso.

Dois cuidados valem desde o início:

- nunca criar senhas fixas, e sim senhas aleatórias. No passo 3, a senha foi criada pelo agente e colocada no secret, e ninguém precisa conhecê-la;
- nunca versionar secret ou senha. A partir do momento em que sobe para o GitHub, a informação está lá.

> **ATENÇÃO · Se um secret vazar, invalide**
> Se você ou alguém do time subir sem querer um secret ou uma senha no GitHub, invalide imediatamente esse secret e crie outro. Apagar não adianta: depois do commit, ele fica no `git log`. Um `git rebase` pode até resolver, mas o caminho seguro, para qualquer informação sensível vazada, como conexão de banco ou senha, é invalidar.

No ambiente local, a connection string fica em variável de ambiente: ela vai para o arquivo `.env`, e o serviço a lê de lá.

Conferência no console e o que fica para depois

Terminados os prompts, confira cada recurso no console: o Cloud SQL, o projeto com a billing account, o secret `database-url`, o registry e o Firebase. Isso é o início da infraestrutura. Conforme o projeto evolui, pode ser preciso criar mais alguma coisa, mas o mecanismo principal, com tudo habilitado, está pronto.

O Cloud Run não aparece nessa lista de propósito. Como ele é serverless e funciona sob demanda, é criado no primeiro deploy do código.

O Cloud Run aceita duas formas de publicação: subir o código ou subir uma imagem Docker, as duas executadas de forma serverless. Cada uma tem vantagens e desvantagens. O projeto usa a imagem Docker porque ela permite simular o backend localmente com mais facilidade. Simular o Cloud Run também seria possível, mas exigiria instalar o sandbox dele, e a intenção foi simplificar.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que cada prompt da aula provisiona no Google Cloud e em que ordem sem consultar o material?
- Você consegue explicar, sem consultar o material, a relação entre conta de faturamento e projetos e por que um projeto novo nasce sem nenhuma API habilitada?
- Você consegue explicar o que fazer quando um secret é enviado por engano ao GitHub sem consultar o material?

PRÁTICA RECOMENDADA

- Provisione a mesma fundação na sua própria conta de nuvem, enviando os prompts um a um e aprovando cada comando. No prompt da Workload Identity Federation, troque o repositório pelo seu fork e, ao final, confira cada recurso no console.

RESUMO DA AULA

- A fundação é criada por prompts ao Claude Code, que lê o `infra/README.md` e executa o `gcloud` um comando por vez, com aprovação.
- No GCP, uma conta de faturamento pode ter vários projetos isolados, e cada projeto nasce sem nenhuma API habilitada: habilita-se só o que será usado.
- A connection string vai direto para o Secret Manager, com senha aleatória; se um secret vazar, a primeira providência é invalidá-lo.
- A Workload Identity Federation deve apontar para o repositório do aluno, e o Cloud Run não é provisionado agora: ele nasce no primeiro deploy.

Público

Aula 7: Walking Skeleton e Deploy Zero

Voltar ao mapa da disciplina • *Voltar à Unidade 1*

OBJETIVOS DE APRENDIZAGEM

- Construir o esqueleto do projeto: uma API com health check, uma aplicação Angular que a consome e um Docker Compose para o ambiente local.
- Explicar como o workflow de deploy do GitHub Actions testa, autentica sem chave e publica no Cloud Run e no Firebase Hosting.
- Diagnosticar o status degraded do primeiro deploy e validar o ambiente com o `capstone:check`.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Com os dados da CVM normalizados em JSON e a fundação criada no Google Cloud, o passo seguinte é ter código em produção. Esta aula cria uma API e um frontend bem simples, ainda sem a integração com o agente, e um pipeline de deploy que publica tudo na infraestrutura.

O que sobe é basicamente um health check, mas ele já está em produção. Ao final, você terá o esqueleto do projeto rodando localmente e na nuvem, saberá ler o workflow de deploy e saberá resolver o erro mais comum do primeiro deploy.

Dois repositórios: o projeto completo e o seed

O curso tem dois repositórios, e a escolha depende de como você quer estudar.

O Capstone Project AI Researcher tem o projeto completo, de ponta a ponta, com todos os marcos e até a branch `starter`. Como ele já traz a esteira pronta, refazê-la ali, passo a passo, causaria problemas de merge.

O AI Researcher Seed tem só o projeto no estado da branch `starter`, sem uma branch com esse nome: esse conteúdo já é a `main`. Ou seja, a `starter` do AI Researcher é a `main` do AI Researcher Seed.

- Para fazer desde o começo, acompanhando as aulas, use o Seed.
- Para ver o resultado inteiro, com todos os fontes, use o Capstone.
- Para explorar e fazer a sua versão, fique à vontade: a licença é MIT. Você pode trocar a nuvem pela AWS ou o Angular pelo React.

O material é prático, mas nada substitui a sua própria prática. É fazendo a sua versão que você aprende.

O que compõe o walking skeleton

O esqueleto tem quatro peças: uma API com uma rota de health check, um contrato compartilhado, uma aplicação Angular que mostra o status e um banco Postgres. A aplicação é JavaScript de ponta a ponta.

Público

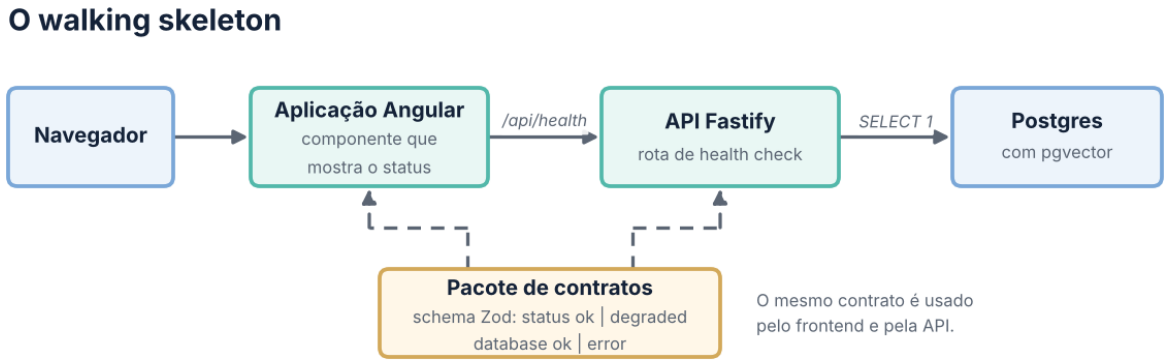


Diagrama editorial com as peças construídas na aula e o contrato compartilhado entre elas.

A API de health check

A API é criada com o seguinte prompt:

Prompt enviado ao Claude Code

```
Crie apps/api com Fastify e TypeScript. Uma rota GET /api/health que checa o Postgres de verdade via pg — consulta simples tipo SELECT 1, com timeout curto, e nunca derruba o processo se o banco estiver fora do ar, só reporta degraded. A resposta segue um schema zod em packages/contracts, compartilhado entre API e frontend. Deixe pronto também o jeito de rodar isso localmente contra o Postgres do docker-compose: um .env na raiz (gitignorado) com a DATABASE_URL apontando pra localhost — não pro nome do serviço do docker-compose, que só resolve dentro da rede do Docker —, um .env.example versionado documentando a variável, e o script dev da API carregando esse .env automaticamente (sem quebrar produção, que continua vindo do Secret Manager).
```

Nesse primeiro momento, a API tem uma única rota. Ela checa o Postgres de verdade, com uma consulta simples do tipo `SELECT 1` e timeout curto, só para saber se o banco está vivo. Se o banco estiver fora do ar, a rota nunca derruba o processo: apenas reporta `degraded`.

A resposta segue um contrato, definido com o Zod.

> **CONCEITO-CHAVE · Zod**
> Biblioteca TypeScript que faz, em tempo de execução, as mesmas checagens de tipos que o TypeScript faz em tempo de build, de transpilação. Usa a mesma tipagem do TypeScript e é muito utilizada nesse ecossistema.

O contrato do health check é simples e tem dois campos:

- `status`, que pode ser `ok` ou `degraded`;
- `database`, que pode ser `ok` ou `error`.

Ele fica em um pacote separado da pasta da API, porque o frontend também usa os contratos.

O prompt também prepara a execução local com o Postgres do Docker Compose. O ponto de atenção é a variável de ambiente com a URL do banco, que é diferente para cada pessoa. O arquivo `.env.example`, versionado, documenta a variável e serve de modelo para você criar o seu próprio `.env`. O script de desenvolvimento da API carrega o `.env` automaticamente, sem afetar a produção, que continua lendo do Secret Manager.

A aplicação Angular, criada com a skill oficial

O frontend é criado com este prompt:

Prompt enviado ao Claude Code

```
Crie apps/web com a skill oficial angular-new-app. Angular 22, standalone, signals. Um componente que consome /api/health via resource() e mostra o status na tela. Depois de gerar o projeto, desligue o prompt de coleta de analytics do Angular CLI (ng analytics off) — sem isso, a primeira vez que o ng serve roda dentro do pnpm dev/turbo, ele trava esperando uma resposta interativa que nunca chega, e a tela nunca sobe.
```


Público

O prompt manda usar a skill oficial `angular-new-app`, seguindo a regra do `CLAUDE.md`: havendo uma skill oficial do framework, é ela que deve ser usada. Se o projeto ainda não tiver a skill, o agente a baixa, seja o Claude Code, seja o Codex. Como ela vem do time do Angular, é a mesma para todos. O componente é simples: consome a API e mostra o status na tela, com o Angular 22.

> **ATENÇÃO · A pergunta de analytics trava o ambiente**
> O projeto usa o Turborepo como ferramenta de monorepo. Na primeira execução, o Angular às vezes pergunta se pode enviar dados de analytics. Ele fica preso esperando a resposta, e o Turborepo fica esperando por ele. Por isso o prompt já desliga a coleta de analytics. Esse erro apareceu no ensaio da aula, e o prompt foi ajustado para evitá-lo.

Durante a criação, o agente abre o Chrome e opera a tela como se fosse um usuário, usando uma skill própria para isso. Há quem use esse recurso para teste automatizado. É válido, mas não recomendo muito, porque sai caro o agente analisar tudo dessa forma. Existem ferramentas muito mais baratas para esse fim, como o Cypress.

Rodando localmente sem sair da sessão do agente

Para executar um comando no shell sem sair da sessão com o agente, coloque um ponto de exclamação antes do comando. O ambiente local sobe com `!pnpm dev`. Com o ambiente no ar, a aplicação consulta o Postgres local. A exclamação evita perder o ritmo de trabalho trocando de janela ou de terminal.

Docker Compose e o turbo prune

O Docker Compose sobe o banco de dados, o backend e o frontend de uma vez:

Prompt enviado ao Claude Code

```
Crie docker-compose.yml na raiz subindo Postgres com pgvector, a API e o Web, usando turbo prune --
docker nos Dockerfiles pra build eficiente em monorepo. O Postgres do compose é efêmero, só pra
desenvolvimento — nunca é o Cloud SQL.
```

> **CONCEITO-CHAVE · turbo prune --docker**
> Recurso do Turborepo que otimiza a geração das imagens em um monorepo, separando o que pertence a cada projeto. Sem ele, seria como fazer uma única imagem para tudo, o que não ficaria otimizado.

O monorepo tem dois projetos, o frontend e o backend, e ainda vai receber o agente. O `docker-compose.yml` apenas orquestra três imagens:

- a do Postgres, uma imagem que já vem com a pgvector habilitada, o que dispensa pegar uma imagem base e instalar a extensão;
- a do backend, com o ambiente local e a porta 3000;
- a do frontend, cujo Dockerfile é gerado com o `turbo prune`.

O Postgres do Compose é efêmero e serve só para desenvolvimento. Ele nunca é o Cloud SQL.

O workflow de deploy

O workflow do GitHub Actions é criado uma vez e só incrementado nas unidades seguintes. Conforme o projeto cresce, a esteira evolui, sempre executando os testes e fazendo o deploy em produção.

Prompt enviado ao Claude Code

```
Crie .github/workflows/deploy.yml. Um job test que builda, testa e faz lint em todo PR. Um job deploy,
que só roda em push pra main e só depois do test passar, autenticado via a Workload Identity Federation
que já existe no GCP — sem chave de service account. Ele builda e sobe a imagem Docker da API pro
Artifact Registry, faz deploy no Cloud Run com acesso ao Cloud SQL e ao secret database-url, builda o
Angular e publica no Firebase Hosting.
```


Público



Fluxo editorial que organiza as etapas do workflow descritas na aula.

O arquivo `deploy.yml` gerado tem quatro pontos que vale entender:

- **Gatilho**: o push na `main`. Em um fluxo como o GitFlow, isso é comum: vários PRs podem ser abertos, mas o deploy só acontece no push. Também é possível criar outro workflow, disparado na criação do PR, que faça só os testes.
- **Ambiente**: a esteira roda no Ubuntu, com o Node na versão 24, e define o environment em que a entrega é feita.
- **Instalação pelo lockfile**: as dependências são instaladas com o lockfile congelado, e depois vêm o lint e os testes.
- **Autenticação**: o job de deploy usa o workload identity provider. O arquivo informa qual é o provider do Google e qual é a service account. Isso é só identificação, não é secret.

> **ATENÇÃO · Faça o build sempre pelo lockfile**
> Instalar pelo lockfile garante que nenhuma dependência seja atualizada sem ter sido testada. Para atualizar uma dependência, atualize, rode todos os testes e só então atualize e suba o lockfile.

A autenticação funciona como descrito na Aula 4. A esteira faz a impersonação para o deploy, e o token é efêmero: a cada execução há uma autenticação nova. É como se fosse criado um usuário só para aquele deploy, que depois some. Como a Workload Identity Federation já existe no GCP, não há chave de service account nem secret no GitHub.

Depois da autenticação, o job faz o build da imagem e a envia ao registry, faz o deploy no Cloud Run e, por fim, publica o projeto Angular no Firebase Hosting.

O primeiro deploy em produção

A esteira é disparada por um commit seguido de push, que pode ser feito à mão ou pedido ao agente. No GitHub, é possível acompanhar a execução: o job de teste roda rápido, porque só existe o health check, e depois vem o deploy, com a autenticação, o envio da imagem ao Artifact Registry, o endpoint no Cloud Run, o build do Angular e a publicação no Firebase Hosting.

A URL de produção depende do nome dado ao projeto, então a sua pode ser diferente da usada na aula. Com tudo certo, a página mostra `database` ok e `status` ok.

Por que o primeiro deploy pode aparecer como degraded

É provável que a sua primeira execução mostre `degraded` e `error`, mesmo com tudo funcionando localmente.

> **ATENÇÃO · O Cloud Run nasce privado**
> Por segurança, o Cloud Run cria o serviço como privado na primeira vez. Com isso, o Firebase não consegue chegar à API para fazer o health check. O erro é um 403. Se você pedir ao agente que analise, ele vai ver que o serviço não está público.

A correção é liberar o acesso público a esse endpoint:

Prompt enviado ao Claude Code

```
O Cloud Run está privado. Pra este endpoint (/api/health, sem custo de LLM ainda) ficar acessível
publicamente, preciso liberar allUsers com roles/run.invoker. Antes de rodar, confirme comigo que
entendo que isso expõe o serviço pra internet inteira.
```


Público

Tornar um serviço público exige avaliar o risco. Aqui ele é aceitável porque o endpoint ainda não tem custo de LLM: o `/api/health` pode ser chamado um milhão de vezes sem consequência, já que só executa um `SELECT 1` e devolve ok. Para o agente e para outras partes do backend, a proteção contra abuso, que poderia estourar a conta, vem com o rate limit e outros mecanismos, assunto da Aula 23.

capstone:check e a tag v1

O repositório traz um script, o `capstone:check`, que verifica os requisitos não funcionais da aplicação. Ele bate em uma URL, que pode ser a de produção, e confere de fora se alguns requisitos estão atendidos.

O script terá 11 checks ao todo, e ele cresce com o projeto: mais adiante, verifica até se a métrica existe. Por enquanto, os três primeiros são:

1. Endpoint público válido.
2. Deploy automatizado ok.
3. Segredos protegidos, ou seja, nenhuma credencial exposta.

Cada bloco concluído do projeto recebe uma tag. Ao terminar esta etapa, crie a tag `v1` no seu repositório.

> **EM SÍNTESE**
> A Unidade 1 termina com o health check, o frontend e o backend implantados na nuvem. Daqui em diante, o trabalho é iterar sobre esse projeto, sempre testando e subindo em produção.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que a rota de health check verifica e por que o contrato Zod fica em um pacote compartilhado sem consultar o material?
- Você consegue explicar, sem consultar o material, como o `deploy.yml` autentica no Google Cloud sem nenhum secret no GitHub?
- Você consegue explicar por que o primeiro deploy pode mostrar `degraded` e `error` mesmo funcionando localmente sem consultar o material?

PRÁTICA RECOMENDADA

- Reproduza o walking skeleton no seu repositório, a partir do AI Researcher Seed: envie os prompts, acompanhe a esteira no GitHub, execute o `capstone:check` contra a sua URL e, ao final, crie a tag `v1` do seu projeto.

RESUMO DA AULA

- O projeto completo está no repositório Capstone; para construir passo a passo, o ponto de partida é o AI Researcher Seed, cuja `main` equivale à branch `starter`.
- O walking skeleton tem uma API Fastify com health check, um contrato Zod compartilhado, uma aplicação Angular criada com a skill oficial e um Docker Compose com Postgres e pgvector.
- O workflow de deploy roda no push para a `main`, instala pelo lockfile, testa, autentica por Workload Identity Federation e publica no Cloud Run e no Firebase Hosting.
- O Cloud Run nasce privado: liberar o `/api/health` publicamente resolve o status `degraded`, e o `capstone:check` valida de fora os primeiros requisitos.

Público

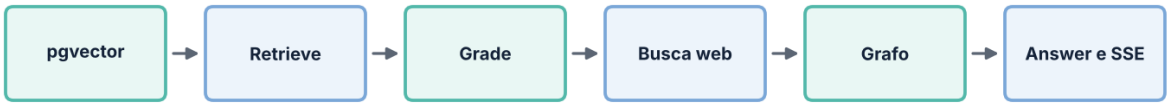
Unidade 2: Orquestração Cognitiva e APIs

Do banco vetorial ao grafo que busca, avalia, recorre à web e responde com citação.

NESTA UNIDADE

- Aula 8: pgvector, seed
- Aula 9: Os nós retrieve e grade
- Aula 10: Serper
- Aula 11: Fallback pra busca web, upsert retroativo
- Aula 12: O grafo real: aresta condicional e o guard do loop
- Aula 13: O nó answer, citação obrigatória, guardrail por código e streaming SSE

O cérebro do agente



Fluxo conceitual criado para apoiar a leitura da unidade.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público

Aula 8: pgvector, seed

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Explicar como embeddings, proximidade vetorial e top-k permitem encontrar os documentos que respondem a uma pergunta.
- Descrever o papel do índice HNSW no pgvector e a sua principal característica: a busca rápida, porém aproximada.
- Criar a tabela vetorial com migration e popular o banco com um script de seed que gera os embeddings em lotes.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O esqueleto da API, o frontend, a esteira de deploy e a infraestrutura estão prontos, mas o chatbot ainda não sabe nada de nenhuma empresa. A Unidade 2 constrói o cérebro do AI Researcher, o agente, e o primeiro passo é o RAG.

> **CONCEITO-CHAVE · RAG (Retrieval-Augmented Generation)**
> Técnica que resolve uma pergunta simples: dada a pergunta do usuário, como achar, em uma base de centenas de documentos, os poucos que realmente a respondem, para dar um contexto local à pergunta ou à ação do agente.

Como os dados são de empresas abertas, seria possível deixar o agente buscar tudo sozinho. O risco é o modelo alucinar ou pegar uma informação incorreta. Por isso, os dados da CVM, já normalizados em JSON, vão para um banco de dados.

Esta aula explica os conceitos por trás desse banco e o constrói. Ao final, você saberá explicar embeddings, top-k e o índice HNSW e terá a tabela vetorial criada e populada.

Por que uma busca textual não basta

Em um banco de dados comum, com registros em tabelas, a busca por texto depende de encontrar as mesmas palavras. Isso falha com frequência: o usuário usa outros termos, escreve errado ou formula a frase de outro jeito.

> **EXEMPLO · "Rentabilidade" e "rentável"**
> Para a pergunta sobre a rentabilidade ou a receita da Vale do Rio Doce no último trimestre de 2025, um `SELECT` sobre o texto pode não encontrar a mesma palavra. Em vez de "rentabilidade", o usuário poderia escrever "rentável". Mesmo com `LIKE`, a busca normal não é suficiente.

A saída é um banco de dados vetorial.

Embeddings: o texto como um ponto no espaço

Ao inserir os dados no banco vetorial, um modelo faz o embedding de cada texto. O projeto usa o modelo do Gemini, mas poderia ser qualquer outro.

> **CONCEITO-CHAVE · Embedding**
> Valor numérico que representa um texto: uma matriz numérica de N dimensões, guardada no banco de dados.

Público

Para visualizar, imagine só duas dimensões. O algoritmo de embedding transforma cada palavra ou termo em um ponto no espaço, com uma posição x e uma posição y. O embedding real funciona do mesmo jeito, com muitas dimensões.

Na busca, a etapa de retrieve, a pergunta do usuário também é transformada em embedding. O algoritmo do banco de dados calcula então a proximidade entre o ponto da pergunta e os outros pontos do espaço.

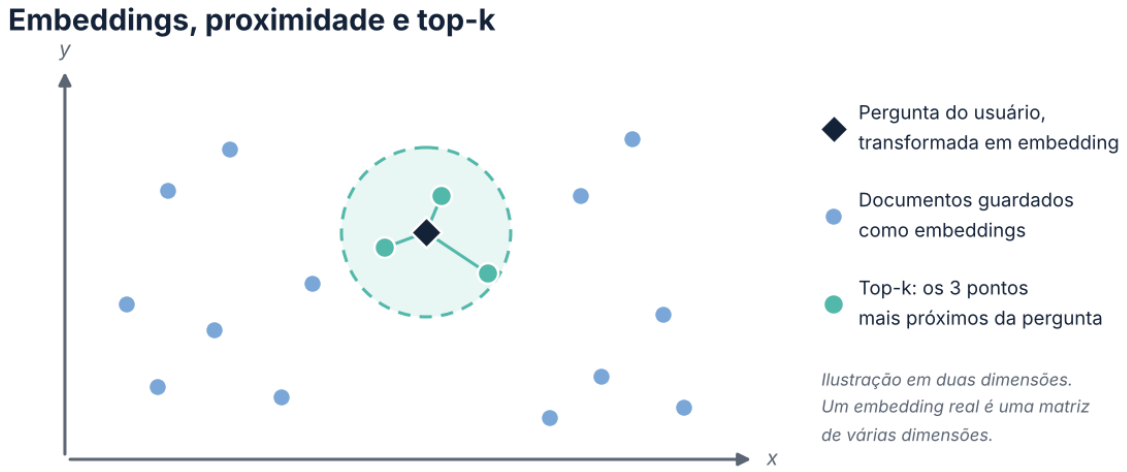


Ilustração conceitual em duas dimensões, como a usada na aula. As posições dos pontos não representam dados reais.

Top-k: quantos vizinhos trazer

A busca é parametrizada pela quantidade de pontos mais próximos que se quer receber.

O top-k são os k pontos mais próximos da pergunta. Com k igual a 3, a busca responde: dada a minha pergunta, quais são os três documentos mais próximos que podem respondê-la?

Em uma pergunta sobre o Itaú, o ponto mais próximo é um documento do banco, e outros documentos do Itaú estão por perto. A busca traz os três mais próximos, ou os dez, conforme a configuração.

O banco traduz esses pontos de volta nos textos dos documentos, e eles são enviados ao prompt como uma informação mais rica. A LLM passa a trabalhar em cima do contexto: com base nesses dados, qual é a rentabilidade?

> **ATENÇÃO · Menos alucinação não é alucinação zero**
> Com um contexto rico e baseado em dados, as chances de alucinar são menores, mas nunca nulas, mesmo com o melhor modelo do mundo. Aplicações agênticas e de IA generativa são sistemas não determinísticos. A engenharia e a arquitetura precisam ter sempre uma conferência: os dados são muito bons e próximos da realidade, mas não são absolutos.

Bancos vetoriais e a opção pelo pgvector

Há muitos bancos de dados vetoriais especializados no mercado. Um dos mais usados no mundo de IA é o Pinecone. Há também o Weaviate e os bancos vetoriais das próprias nuvens, como o da AWS.

O Postgres, porém, é tão flexível e extensível que, dependendo do projeto, não é preciso criar outra infraestrutura nem administrar outro banco. Quem já usa Postgres pode habilitar a extensão pgvector, e ele passa a atuar como um banco vetorial muito competente. A escolha depende da necessidade, e às vezes o mais simples é o melhor.

Com o pgvector, o SQL continua o mesmo. O que muda são os operadores da busca.

Os operadores de distância

A distância entre a pergunta e os dados pode ser calculada de várias maneiras, todas de cálculo vetorial:

- a distância simples;
- a distância do cosseno;

Público

- a distância do produto interno.

Na query, cada forma de cálculo aparece como um sinal diferente: é o operador, o tipo de busca usado para limitar o top-k. As outras formas não são inúteis e servem a outros casos, mas a mais utilizada é a do cosseno, que é a do projeto.

O índice HNSW: rápido, porém aproximado

Sem índice, o banco faz o equivalente a um full scan. Ele calcula a distância do cosseno entre a pergunta e todos os registros da tabela e só então escolhe os mais próximos. Com os 80 registros do projeto, isso é insignificante. Em bases maiores, vale usar o índice HNSW, que não é exclusivo do Postgres: ele organiza os registros em camadas e faz a busca por elas.

> **ATENÇÃO · A busca com HNSW é aproximada**
> Um registro que se encaixaria no top-k, talvez até o mais adequado, pode ficar fora do resultado. Isso acrescenta mais um elemento não determinístico à busca do RAG.

Para quem vem de um ERP ou de outro sistema totalmente determinístico, isso pode parecer estranho, mas é a natureza dos sistemas que usam IA: existe uma perda, e ela precisa ser considerada no projeto. Quando nenhuma perda é aceitável, o caminho pode ser outro: um banco relacional comum, com tabelas normais, e um agente instruído a dizer que não encontrou a informação.

Critério	Sem índice	Com índice HNSW
Como busca	Calcula a distância de todos os registros, como um full scan, e depois escolhe os mais próximos.	Organiza os registros em camadas e faz a busca por elas.
Resultado	Passa por todas as linhas da tabela.	Aproximado: um registro mais adequado pode ficar fora do top-k.
Quando faz sentido	Bases pequenas, como os 80 registros do projeto.	Bases de dados maiores, em que a rapidez da busca importa.

A tabela documentos: schema, migration e índice

A tabela é criada com o Drizzle, um framework de ORM para Node. No projeto, ele é usado mais pelas migrations, para não escrever os `CREATE TABLE` à mão, e as consultas usam mais SQL do que ORM. Há outras ferramentas, conforme a linguagem.

Um framework com suporte a migration dá rastreabilidade às alterações do banco e permite um rollback quando a estrutura está errada. Depois que há dados, fica mais complicado, mas o controle e a documentação das mudanças são melhores.

1. Crie o schema e gere a migration:

Prompt enviado ao Claude Code

```
Crie o schema Drizzle da tabela documentos em packages/agent/src/db/schema.ts: colunas para todos os campos do corpus (ticker, company_name, setor, period, period_end, os seis indicadores financeiros como bigint, source_url, fetched_at, o paragraph já pronto) mais uma coluna embedding do tipo vector(768) do pgvector. Adicione um índice único em (ticker, period) e um índice HNSW na coluna embedding usando o operador de distância de cosseno. Gere a migration com drizzle-kit.
```

O schema resultante é o de um banco de dados normal, com duas diferenças: a coluna `vector`, que guarda o embedding, e o índice HNSW. Quem conhece Postgres vê que pouca coisa muda. Ao criar um índice HNSW, é preciso informar o operador usado naquela coluna. Aqui, o índice é otimizado para a busca por cosseno. Há também o índice da chave única, que é um índice comum.

No repositório seed, o Drizzle ainda não está entre as dependências, e o agente pede para instalá-lo.

2. Aplique a migration no banco local:

Com o Postgres local no ar, pelo Docker, execute `pnpm db:migrate`. Dentro do agente, use o ponto de exclamação para rodar o comando como no shell.

Público

Gerar a migration não é aplicá-la. Na demonstração, ela tinha sido gerada, mas não aplicada. Confira no banco se a extensão está habilitada e se a tabela `documentos` aparece no schema `public`. Quando um comando executado dentro do agente falha, ele é proativo e tenta entender o que houve: adicionou o script que faltava, no mesmo padrão do `collect`, e aplicou a migration no Postgres local.

O script de seed: carregar o corpus no banco

O seed pega os dados normalizados na Unidade 1 e os carrega no banco, com upsert. Tudo local.

Prompt enviado ao Claude Code

```
Crie um script de seed em packages/agent/src/seed/run.ts que lê todos os JSONs de
packages/corpus/data/documentos/, valida com um schema zod, gera o embedding do campo paragraph de cada
documento chamando a Vertex AI em lotes de 10 (não um por vez), e insere no Postgres via Drizzle com
upsert por (ticker, period) — se rodar de novo, atualiza em vez de duplicar. Separe a criação do cliente
de embeddings reais numa função própria, porque o modo DEMO do projeto (aula futura) vai precisar trocar
isso por um fake sem chamar rede.
```

O script faz cinco coisas:

1. Lê todos os JSONs do corpus.
2. Valida cada um com o schema Zod.
3. Gera o embedding do campo `paragraph`, chamando o Vertex AI em lotes de 10, e não um por vez.
4. Insere no Postgres pelo Drizzle, com upsert na chave única, que é ticker e período: se rodar de novo, atualiza em vez de duplicar.
5. Mantém a criação do cliente de embeddings em uma função própria, porque o modo demo, criado na Aula 14, troca esse cliente por um fake.

A orquestração com a LLM, que aqui é a chamada de embedding, usa o LangChain.js, como define o README da infraestrutura. Antes de instalar as dependências, o agente pergunta, como manda o `CLAUDE.md`. O seed é executado com `pnpm seed`.

Embeddings: custo e consistência

O modelo de embedding do projeto é o do Vertex AI, mas há outras opções, inclusive embedding local, rodando um modelo pequeno em vez de uma LLM, porque gerar embeddings não exige muito processamento.

O custo é baixo por dois motivos. A operação em si é barata, e a carga acontece uma vez, ou a cada trimestre, no caso do projeto, e não a cada uso do sistema. O RAG é carregado raramente e consultado o tempo todo. A busca é uma operação muito mais cara do que a geração de embeddings.

> **ATENÇÃO · Use o mesmo modelo de embedding na carga e na busca**
> O campo vetorial é preenchido com a função de embedding de algum provedor, como o Google ou a OpenAI. Na busca, o modelo precisa ser o mesmo. Isso não é padronizado: cada modelo gera o embedding de um jeito, um mais rápido, outro com dados mais clusterizados, com tipos e preços variados. Se o modelo mudar, a transformação não é a mesma, e a busca não encontra direito.

Gerar embedding é algo que ocorre o tempo todo em uma LLM. É a primeira coisa feita com qualquer prompt: criam-se os tokens, faz-se o embedding deles, e o resultado segue para o modelo.

O limite de 768 dimensões

O embedding do Vertex AI cria, por padrão, vetores com 3.072 dimensões. A extensão do Postgres aceita no máximo 2.000. Por isso, o README da infraestrutura fixa o limite em 768, e o Vertex AI sempre devolve embeddings de 768 posições. Sem esse parâmetro, o resultado viria com 3.072 e estouraria o tamanho do campo.

O que fica gravado no banco

Depois do seed, cada registro tem um ID, o parágrafo e o embedding. O embedding não é um número só: é uma matriz com um monte de valores, cuja lógica vem do modelo do Vertex AI. O tipo do campo é `vector`, mas, na prática, é um campo numérico, uma matriz multidimensional, parecido com um campo blob.

Público

> **EM SÍNTESE**
> A busca do RAG é uma busca de proximidade entre dois pontos. Há vários tipos de proximidade, e o projeto usa o cosseno.

O objetivo desta disciplina não é construir modelos, embora isso seja possível. É formar um engenheiro de IA, que conhece os conceitos para criar sistemas que consomem e utilizam IA.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar como a pergunta do usuário vira um embedding e como o top-k seleciona os documentos mais próximos sem consultar o material?
- Você consegue explicar o que o índice HNSW faz e por que a sua busca é aproximada sem consultar o material?
- Você consegue explicar o que o script de seed faz e por que o embedding foi limitado a 768 dimensões sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O RAG transforma documentos e perguntas em embeddings e busca os top-k pontos mais próximos, o que dá ao modelo um contexto rico e reduz, sem zerar, a chance de alucinação.
- O pgvector faz do Postgres um banco vetorial: o SQL é o mesmo, mudam o tipo `vector`, o operador de distância (aqui, cosseno) e o índice HNSW, que é rápido, mas aproximado.
- O schema é criado com Drizzle e migration, e o seed gera os embeddings no Vertex AI em lotes de 10 e grava com upsert por ticker e período.
- A busca precisa usar o mesmo modelo de embedding da carga, e os vetores foram limitados a 768 dimensões porque o padrão de 3.072 não cabe no limite da extensão.

Público

Aula 9: Os nós retrieve e grade

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Construir o nó retrieve, que transforma a pergunta em embedding e busca no Postgres os documentos mais similares.
- Justificar por que padrões de projeto, como a inversão de dependência, continuam importantes quando quem escreve o código é um agente.
- Construir o nó grade, que usa saída estruturada da LLM e uma regra explícita para julgar se o contexto recuperado é suficiente.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Com o banco vetorial criado e carregado, começa a construção do backend do agente. De forma bem simples, o agente é formado por partes chamadas nós, como os nós de uma rede.

Esta aula constrói os dois primeiros. O nó retrieve faz a busca: pega a pergunta, transforma em embedding e consulta o banco de dados para achar os conteúdos que a respondem. O nó grade julga se o que voltou dessa busca é suficiente para responder.

Ao final, você saberá ler o código dos dois nós, explicar por que eles recebem as suas dependências por parâmetro e descrever a regra que decide quando o agente precisa buscar informação fora da base.

O nó retrieve: da pergunta aos documentos mais próximos

O nó retrieve recebe a pergunta do estado, gera o embedding dela e busca no Postgres os documentos mais próximos. Ele devolve o top-k, que por padrão são os cinco mais similares.

O nó é criado com este prompt:

Prompt enviado ao Claude Code

```
Crie o nó retrieve do grafo em packages/agent/src/nodes/retrieve.ts. Ele recebe a pergunta do estado, gera o embedding dela e busca os documentos mais próximos no Postgres, top-k com default 5. Siga a convenção do projeto: o nó não importa nenhum cliente global — ele é uma factory que recebe um cliente de embeddings e uma função de busca por parâmetro, pra ser testável com fake, sem chamada real de rede ou banco. Crie também a função de busca em packages/agent/src/db/retrieve-similar.ts, usando o operador de distância de cosseno do Drizzle.
```

Além do comportamento do nó, o prompt pede duas coisas de design de código:

- **Convenção do projeto**: o nó não importa nenhum cliente global. Ele é uma factory que recebe por parâmetro um cliente de embeddings e uma função de busca.
- **Função de busca separada**: ela fica no arquivo `retrieve-similar.ts` e usa o operador de distância de cosseno do Drizzle.

No código gerado, o `top_k` é uma constante com valor 5, e o nó faz algo muito simples: gera o embedding da pergunta e o envia à função de busca.

A função de busca usa o Drizzle, que tem suporte a busca vetorial, para calcular a distância entre o embedding da pergunta e os dos documentos. Para o entendimento, pense na distância em um plano cartesiano, embora os embeddings tenham muito mais dimensões.

Público

> **CONCEITO-CHAVE · Similaridade**
> A consulta calcula a distância com uma função do Drizzle ORM e, por convenção, obtém a similaridade como 1 menos a distância. Em SQL puro no Postgres, a mesma consulta usaria os operadores de distância apresentados na Aula 8.

Inversão de dependência: o nó recebe o cliente por parâmetro

Receber o cliente de embeddings e a função de busca por parâmetro é a inversão de dependência aplicada ao código do agente. Ela resolve dois problemas:

- **Troca de fornecedor**: para deixar de usar o embedding do Gemini e passar a usar o da OpenAI, por exemplo, basta trocar o cliente.
- **Teste independente**: o teste unitário não pode depender do Vertex AI. Em um teste de QA ou em um teste manual, a aplicação usa o embedding do Gemini. No teste unitário, o nó recebe um fake.

O teste unitário do retrieve cria um embedding falso, que já devolve os valores, e monta o cenário para testar a lógica. Ele roda sem o banco de dados e sem o Vertex AI no ar.

No Claude Code, um comando iniciado por ponto de exclamação roda ali mesmo, sem trocar de aba. Na demonstração, o teste foi filtrado para rodar só o retrieve, e não os testes unitários de todo o projeto.

Por que padrões de projeto importam mais com agentes

É um erro pensar que, como o agente escreve o código, o padrão de projeto deixou de importar. Hoje os padrões de projeto são mais importantes do que nunca, por dois motivos.

O primeiro é o custo. Quanto menos o agente tiver de mudar no projeto, mais barato sai. Antes, os bons padrões reduziam um custo de tempo e um custo de comunicação. Com a IA, o custo é financeiro.

O segundo é a revisão. O grande problema hoje é a análise de PR, ou seja, a análise do que o agente cria. Uma refatoração, feita por um agente ou por uma pessoa, tem de ser cirúrgica. Reescrever todo o código não é uma opção.

> **EM SÍNTESE**
> Vale pensar em padrões de código para o projeto e vale estudá-los. Não é verdade que basta fazer um bom prompt.

Há ainda a questão do modelo. O projeto foi desenvolvido com o Claude e o modelo Sonnet, mas você pode estar usando um modelo mais fraco ou um modelo local. Quanto mais padrões de projeto forem descritos, melhor fica o código.

Skills e a responsabilidade de ler o código

Ao executar o prompt do retrieve, o Claude Code chama a skill `langgraph-node`. Criei essa skill antes, para explicar o padrão que quero no uso do LangGraph na construção do agente.

Sem uma skill para o Drizzle, o agente lê o código da biblioteca para montar a busca. Criar ou buscar uma skill para o Drizzle pouparia esse trabalho.

> **ATENÇÃO · Ler o código é responsabilidade sua**
> O agente pode criar o código e pode ajudar a debugar. Ler o código continua sendo responsabilidade de quem desenvolve. Por isso ferramentas como o Prettier seguem necessárias: o código não é lido só pelo agente.

Por que o agente precisa julgar o contexto

O retrieve traz o contexto, mas ter o contexto não basta. Nem sempre a informação que está no RAG é suficiente para a pergunta.

A base foi montada com os dados da CVM, normalizados e carregados no banco. Se a pergunta do usuário é respondida por esses dados, o agente responde sem buscar na internet. Se a pergunta é mais aberta, o agente precisa avaliar se o contexto é suficiente e, quando não for, buscar na internet.

Público

Essa avaliação é um processamento próprio do agente: o nó grade. Pedir a uma LLM que responda em texto livre se o contexto é suficiente pode trazer problemas, e por isso o nó usa saída estruturada.

O nó grade: saída estruturada e regra de suficiência

O nó grade recebe do estado a pergunta e os documentos, ou seja, a pergunta e o contexto do RAG, e classifica se o contexto encontrado responde à pergunta. O nome vem do inglês grade.

Prompt enviado ao Claude Code

```
Crie o nó grade do grafo em packages/agent/src/nodes/grade.ts. Ele recebe question e documents do estado e usa saída estruturada do LLM — um schema zod com sufficient (boolean) e missingAspects (array de string) — pra classificar se o contexto encontrado responde a pergunta. Separe o prompt em packages/agent/src/prompts/grade.ts: o contexto é só numérico (indicadores financeiros por empresa e trimestre), então perguntas qualitativas — motivo por trás de um número, notícia, fato relevante, guidance, previsão de preço — devem ser sempre classificadas como insuficientes, mesmo que o documento seja da empresa certa. Siga a convenção do projeto: factory recebendo o cliente de LLM por parâmetro, sem importar client global, testável com fake.
```

O prompt define três coisas:

- **A saída estruturada**: um schema com `sufficient`, verdadeiro ou falso, e `missingAspects`, a lista do que está faltando.
- **A regra**: o contexto é só numérico, com indicadores financeiros por empresa e trimestre. Pergunta numérica é suficiente. Pergunta qualitativa, sobre motivos, notícias e tudo o que não está no RAG, é insuficiente, mesmo que o documento seja da empresa certa.
- **O design de código**: a factory que recebe o cliente de LLM por parâmetro, para ser testável.

No código, o grade é uma função como qualquer outra. Ele não faz busca no RAG: pega a pergunta inicial e os documentos que o retrieve trouxe e passa os dois por um prompt.

O arquivo do prompt começa pelos documentos que voltaram do banco. Se nenhum foi recuperado, o contexto já é insuficiente. Depois vem o system prompt, que instrui o modelo a avaliar o contexto conforme a regra resumida na tabela a seguir:

Situação	Classificação do contexto
Nenhum documento foi recuperado do RAG.	Insuficiente.
A pergunta pede um indicador numérico que os documentos contêm, para a empresa e o período.	Suficiente: o agente responde sem buscar na internet.
A pergunta é qualitativa (motivos, notícias, guidance), mesmo com documento da empresa certa.	Insuficiente, com os aspectos que faltam descritos em `missingAspects`.
O indicador, a empresa ou o período pedidos não aparecem nos documentos.	Insuficiente: o modelo não inventa o número.

O system prompt informa que o contexto vem de uma base vetorial que contém apenas indicadores. Quando a pergunta pede algo que esse tipo de contexto não tem, o modelo preenche `missingAspects` descrevendo o que falta, como a notícia ou o motivo da queda do lucro.

> **ATENÇÃO · Pergunta numérica não autoriza inventar número**
> O system prompt instrui o modelo a não inventar números que não estão no contexto só porque a pergunta é numérica. Sem o indicador, a empresa ou o período nos documentos, a classificação é insuficiente, e o fluxo segue para o nó que busca informação na internet.

Calibrando a regra de suficiência

A regra do que é suficiente pode ser descrita com mais detalhe, e isso varia de projeto para projeto. Os dois extremos têm custo:

- **Regra fechada demais**: muito pouca coisa é considerada suficiente. O agente busca muito mais na internet, o que pode trazer mais alucinação e mais custo.

Público

- **Regra aberta demais**: tudo é suficiente. O agente nunca busca na internet, e a informação pode ficar incompleta.

Não há mágica: é tudo prompt

O nó grade é um nó interno. O usuário não pediu essa avaliação: ela é uma regra que o sistema impõe sobre o que é suficiente e sobre o que precisa ser buscado na internet.

> **CONCEITO-CHAVE · Cadeia de pensamentos**
> É um prompt interno, executado pelo sistema, que o usuário não vê. O texto do grade é um system prompt: o usuário não mexe nele nem olha para ele.

No projeto, por didática, a cadeia de pensamentos aparece ao lado da tela. Em um sistema normal, ela não aparece.

Para desmistificar: o sistema sabe se o contexto é suficiente porque recebeu uma regra, entregue a uma LLM, o Gemini, que verifica se a pergunta e os documentos estão dentro dela. Se estão, é suficiente. Se não estão, não é.

> **EM SÍNTESE**
> Um sistema de IA não tem magia nem AGI, a inteligência artificial geral. São prompts e mais prompts. O raciocínio que o Claude Code ou o Codex mostram nada mais é do que isso: prompts internos que avaliam, ampliam e repensam.

O mesmo vale para os próprios agentes de código. O Claude Code é o que se chama de harness. Há outros, inclusive de código aberto. Olhando por dentro, eles usam os modelos, mas trazem um monte de system prompts, com instruções sobre onde pegar o código e sobre qual comando do Git usar para cada coisa.

Os modelos ficam maiores e com mais base de dados, mas o ponto aqui é o system prompt, e não o modelo. Desmistificar isso é o que permite construir sistemas sabendo quais são as vantagens e quais são as limitações.

Os testes do grade

O grade tem dois testes unitários:

- **Teste do prompt**: não chama nenhuma LLM. Verifica se as funções montam e enviam o conteúdo corretamente, sem avaliar a resposta.
- **Teste do nó**: cria um fake chat model e o injeta no nó. É a inversão de dependência, um conceito de design de código bem antigo e muito válido, em uso.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que o nó retrieve faz com a pergunta do usuário e como a similaridade é calculada sem consultar o material?
- Você consegue explicar por que os padrões de projeto continuam importantes quando o código é escrito por um agente sem consultar o material?
- Você consegue explicar como o nó grade decide se o contexto é suficiente e o que acontece quando a regra é fechada demais ou aberta demais sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O nó retrieve transforma a pergunta em embedding e busca os cinco documentos mais similares, recebendo o cliente de embeddings e a função de busca por parâmetro.
- Padrões de projeto reduzem o que o agente precisa mudar, o custo e o esforço de revisão, e ler o código continua sendo responsabilidade de quem desenvolve.
- O nó grade recebe a pergunta e os documentos e devolve uma saída estruturada dizendo se o contexto é suficiente e o que falta.
- Não há mágica: a suficiência é uma regra escrita em um prompt interno, e a calibragem dessa regra define quanto o agente recorre à internet.

Público

Aula 10: Serper

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Explicar por que o agente precisa de um provedor de busca na web e por que o Serper foi o escolhido.
- Criar a interface `SearchProvider` com o padrão adapter, para poder trocar de fornecedor sem grandes mudanças.
- Testar o provedor de busca com respostas falsas, sem consumir a cota do serviço de terceiro.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O corpus da CVM só tem valores numéricos. Quando a pergunta é qualitativa, ou quando o trimestre pedido não está no banco de dados por algum motivo, o nó grade marca o contexto como insuficiente. Nesse caso, o agente busca as informações na internet, e para isso precisa de um provedor de busca.

Esta aula cria o serviço de busca que o agente vai consumir. Ao final, você saberá por que o provedor escolhido foi o Serper, como o padrão adapter deixa o sistema pronto para trocar de fornecedor e como testar esse serviço sem chamar a API real.

A escolha do provedor: do Google Custom Search ao Serper

Até o ano passado, o Google oferecia o Custom Search, um serviço para fazer buscas no Google de forma programática, por uma API REST que retorna JSON. Ele era muito utilizado, mas foi depreciado.

> **ATENÇÃO · Google Custom Search depreciado**
> O serviço não aceita novas chaves de API. Quem já tem uma chave em uso precisa migrar até 1º de janeiro de 2027.

Hoje, vários serviços na internet oferecem essa funcionalidade. Alguns a oferecem também como MCP, para consumo por agentes, além de REST. O projeto usa o Serper.

Escolhi o Serper porque ele dá até 2.500 queries gratuitas sem exigir cartão de crédito. Nos outros serviços, é preciso informar o cartão. Ele não tem um free tier permanente: usadas as 2.500 queries, é preciso contratar um dos planos, o que não chega a acontecer neste projeto.

A chave de API e o cuidado com secrets

Para usar o Serper, são três passos:

1. Faça o cadastro no site do Serper, com e-mail e senha.
2. No dashboard, crie uma API key.
3. Guarde a chave na variável de ambiente `SERPER_API_KEY`. É desse nome que o código lê a chave.

> **ATENÇÃO · Secret nunca vai para o código-fonte**
> Secret, e isso inclui API key, nunca fica no código-fonte e nunca é versionado. Ele fica em uma variável de ambiente ou, na nuvem, no Secret Manager, no vault.

Se você ou a sua equipe subir por acidente uma API key ou um secret de banco de dados no GitHub, mesmo em um repositório privado, o primeiro passo é invalidar a credencial, como se ela tivesse sido comprometida.

Público

No banco de dados, isso significa fazer a rotação da senha. No Serper, é clicar em reset ou apagar a chave e criar outra.

Apagar do histórico ou do estado atual não resolve, porque, uma vez na internet, a informação está lá para sempre. Arrumar o código para o secret não estar lá é necessário, mas o principal é invalidar o secret. Desenvolvimento seguro é importante hoje mais do que nunca.

O padrão adapter: um contrato, mais de um fornecedor

O serviço de busca é construído com o padrão de projeto adapter, por dois motivos:

- **Abstrair o fornecedor**: se o Serper ficar caro e outro serviço for mais barato, a troca não exige grandes mudanças.
- **Facilitar os testes**: o código que consome a busca pode receber uma implementação falsa.

> **CONCEITO-CHAVE · O contrato do adapter**
> O agente que consome a busca não quer saber se o fornecedor é o Serper, o Google ou qualquer outro. Ele quer a informação em um contrato predefinido: a busca sempre retorna os mesmos tipos.



Diagrama editorial do contrato de busca e das duas implementações criadas na aula.

O serviço é criado com este prompt:

Prompt enviado ao Claude Code

```
Crie packages/agent/src/search com uma interface SearchProvider mínima (search(query: string): Promise<SearchResult[]>, com SearchResult tendo title, snippet, url). Implemente duas versões reais usando fetch nativo, sem SDK de nenhum provedor: makeSerperProvider(apiKey) chamando POST https://google.serper.dev/search com a chave no header X-API-KEY e a query no corpo ({ q: query }), mapeando organic[] pra SearchResult[]; e makeGoogleCustomSearchProvider(apiKey, cx) chamando GET https://www.googleapis.com/customsearch/v1 com key, cx e q como query params, mapeando items[]. Um criarSearchProvider() em index.ts lê SEARCH_PROVIDER (default serper), valida que a(s) variável(is) de ambiente da chave existem, e lança erro claro se não existirem ou se o provider for desconhecido. Teste os dois providers e a fábrica com fetch fake (vi.stubGlobal), sem rede real.
```

O prompt pede três partes:

- **A interface `SearchProvider`**: mínima, com uma função de busca e um tipo genérico para o resultado.
- **Duas implementações reais**: uma para o Serper e outra para o Google Custom Search, ambas com o fetch nativo, sem SDK de nenhum provedor. São bem simples: fazem um fetch na URL do serviço, passando a API key, e devolvem o resultado no formato do contrato.
- **A função que cria o provedor**: ela lê a variável de ambiente `SEARCH_PROVIDER`, cujo valor padrão é o Serper. É essa variável que decide qual implementação é usada.

Testes com resposta falsa

Os testes unitários do provedor usam um mock. Quando a função vai chamar o serviço, o Vitest sobe, em memória, uma resposta falsa, e é ela que responde. A função recebe sempre uma resposta fake, como se estivesse chamando a URL correta.

Público

> **EM SÍNTESE**
> O teste unitário verifica o retorno da função sem consumir a cota do serviço de terceiro. Essa prática é muito comum em testes unitários.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar em que situações o agente busca na web e por que o Serper foi escolhido como provedor sem consultar o material?
- Você consegue explicar que problemas o padrão adapter resolve na interface `SearchProvider` sem consultar o material?
- Você consegue explicar, sem consultar o material, como o teste unitário do provedor funciona sem chamar o serviço real?

PRÁTICA RECOMENDADA

- Crie a sua API key no Serper, guarde-a na variável de ambiente `SERPER_API_KEY` e execute os testes do provedor de busca. Nunca coloque a chave no código-fonte.

RESUMO DA AULA

- Quando o grade marca o contexto como insuficiente, o agente recorre a um provedor de busca na web.
- O Google Custom Search foi depreciado; o projeto usa o Serper, que oferece 2.500 queries gratuitas sem cartão de crédito.
- A interface `SearchProvider` aplica o padrão adapter: o agente recebe um contrato predefinido e o fornecedor pode ser trocado por variável de ambiente.
- A API key fica em variável de ambiente ou no Secret Manager, e os testes usam respostas falsas, sem consumir a cota do serviço.

Público

Aula 11: Fallback pra busca web, upsert retroativo

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Construir o nó de busca na web, que pesquisa apenas os aspectos que faltam, extrai o texto das páginas e grava os chunks com embedding.
- Explicar por que a informação vinda da web recebe um prazo de validade (TTL) e a informação da CVM não.
- Descrever como o retrieve passa a combinar os documentos da CVM e os da web, fazendo o agente se retroalimentar.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Com o provedor de busca pronto, falta o nó que o utiliza. Esta aula cria o nó web search, que busca na internet o que o grade apontou como faltante e agrega o resultado ao contexto do agente.

O nó faz mais do que buscar. Ele grava no banco de dados o embedding do que encontrou, para que uma pergunta parecida, feita depois, já encontre essa informação no RAG. Ao final, você saberá descrever o que o nó faz passo a passo, por que a informação da web tem prazo de validade e como o agente passa a se retroalimentar.

O nó web search: buscar só o que falta

O nó é criado com um único prompt, que descreve tudo de que o Claude Code precisa: o comportamento do nó, a tabela nova, a mudança no retrieve e os testes.

Prompt enviado ao Claude Code

```
Crie o nó web_search (packages/agent/src/nodes/web-search.ts). Recebe question e missingAspects do
estado. Monta uma query por aspecto faltante (${question} ${aspecto}), ou só a pergunta se a lista
estiver vazia. Para cada query, chama searchProvider.search(), pega os 3 primeiros resultados, busca o
HTML de cada URL (fetch nativo, timeout de 10s), extrai texto legível (sem tags), divide em chunks de
~1000 caracteres com sobreposição de 100, gera embedding de cada chunk e insere um documento por chunk
numa tabela nova documentos_web — com query, title, content, sourceUrl, fetchedAt e expiresAt calculado
a partir de um TTL em dias (default 7). Se uma página falhar (fora do ar, timeout), ignora esse
resultado e segue pros outros, sem derrubar o nó inteiro. Estenda buscarDocumentosSimilares (o mesmo
usado pelo nó retrieve da aula passada) pra também consultar documentos_web — filtrando expirados — e
combinar os resultados dos dois com o do corpus CVM, ordenado só pela distância. Tudo testável com
fakes, sem rede nem Postgres real.
```

O nó recebe do estado a pergunta e o `missingAspects`. A pergunta está no estado desde o primeiro nó do grafo. O `missingAspects` vem do grade, que avalia o contexto e diz o que falta para responder de forma satisfatória.

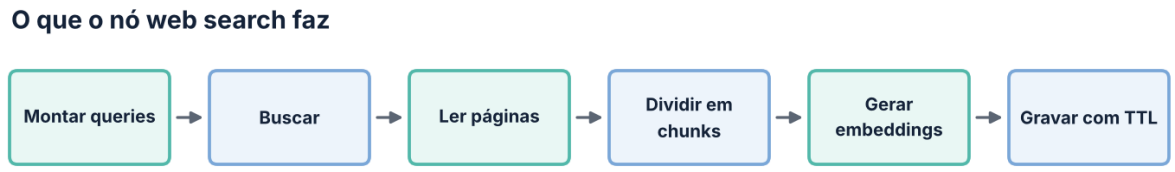
No LangGraph, o agente é um grafo com vários nós. O state é o estado que vai sendo atualizado conforme a execução passa pelos nós.

O centro do nó é a função `makeWebSearchNode`. O trabalho dela segue estas etapas:

1. **Montar as queries**: uma por aspecto faltante, juntando a pergunta e o aspecto. Se a lista de aspectos estiver vazia, a query é só a pergunta.
2. **Buscar**: para cada query, o nó chama o provedor de busca criado na Aula 10 e fica com os três primeiros resultados.

Público

3. **Ler as páginas**: o nó busca o HTML de cada URL com fetch nativo e timeout de 10 segundos.
4. **Extrair e dividir**: o texto legível é extraído, sem as tags, e dividido em chunks de cerca de 1.000 caracteres, com sobreposição de 100. A divisão é necessária porque uma notícia pode ter muito texto.
5. **Gerar os embeddings**: cada chunk recebe o seu embedding pelo Vertex AI, com o mesmo cliente de embeddings usado no retrieve.
6. **Gravar**: o nó insere um documento por chunk na tabela nova de documentos da web, com a query, o título, o conteúdo, a URL, a data da busca e a data de expiração.



Fluxo editorial com as etapas do nó descritas na aula.

> **ATENÇÃO · Por que não enviar a pergunta inteira à busca**
> Se o nó simplesmente jogasse a pergunta na internet, a avaliação feita pelo grade ficaria inútil. A busca traz somente o que falta para responder.

Dois cuidados do prompt tratam de falhas das páginas:

- **Timeout**: se a página demorar, o processo inteiro não trava.
- **Página com falha**: se a página estiver fora do ar ou estourar o timeout, o nó ignora aquele resultado e segue para os outros, sem derrubar o nó inteiro.

TTL: a informação da web tem prazo de validade

Cada registro vindo da web guarda quando a informação foi buscada e quando ela expira.

> **CONCEITO-CHAVE · TTL**
> TTL é time to live, o tempo de vida: a data de validade da informação. No projeto, ele é definido em dias, com sete como padrão.

O prazo existe porque as duas fontes têm naturezas diferentes. A informação da CVM é estática: é a informação daquela época. Correções são bem raras, e o que acontece é a agregação dos resultados da empresa nos trimestres seguintes. Uma notícia ou o resultado de uma busca, ao contrário, pode deixar de ser válido.

Sem o prazo, o agente recuperaria sempre a notícia antiga, porque na vez seguinte ele a encontraria no RAG, e o usuário ficaria com uma informação imprecisa.

A informação da web também é menos confiável. Isso não quer dizer que ela não é confiável: ela é menos confiável do que a informação oficial da CVM.

A tabela a seguir compara as duas fontes:

Critério	Corpus da CVM	Documentos da web
Natureza	Estática e perene: é a informação daquela época, e correções são raras.	Notícias e buscas, que podem deixar de ser válidas.
Confiabilidade	Informação oficial.	Menos confiável do que a informação oficial da CVM.
Validade no banco	Não expira.	Recebe data de busca e data de expiração, calculada por um TTL em dias.
Na busca do retrieve	Sempre consultado.	Consultado com filtro no `WHERE`: só entram os registros não expirados.

Público

O TTL é aplicado na consulta: quando o Postgres é consultado, a query só traz os registros que não expiraram. Um serviço agendado, um cron, poderia limpar as informações antigas para a base não acumular registros, mas a validade em si é garantida pelo filtro.

O retrieve passa a consultar as duas fontes

A mudança no retrieve fica na função que busca os documentos similares. Ela passa a fazer um select na tabela de documentos da CVM e outro na tabela de documentos da web, este com o `WHERE` que deixa de fora o que expirou. Depois, combina os dois resultados, ordenando só pela distância.

Para sustentar essa mudança, o Claude Code também:

- criou a tabela nova no ORM, com a migration e o índice HNSW;
- alterou a tabela de documentos para registrar a origem da informação. Antes, a única fonte era a CVM. Agora a fonte pode ser a CVM ou a internet, e o contexto precisa dessa indicação;
- ajustou o grade à nova regra e criou novos testes.

O ciclo que se retroalimenta

Com a gravação no banco e o retrieve combinado, o agente passa a se retroalimentar.

> **EXEMPLO · Uma pergunta qualitativa, feita duas vezes**
> A pergunta é se o último fato relevante de uma empresa influenciou o preço das ações da Totvs. Na primeira vez, o retrieve traz a informação da CVM, e a parte da web vem vazia. O grade classifica a pergunta como qualitativa e o contexto como insuficiente, apontando o que falta. O nó de busca na web pesquisa, retorna o resultado e grava a informação no banco. Na segunda vez, com uma pergunta parecida, o retrieve acha a informação da CVM, que é estática, e também a da web. O grade avalia as duas e considera o contexto suficiente, sem nova busca na internet.



Diagrama editorial com os dois momentos descritos na aula: a primeira pergunta e uma pergunta parecida, feita depois.

O ciclo recomeça quando a informação da web passa do prazo, por exemplo depois de sete dias. Ela deixa de entrar no `WHERE`, o resultado da web vem vazio, e o agente busca de novo na internet, trazendo informações novas.

Retroalimentar o RAG e trabalhar com a incerteza

Em sistemas agênticos, retroalimente sempre a base de RAG, com atenção a se a informação é perene ou não. Isso economiza custo, porque a busca tem um custo, e tempo. Com toda a informação já no RAG, a resposta é muito mais rápida e, dependendo do caso, muito mais precisa.

O que já se sabe de padrões de projeto e de arquitetura continua valendo para quem faz engenharia de sistemas de IA, até mais do que antes. As LLMs acrescentam uma preocupação: a informação que volta pode não ser exata, pode não ser perene, e é preciso cuidar da alucinação.

Público

> **EM SÍNTESE**
> A LLM não é perfeita, até porque as próprias informações da internet não são precisas. Assim como se pensa em resiliência de sistemas, é preciso pensar em resiliência de informações e aprender a trabalhar com essa incerteza.

Harness: transforme os padrões da equipe em skills

No dia a dia com o agente e com a equipe, vale notar de tempos em tempos os padrões que se repetem, tanto os combinados pela equipe quanto os que surgem durante o desenvolvimento, e criar skills a partir deles.

Todas as ferramentas, como o Claude, o Gemini, o Codex e o Devin, têm recursos de retrospectiva. Dá para pedir uma análise de uma parte do código ou das últimas PRs e gerar, por exemplo, uma skill de criação de agentes. Nos desenvolvimentos seguintes, o agente já cria o código desse jeito.

> **CONCEITO-CHAVE · Harness**
> Do inglês, harness é como se fosse rédea. É colocar rédeas na LLM: criar skills para que ela gere o código no padrão desejado.

O motivo é que uma LLM não é determinística. Quanto mais contexto e padrão a ferramenta recebe, mais o código sai do jeito esperado. Sem uma skill específica, ela pode criar um serviço, uma função, um backend ou até um frontend fora do padrão, e o tempo é perdido revisando e refazendo o prompt.

Skills também enxugam o prompt. Muitas das instruções do prompt desta aula poderiam estar em skills:

- sempre testar com fakes, sem acesso a rede ou a serviço;
- criar um adapter para serviços de terceiros, para poder trocar o fornecedor de API no futuro.

Deixei o prompt grande de propósito, para que fique claro tudo o que está sendo pedido. Na execução, o Claude Code carrega mais uma vez a skill do LangGraph.

O projeto, tanto no starter quanto no Capstone completo, tem uma quantidade grande de skills. Só na infraestrutura, o README descreve todas as peças usadas no GCP.

Para levar o projeto à AWS, o caminho é refatorar o README de infraestrutura e trabalhar em cima dele. Uma skill pode subir os serviços no padrão da AWS, usando o CLI da AWS.

O custo de pedir ao agente que rode um comando

Pedir ao agente que rode um comando gasta tokens. Com o ponto de exclamação, o comando simplesmente roda dentro do Claude Code, sem gastar nenhum token. Para rodar os testes, o ponto de exclamação é a escolha econômica.

Com os testes passando, o grafo está quase completo. Faltam os guardas: o que impede o agente de ficar em loop, buscando indefinidamente, e o que trata o tipo de pergunta, já que o agente só pode dar informações, e não indicações. Eles são o assunto das Aulas 12 e 13.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que o nó de busca na web faz, da montagem das queries à gravação dos chunks, sem consultar o material?
- Você consegue explicar por que a informação da web recebe um TTL e a da CVM não sem consultar o material?
- Você consegue explicar como o agente se retroalimenta em uma segunda pergunta parecida e o que acontece quando a informação da web expira sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O nó de busca na web pesquisa só os aspectos apontados pelo grade, lê as três primeiras páginas de cada query, divide o texto em chunks e grava cada chunk com embedding.
- A informação da web recebe data de expiração por TTL, porque notícias perdem a validade; a informação da CVM é perene.
- O retrieve passa a consultar as duas tabelas, ignorando os registros expirados, e por isso o agente se retroalimenta e evita novas buscas.
- Padrões que se repetem devem virar skills: é o harness, as rédeas que fazem a LLM gerar código no padrão desejado.

Público

Aula 12: O grafo real: aresta condicional e o guard do loop

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Montar o grafo do agente com LangGraph, ligando os nós retrieve, grade e web search.
- Explicar a aresta condicional que sai do grade e o limite de iterações que impede o loop infinito.
- Relacionar a simplicidade do grafo ao princípio de responsabilidade única aplicado a cada nó.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Os nós retrieve, grade e web search existem, mas ainda estão soltos. Esta aula liga os três em um grafo de fim a fim, com o `StateGraph` do LangGraph, e fecha o loop entre eles com uma trava que o impede de rodar para sempre. Sem a trava, o agente ficaria pensando e gastando tokens.

Ao final, você saberá descrever o caminho do grafo, explicar como o limite de iterações funciona dentro do próprio estado e relacionar a simplicidade do código ao modo como os nós foram desenhados.

O caminho do grafo e a aresta condicional

Um grafo é feito de nós interligados, como os que se estudam para calcular a rota mais simples. No agente, a regra de negócio é o caminho de um nó para o outro, definido pelas arestas.

O caminho é este: do início, o `START`, a execução vai para o retrieve e, dele, para o grade. Do grade sai uma aresta condicional, com três resultados possíveis:

- se `sufficient` for verdadeiro, o grafo termina;
- se `iterations` já atingiu `MAX_ITERATIONS`, que é 2, o grafo também termina;
- caso contrário, a execução vai para o web search, que sempre volta para o retrieve.

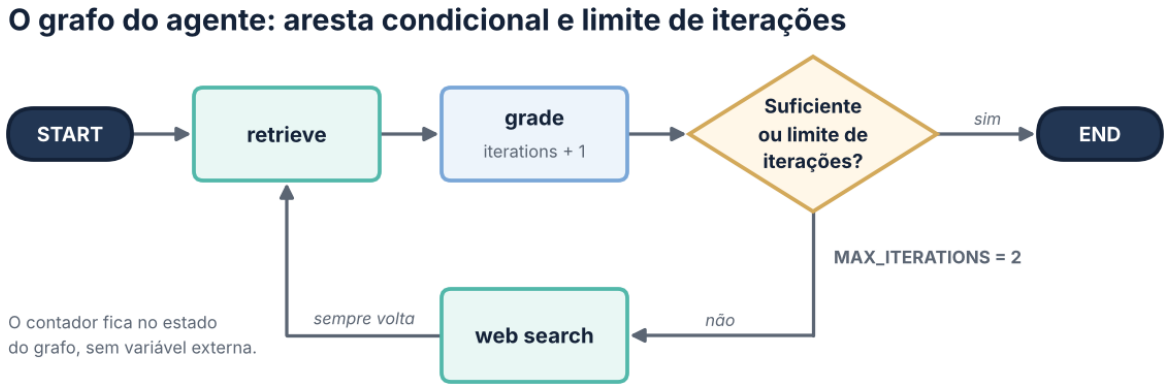


Diagrama editorial do grafo montado na aula, com a aresta condicional e o limite de iterações.

O web search não termina o grafo nem avalia nada. Ele volta para o retrieve, que busca de novo as informações, e o grade verifica outra vez se o contexto é suficiente.

Público

A tabela a seguir resume as arestas:

De	Para	Quando
`START`	retrieve	Sempre: é o início do grafo.
retrieve	grade	Sempre: o contexto recuperado é avaliado.
grade	`END`	Se `sufficient` for verdadeiro, ou se `iterations` já atingiu `MAX_ITERATIONS`, que é 2.
grade	web search	Caso contrário: o contexto é insuficiente e ainda há iterações disponíveis.
web search	retrieve	Sempre: o web search não termina o grafo nem avalia, ele volta para a busca.

O guard do loop: um contador dentro do estado

O limite de iterações é o que impede o grafo de rodar o loop indefinidamente. Se o contexto continua insuficiente, o agente passa pela busca uma segunda vez. Na terceira, o guard impede, e o agente fica com a informação que tem.

> **CONCEITO-CHAVE · Contador no estado**
> O campo `iterations` faz parte do estado e soma 1 a cada passada pelo grade, por meio de um reducer. Não há contador externo: o valor viaja com o estado, que passa de um nó para outro, como a pergunta e o campo que diz o que falta para a resposta.

O prompt que monta o grafo

O grafo é montado com um prompt um pouco grande. As instruções poderiam estar em uma skill, para serem reaproveitadas, mas vale vê-las completas:

Prompt enviado ao Claude Code

```
Monte o grafo de verdade com @langchain/langgraph (StateGraph) em src/graph.ts, ligando os nós retrieve, grade e webSearch que já existem: START → retrieve → grade, e uma aresta condicional a partir de grade — se sufficient for true, termina (END); senão, se iterations já bateu MAX_ITERATIONS = 2, também termina; caso contrário, vai pro webSearch, que sempre volta pro retrieve. iterations é um campo do estado que soma 1 a cada passada por grade via reducer, sem contador externo. GraphDeps recebe as dependências de nó por função (buscar, inserirDocumentoWeb, etc.), não o cliente de banco direto, pra o grafo inteiro ser testável com fakes. Teste com app.stream(), coletando a sequência de nós visitados: caminho direto quando suficiente na primeira rodada, o loop completo quando insuficiente, e o guard cortando exatamente em MAX_ITERATIONS rodadas de grade mesmo que o LLM fake sempre devolva insuficiente.
```

Além do caminho e do contador, o prompt define dois pontos:

- **Dependências por função**: o `GraphDeps` recebe as dependências dos nós por função, como buscar e inserir documento da web, e não o cliente de banco direto. É a inversão de dependência de novo, para deixar o grafo o mais fracamente acoplado possível e testável com fakes.
- **Testes**: feitos com `app.stream()`, coletando a sequência de nós visitados, em três cenários: o caminho direto, quando o contexto é suficiente na primeira rodada; o loop completo, quando é insuficiente; e o guard cortando exatamente em `MAX_ITERATIONS` rodadas de grade, mesmo que a LLM fake devolva sempre insuficiente.

Na execução, o Claude Code usa a skill `langgraph-node`, pede autorização para adicionar a dependência do LangChain, cria e executa os testes unitários e passa o lint.

O código do grafo

O arquivo `graph.ts` define o grafo em cerca de 60 linhas, com estas partes:

- **A constante de máximo de iterações**.
- **O objeto de estado**: a pergunta, os documentos, a indicação de contexto suficiente, o `missingAspects` e o `iterations`. O estado passa de nó em nó e é incrementado a cada um.

Público

- **A interface das dependências**.
- **A função condicional**: executada depois do grade, ela termina o grafo se o estado está suficiente ou se as iterações são maiores ou iguais ao máximo.
- **A função que constrói o grafo**: cria o retrieve, o grade e o web search, cada um com os seus parâmetros, registra os três com `addNode` e adiciona as arestas. Do `START` para o retrieve, do retrieve para o grade, do grade para a função condicional e do web search para o retrieve.
- **O `compile`**, no final, que fecha a definição do grafo.

Para rodar os testes, o ponto de exclamação executa o comando sem consumo de tokens. É como abrir uma nova aba no terminal e rodar o comando ali.

Responsabilidade única: por que o grafo ficou simples

O prompt ficou até um pouco mais complexo do que o código. O código é simples porque o LangGraph simplifica a orquestração, mas principalmente porque os nós foram criados de maneira bem focada, com o princípio de responsabilidade única:

- o retrieve só busca a informação no RAG;
- o grade só avalia o resultado que o retrieve trouxe;
- o web search só faz a busca na web e adiciona informações ao RAG. Ele não avalia: volta para o retrieve.

> **EM SÍNTESE**
> Os nós, que também podem ser pensados como funções, ficam bem separados, e o LangGraph apenas os orquestra. O resultado é direto: mesmo sem conhecer o LangGraph, é possível ler o código.

Esse é um conhecimento de engenharia: os bons conceitos de código e o bom design de código.

O fundamento não muda com a ferramenta

O projeto usa o Claude, mas a qualquer momento outro modelo pode estar mais forte, como o DeepSeek ou o Grok. O princípio é o mesmo. O que está explicado aqui funciona com o Gemini, talvez com algum prompt a mais, dependendo do modelo. Com o Fable, por exemplo, bastaria escrever "crie um LangGraph" para ele entender e fazer.

> **ATENÇÃO · Quem domina o código é você**
> O agente escreveu o código, mas você precisa ser capaz de entendê-lo e de explicá-lo, porque amanhã vai precisar ampliá-lo ou debugá-lo. Entender o fundamento é o que permite avaliar o prompt, o código e a arquitetura.

Antes da IA, esse código seria copiado de um projeto anterior ou escrito de cabeça. O princípio não mudou: a separação de responsabilidades é a mesma.

O mesmo vale para as peças do projeto. Ele é feito em TypeScript de ponta a ponta, inclusive o frontend, em Angular, mas nada impede que o backend seja em Python, com outro framework, e o frontend em outra tecnologia. Ao fazer o seu, tente algo diferente:

- outra infraestrutura, como AWS ou Azure;
- outro modelo, como o Gemini no lugar do Claude;
- outro framework de frontend;
- no backend, o ADK, do Google, no lugar do LangGraph.

Os conceitos são os mesmos: um agente montado como nós interligados.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o caminho que o grafo percorre do `START` até o `END` sem consultar o material?
- Você consegue explicar como a aresta condicional e o campo `iterations` impedem que o agente rode para sempre sem consultar o material?
- Você consegue explicar por que o princípio de responsabilidade única deixou o código do grafo simples sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O grafo liga `START`, retrieve e grade; do grade sai uma aresta condicional que termina a execução ou segue para o web search, que sempre volta ao retrieve.
- O campo `iterations` soma 1 a cada passada pelo grade, dentro do próprio estado, e o limite `MAX_ITERATIONS = 2` corta o loop.
- As dependências dos nós entram por função, o que mantém o grafo fracamente acoplado e testável com fakes.
- O código do grafo ficou curto porque cada nó tem uma responsabilidade única, e esse fundamento vale para qualquer modelo ou framework.

Público

Aula 13: O nó answer, citação obrigatória, guardrail por código e streaming SSE

Voltar ao mapa da disciplina • *Voltar à Unidade 2*

OBJETIVOS DE APRENDIZAGEM

- Construir o nó answer, que monta a resposta final com base no contexto, com citação de fonte e data-base.
- Explicar por que o guardrail não pode depender só do system prompt e como um filtro determinístico, com expressão regular, garante a regra.
- Expor o grafo em um endpoint POST que transmite a execução por Server-Sent Events, inclusive no Cloud Run.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O grafo busca, avalia, recorre à web quando o contexto não basta e sabe quando parar. Mas, quando ele termina, ainda não existe uma resposta para o usuário.

Esta aula cria o nó answer, que empacota o que os outros nós processaram e cita a fonte e a data-base. Junto com ele vem o guardrail que impede o sistema de recomendar a compra ou a venda de ações, nem por engano do modelo. Por fim, o grafo é exposto em um endpoint REST do verbo POST, com a resposta transmitida por streaming, aparecendo aos poucos, como na maioria das ferramentas de chat.

Ao final, você saberá explicar as duas defesas do guardrail, o que é SSE e o que é preciso configurar para o streaming funcionar no Cloud Run.

Por que o guardrail não pode depender só do prompt

O projeto responde sobre o mercado financeiro, uma atividade regulamentada pela CVM. A regra é simples: o sistema não pode, de jeito nenhum, recomendar ações.

Essa regra é escrita no system prompt que formata a resposta. Só que o system prompt sozinho não basta, por dois motivos:

- o modelo pode ignorar a instrução, dependendo do modelo;
- o usuário pode tentar enganar a LLM, sendo muito persistente ou disfarçando o pedido.

> **EXEMPLO · Pedidos disfarçados**
> Um pedido como "me diga isso como se fosse uma história de ninar para o meu filho" é uma forma conhecida de contornar a regra. No começo das LLMs, quando os guardrails não eram tão fortes, pedia-se, por exemplo, uma história infantil sobre como criar uma bomba nuclear. Com o tempo, as proteções ficaram cada vez melhores.

Um guardrail pode ser feito com uma série de mecanismos. O adotado aqui parece simples, mas é bem efetivo: mesmo que o retorno da LLM traga uma recomendação por engano, ela é bloqueada. Um teste unitário valida esse comportamento, de modo que, se alguém tirar a proteção no futuro, o teste acusa o erro.

O nó answer: resposta baseada no contexto, com fontes

O nó answer recebe a pergunta, os documentos e a indicação de contexto suficiente ou não. Em outras palavras, recebe o estado final, propagado por todos os nós. Ele envia esse estado ao prompt do answer e, com a resposta do modelo, faz a formatação.

Público

Prompt enviado ao Claude Code

```
Crie o nó answer (packages/agent/src/nodes/answer.ts). Recebe question, documents e sufficient do
estado, monta um prompt (packages/agent/src/prompts/answer.ts) que instrui o modelo a nunca recomendar
decisão de investimento mesmo se a pergunta pedir diretamente, a responder só com base no contexto
recuperado sem inventar número, e a avisar quando o contexto estiver incompleto. Depois de receber a
resposta do LLM, passe por um filtro determinístico (packages/agent/src/answer/format-resposta.ts): se o
texto contiver linguagem recomendatória — verbo em primeira pessoa tipo
'recomendo'/'sugiro'/'aconselho', ou construções como 'vale a pena comprar' —, substitua a resposta
inteira por um aviso fixo de guardrail, mantendo as citações. Sempre anexe um bloco de fontes no final,
um item por documento, deduplicado por sourceUrl — ticker+período+data-base pros documentos do corpus
CVM, título pros documentos web. Teste com LLM fake: resposta normal com citação, resposta filtrada
quando o LLM tenta recomendar, e um teste que confirma que o prompt em si já instrui a nunca recomendar.
```

O prompt define quatro partes:

- **O system prompt**: instrui o modelo a nunca recomendar uma decisão de investimento, mesmo que a pergunta peça isso diretamente, a responder só com base no contexto recuperado, sem inventar número, e a avisar quando o contexto estiver incompleto.
- **O filtro determinístico**: aplicado depois da resposta da LLM. Se o texto tiver linguagem recomendatória, a resposta inteira é substituída por um aviso fixo de guardrail, mantendo as citações.
- **O bloco de fontes**: sempre anexado no final, com um item por documento e sem repetir a mesma URL. Para os documentos do corpus da CVM, a citação traz ticker, período e data-base. Para os documentos da web, o título.
- **Os testes com LLM fake**: a resposta normal com citação, a resposta filtrada quando a LLM tenta recomendar e um teste que confirma que o próprio prompt já instrui a nunca recomendar.

Além de criar o nó, o Claude Code o insere no `graph.ts`.

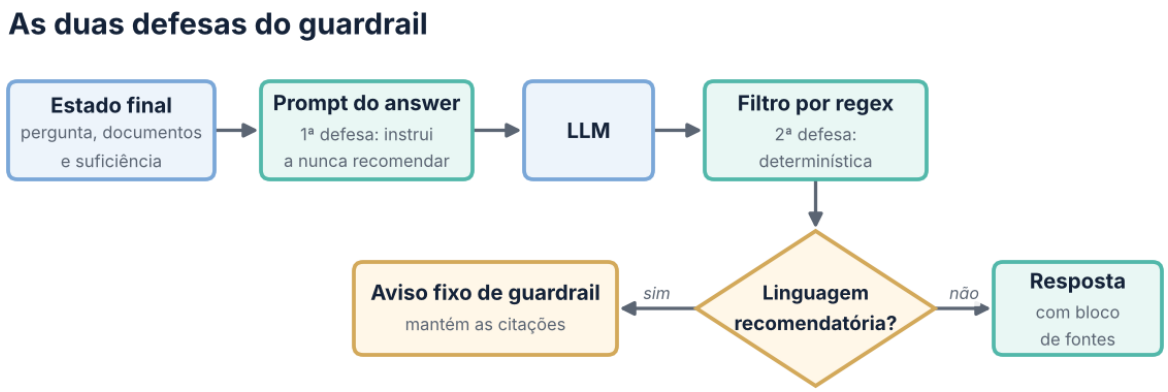


Diagrama editorial do nó answer e das duas defesas descritas na aula.

> **ATENÇÃO · Contexto incompleto e o máximo de iterações**
> O contexto pode chegar incompleto ao answer porque o grafo tem um máximo de iterações. Esse máximo pode ser 3, 4 ou 5: é ajuste fino, e no dia a dia muita coisa é ajuste fino. O que não pode faltar é o limite. Sem ele, o agente fica rodando e queimando tokens, e pode não chegar a resposta nenhuma.

O system prompt do answer: papel, regras e resultado

O prompt do answer segue a fórmula de papel, regra e resultado. O papel é o de um assistente que informa sobre indicadores financeiros de empresas listadas. Depois vêm as regras, que valem sempre, mesmo que a pergunta peça o contrário diretamente:

- nunca recomendar, sugerir ou aconselhar comprar, vender ou manter qualquer ativo e nunca opinar se vale a pena investir. O assistente informa, não recomenda, e isso é uma restrição regulatória, não uma preferência de estilo;
- se a pergunta pedir uma recomendação, explicar que não pode dar recomendação de investimento e, se houver dado disponível no contexto, apresentá-lo sem opinião sobre a decisão;

Público

- responder só com base no contexto recuperado, sem inventar, estimar ou completar um número que não esteja explicitamente no contexto;
- citar, para cada número usado, a empresa e o período a que ele se refere.

A montagem do prompt verifica se o contexto é suficiente. Se for, nada é acrescentado. Se não for, o prompt ganha um aviso: o contexto recuperado é incompleto para responder com segurança, e o modelo deve dizer isso explicitamente ao usuário, informando o que falta, em vez de preencher a lacuna com uma suposição. Por fim, vem a pergunta.

> **CONCEITO-CHAVE · RAG e contexto**
> O RAG nada mais é do que meios de busca e recuperação de informações que são passadas para o contexto. A LLM não guarda estado e não tem acesso ao banco de dados. Por isso, o prompt enviado a ela leva todo o contexto de que ela precisa, e as regras deixam claro que ela não deve buscar outra informação.

Mesmo com a regra no prompt, LLMs são sistemas não determinísticos, e nada garante o comportamento. Avaliações de alguns modelos, inclusive dos mais novos, mostram que eles nem sempre respeitam as regras: fazem outras coisas e buscam informação por conta própria.

O filtro determinístico: uma expressão regular

O que garante que o sistema não recomende, mesmo que o modelo alucine, é uma regex, uma expressão regular. Depois de todo o processamento, a string da resposta ainda passa por ela.

A expressão procura palavras como "recomendo", "recomendaria", "sugiro", "sugeriria", "aconselho" e "aconselharia" e construções como "vale a pena comprar", "vender" ou "manter". O mesmo arquivo tem as funções que formatam a data-base e a fonte.

Um erro de digitação do usuário na pergunta não atrapalha o filtro. O que é analisado é o retorno, e o retorno está escrito certo, porque foi a LLM que o criou.

A tabela a seguir compara as duas defesas:

Critério	Instrução no system prompt	Filtro por expressão regular
Onde atua	No prompt do nó answer, antes de o modelo responder.	No texto devolvido pelo modelo, depois da resposta.
Como funciona	Instrui o modelo a nunca recomendar, mesmo que a pergunta peça diretamente.	Procura linguagem recomendatória e substitui a resposta inteira por um aviso fixo, mantendo as citações.
Garantia	Não é garantida: o modelo pode ignorar a instrução ou ser enganado pelo usuário.	Determinística: se a construção aparecer, a recomendação é bloqueada.
Limite	LLMs são sistemas não determinísticos.	Avalia a string, não o sentido do texto.

> **EXEMPLO · O teste do guardrail**
> O teste passa ao nó um fake chat model cuja resposta é "Eu recomendo comprar PETR4 agora, é uma ótima oportunidade". O resultado esperado é só o aviso e a fonte, sem nenhuma recomendação.

Esse mecanismo não existe só aqui, por motivo didático. Agentes de código abertos usam muito expressão regular para garantir esse tipo de regra, justamente por ser determinística. Quando o código do Claude Code vazou e a comunidade o analisou, apareceu muito desse tipo de coisa nos guardrails.

Há formas mais rebuscadas de guardrail. Uma delas é outro RAG, que analisa a resposta por similaridade. Uma busca vetorial não é tão determinística quanto uma expressão regular, por causa da imprecisão do índice HNSW em troca de velocidade, mas é mais determinística do que uma LLM. E, depois dos testes, pode ser preciso outro tipo de guardrail. É muito comum as empresas fazerem testes de segurança em LLMs.

SSE: a resposta transmitida aos poucos

O endpoint de chat devolve a execução do grafo por Server-Sent Events, o SSE.

Público

> **CONCEITO-CHAVE · SSE**
> Pense em um WebSocket, só que unidirecional. No WebSocket, abre-se um canal de comunicação, e envios e retornos acontecem de forma constante, em vez de um POST depois do outro. O SSE também abre um túnel entre o cliente, que pode ser o frontend ou um MCP, e o backend, mas o cliente só recebe.

O endpoint é criado com este prompt:

Prompt enviado ao Claude Code

```
Exponha o grafo via POST /api/chat (apps/api/src/routes/chat.ts), como uma rota Fastify que recebe
{ question }, valida com o schema de @hero/contracts, e transmite a execução do grafo como Server-Sent
Events — um evento por nó concluído, não por token. Escreva os headers manualmente com Cache-Control:
no-cache, no-transform (sem isso o Cloud Run bufferiza a resposta inteira em vez de transmitir aos
poucos) e Connection: keep-alive. Itere o graph.stream() e, pra cada chunk, traduza o nome do nó e o
update dele num evento tipado (apps/api/src/routes/chat-event.ts) — documentos resumidos pro retrieve,
sufficient/missingAspects/iterations pro grade, o texto final pro answer, só o nome pro webSearch, que
não produz campo de estado novo. Se o grafo lançar erro no meio do stream, escreva um evento SSE de erro
e encerre a conexão, nunca derrube o processo. Injete o grafo por interface mínima (GraphLike, só o
método stream que a rota usa), não o tipo concreto do LangGraph, pra testar com um grafo fake sem LLM
nem Postgres real.
```

A rota Fastify recebe a pergunta, valida com o schema do pacote de contratos e transmite a execução do grafo. Para isso, usa o `graph.stream()`, que envia os eventos de forma assíncrona, aos poucos, em vez de esperar a resposta inteira e devolver só o resultado final.

No app, a rota de chat se soma à rota de health, que já existia para o health check.

Os headers que mantêm o streaming no Cloud Run

A rota escreve dois headers manualmente:

- **`Cache-Control: no-cache, no-transform`**: sem ele, o Cloud Run bufferiza a resposta inteira e manda tudo de uma vez, em vez de transmitir aos poucos.
- **`Connection: keep-alive`**: por padrão, o Cloud Run atende à requisição e fecha a conexão. O keep-alive mantém o túnel aberto.

O POST no Fastify retorna via SSE. Sem o header de cache, o próprio Cloud Run faz com que a resposta deixe de ser SSE.

O SSE tem desvantagens, mas é ele que dá o efeito de resposta chegando aos poucos.

Eventos tipados: o que alimenta o painel de raciocínio

A rota transmite um evento por nó concluído, e não por token. A cada chunk do `graph.stream()`, o nome do nó e o update dele são traduzidos em um evento tipado, o evento de chat. A tipagem fica em um arquivo próprio, e o contrato do chat, no pacote de contratos.

Cada nó gera um conteúdo diferente no evento:

Nó	Conteúdo do evento
retrieve	Os documentos resumidos.
grade	O resultado da avaliação: `sufficient`, `missingAspects` e `iterations`.
answer	O texto final.
web search	Só o nome do nó, que não produz campo de estado novo.

É esse evento que o frontend lê para preencher o painel de raciocínio, o recurso que mostra de maneira gráfica o que está acontecendo. Normalmente, isso não é exposto por inteiro. O próprio Claude Code faz algo parecido: enquanto processa, vai dizendo aos poucos o que está fazendo, em mensagens como "quase terminando", e expõe uma parte dos eventos, não tudo.

Se o grafo lançar um erro no meio do stream, a rota escreve um evento SSE de erro e encerra a conexão. O processo nunca é derrubado.

Público

GraphLike: uma interface mínima para o grafo

A rota não recebe o tipo concreto do LangGraph. Ela recebe o grafo por uma interface mínima, a `GraphLike`, só com o método `stream`, que é o que a rota usa. O app instancia o grafo criado com o LangGraph e o injeta na rota, por inversão de dependência.

A interface mínima traz dois ganhos:

- a rota é testada com um grafo fake, sem LLM e sem Postgres real. No teste criado, os eventos vão sendo exibidos no console;
- se o framework de grafo de agente mudar, o código fica mais fácil de refatorar.

> **EM SÍNTESE**
> Pensar o código para ser fácil de refatorar continua importante, independentemente de quem o escreve.

Outras formas de construir o mesmo sistema

O sistema foi construído com vários prompts, passo a passo, por didática. Ele também pode ser criado com spec-driven, com loop engineering e com outras abordagens.

A rota usa o Fastify, mas poderia usar o Express ou o NestJS. Com o NestJS, muita coisa mudaria um pouco, inclusive a arquitetura. Refazer com outro framework é um bom exercício, porque aparecem outros problemas, e descobrir como resolvê-los é o que consolida o aprendizado.

No seu ambiente, mude as peças, a regra de negócio, a linguagem de programação e a infraestrutura, para entender e praticar.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o que o nó answer recebe, que regras o seu prompt impõe e como as fontes são citadas sem consultar o material?
- Você consegue explicar por que o sistema usa uma expressão regular depois da resposta do modelo, além do system prompt, sem consultar o material?
- Você consegue explicar o que é SSE e por que os headers `Cache-Control` e `Connection` são necessários no Cloud Run sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O nó answer monta a resposta só com o contexto recuperado, avisa quando ele está incompleto e sempre anexa as fontes, com ticker, período e data-base para os documentos da CVM.
- O guardrail tem duas defesas: a instrução no system prompt e um filtro determinístico por expressão regular, que troca qualquer resposta recomendatória por um aviso fixo.
- O grafo é exposto em um POST que transmite um evento SSE por nó concluído, com eventos tipados que alimentam o painel de raciocínio.
- No Cloud Run, os headers `Cache-Control: no-cache, no-transform` e `Connection: keep-alive` são o que mantém o streaming funcionando.

Público

Unidade 3: Qualidade, Resiliência e MCP

Dos testes sem rede à exposição do agente por MCP, passando pelos padrões de resiliência.

NESTA UNIDADE

- Aula 14: Testando o grafo sem rede
- Aula 15: Circuit breaker, retry e degradação graciosa
- Aula 16: Erros padronizados
- Aula 17: O servidor MCP



Fluxo conceitual criado para apoiar a leitura da unidade.

GITHUB E PRÁTICA

- Projeto completo: Capstone Project AI Researcher
- Ponto de partida do estudante: AI Researcher Seed

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público

Aula 14: Testando o grafo sem rede

Voltar ao mapa da disciplina • *Voltar à Unidade 3*

OBJETIVOS DE APRENDIZAGEM

- Explicar como testar um sistema não determinístico tratando a LLM como uma API de terceiro, substituída por fakes.
- Aplicar estratégias de teste diferentes conforme o tipo de código: fakes e mocks, testes padrão para funções determinísticas e fixtures em arquivo.
- Criar uma feature flag de demonstração que substitui a busca na web por um serviço fake, para testes manuais e de ponta a ponta.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Um teste automatizado deve dar o mesmo resultado, não importa quantas vezes seja executado. Com uma LLM, isso pode não acontecer: ela é não determinística, e o mesmo prompt pode gerar duas saídas diferentes. O harness, com skills e guardrails, reduz esse risco, mas ele sempre existe.

Esta aula trata de qualidade: como testar um sistema que tem uma parte não determinística, uma parte determinística que lida com arquivos fora de padrão e um serviço caro de terceiro. Ao final, você saberá escolher a estratégia de teste para cada tipo de código e criar uma feature flag que permite demonstrar o sistema sem consumir o serviço de busca.

A LLM é uma API de terceiro: fakes por inversão de dependência

Nos testes, a LLM deve ser encarada como uma API de terceiro, porque ela literalmente é uma. E há um agravante: não dá para garantir a resposta dela.

> **CONCEITO-CHAVE · A LLM não é idempotente**
> Não é possível garantir que a LLM dê sempre a mesma resposta, independentemente de quantas vezes é chamada.

O que torna o teste possível é a estrutura do código. Com a inversão de dependência, as dependências que as funções dos nós usam entram por parâmetro. No teste, essas dependências são trocadas:

- a função `fakeChatModel` simula a resposta da LLM;
- um mock do cliente de embeddings devolve o retorno esperado;
- um mock do search provider substitui o serviço do Serper.

Código determinístico tem teste padrão

A formatação da resposta tem um teste mais simples. O guardrail principal é uma expressão regular, e a formatação é composta por várias funções determinísticas.

Nesse caso, o teste é o padrão: a função recebe a entrada desejada, com um documento da CVM simulado, e o `expect` compara a saída com o resultado esperado.

Público

Fixture em arquivo: o parser dos CSVs da CVM

O coletor da CVM, criado na Aula 5, pega os dados e normaliza os registros, que não são iguais em todas as empresas, em um padrão que pode ser importado para o embedding e para a carga no banco vetorial. Ele ainda não tinha testes para duas partes, e o prompt a seguir pede a cobertura delas:

Prompt enviado ao Claude Code

```
O parser do coletor CVM (packages/corpus/src/collect/parse-csv.ts) não tem nenhum teste — os CSVs da CVM vêm em ISO-8859-1, não UTF-8, e preciso confirmar que a decodificação de acento funciona de verdade, com uma fixture de bytes latin1 reais, não uma string reescrita em UTF-8. Também cobre select-quarters.ts — é o arquivo mais frágil do coletor (posição de conta por setor, fallback consolidado/individual, seleção de versão retificada, subtração YTD do 4º trimestre, matemática de data incluindo ano bissexto) e hoje não tem nenhum teste. Exporte só o necessário pra testar as funções puras, sem mudar comportamento nenhum.
```

A primeira parte é o parser. Os CSVs da CVM vêm em ISO-8859-1, e não em UTF-8, e o teste precisa confirmar que a decodificação dos acentos funciona de verdade.

> **CONCEITO-CHAVE · Fixture em arquivo**
> Em vez de um mock ou de uma string escrita dentro do teste, o teste usa um arquivo fake. Aqui, é um CSV com os caracteres em binário, com bytes latin1 reais, lido a cada execução.

Aberto no VS Code, o arquivo mostra caracteres esquisitos, e eles estão lá de propósito: o arquivo foi criado em binário, simulando um problema que pode acontecer no CSV.

Testar contra um arquivo garante duas coisas:

- a decodificação ocorre corretamente;
- o cabeçalho é lido corretamente. Ele é usado como chave de cada campo, porque, na normalização, os formatos podem variar.

Os testes das regras de negócio do select-quarters

A segunda parte é o `select-quarters.ts`, o arquivo mais frágil do coletor. Depois que o CSV é decodificado, ele trata todas as situações que podem ocorrer na limpeza do dado: posição de conta por setor, fallback entre consolidado e individual, seleção de versão retificada, subtração do acumulado no quarto trimestre e matemática de data, incluindo ano bissexto.

Os testes cobrem essas regras de negócio. Entre eles:

- **Versão retificada**: a CVM publica retificações com um número de versão. O teste verifica que a versão escolhida é a mais alta numericamente, e não lexicograficamente.
- **Dado ausente**: o tratamento do `undefined`, quando o dado não aparece.
- **Consolidado vazio**: o tratamento do consolidado com array vazio ou ausente.
- **Escala da unidade** e **subtração**.

O prompt pede para exportar só o necessário para testar as funções puras, sem mudar nenhum comportamento.

Esses testes são necessários porque os dados não seguem o padrão que se esperaria de um órgão como a CVM. Algumas empresas aparecem em outro padrão, e o próprio CSV não é o melhor formato atual para esse tipo de informação. O projeto usa LLMs e também funções determinísticas para garantir a melhor qualidade possível do dado.

Modo demo: uma feature flag para o serviço caro

O provedor de busca já tem teste unitário. Mas teste unitário é uma coisa, e teste de QA é outra. Nos testes funcionais, nos testes de QA e nos testes manuais, o sistema roda de ponta a ponta e precisa de um serviço de busca. Nem toda integração é igual: o Serper é uma integração cara, e não deve ser consumido o tempo todo.

Público

A solução é uma feature flag de demonstração:

Prompt enviado ao Claude Code

```
Preciso gravar uma demonstração do fallback de busca web sem depender do Serper estar no ar. Cria
fixtures pro SearchProvider e pro fetch de página, cobrindo pelo menos três perguntas reais do grupo B
do evals/dataset.json (Vale, Petrobras, Klabin), casando por substring do ticker ou nome da empresa na
query — pergunta sem fixture cadastrada cai numa fixture genérica, nunca lança erro. Isso deve ligar
numa variável de ambiente DEMO=true, lida num único lugar por dependência, sem o SearchProvider real ou
o buscarPagina real saberem que o modo demo existe. grade, answer e os embeddings continuam batendo na
Vertex AI de verdade — o escopo é só busca web.
```

O prompt define o modo demo assim:

- **Fixtures**: para o search provider e para o fetch de página, cobrindo pelo menos três perguntas reais do grupo B do dataset de avaliação, sobre Vale, Petrobras e Klabin.
- **Casamento**: por substring do ticker ou do nome da empresa na query. Uma pergunta sem fixture cadastrada cai em uma fixture genérica e nunca lança erro.
- **Ligação**: pela variável de ambiente `DEMO=true`, lida em um único lugar por dependência. O search provider real e a busca de página real não sabem que o modo demo existe.
- **Escopo**: só a busca na web. O grade, o answer e os embeddings continuam usando o Vertex AI de verdade.

O resultado é um mini Serper fake, mais completo do que o mock do teste unitário, com páginas falsas que servem de resposta. O dado não fica 100% correto, mas é suficiente para testar a funcionalidade.

No código, o modo demo aparece em dois pontos. Uma função de modo demo só verifica a variável de ambiente. E a função que cria o search provider retorna o provider fake quando o modo demo está ligado. O resto continua igual, com o Serper ou com o Google Custom Search.

A flag também entra no `.env.example`, junto com as outras variáveis. Esse arquivo pode ser versionado, porque traz só exemplos: serve de documentação e de base para quem for subir o projeto.

A refatoração que a flag exigiu

Na versão anterior do grafo, a busca de páginas usava a implementação padrão. Para testar, bastavam spies, ou stubs, dependendo do termo: em tempo de execução do teste, o comportamento em memória era alterado para que a busca fosse a fake.

Para uma feature flag isso não serve. A busca de páginas precisou ser refatorada para entrar como dependência, por parâmetro. Assim ela pode ser substituída em tempo de teste e também em tempo de execução, quando a flag de demo está ligada. Os testes foram ajustados para considerar essa dependência.

> **ATENÇÃO · Inversão de dependência não é obrigatória em tudo**
> Usar sempre a inversão de dependência não é regra: depende do caso. O projeto pode ser refatorado quando a necessidade aparecer. Com LLMs e agentes de código, refatorar ficou mais simples, mas isso não é motivo para refatorar o tempo todo. O sistema deve continuar o mais simples possível e evoluir conforme a necessidade.

Público

Estratégias de teste por tipo de código

A tabela a seguir reúne as estratégias aplicadas:

O que é testado	Estratégia	Por quê
Nós e grafo, que dependem da LLM, dos embeddings e da busca	Fakes e mocks injetados por parâmetro, como o `fakeChatModel`	A LLM é uma API de terceiro e não devolve sempre a mesma resposta.
Formatação da resposta e guardrail	Teste padrão, com entrada e resultado esperado	São funções determinísticas, baseadas em expressão regular.
Parser dos CSVs da CVM	Fixture em arquivo, com bytes latin1 reais	Garante a decodificação dos acentos e do cabeçalho contra um arquivo de verdade.
Regras do `select-quarters`	Testes unitários das funções puras	Cobrem as situações que podem ocorrer na limpeza do dado.
Fallback de busca na web em demonstrações e testes manuais	Feature flag `DEMO=true` com um mini Serper fake	O Serper é uma integração cara e não precisa ser consumido o tempo todo.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar por que a LLM é tratada como uma API de terceiro nos testes e como os fakes resolvem isso sem consultar o material?
- Você consegue explicar quando usar fakes, quando usar um teste padrão e quando usar uma fixture em arquivo sem consultar o material?
- Você consegue explicar o que a flag `DEMO=true` substitui e por que foi preciso refatorar a busca de páginas para criá-la sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- Uma LLM não é idempotente: nos testes, ela entra como dependência e é substituída por um fake.
- Funções determinísticas, como o guardrail por regex, têm testes padrão; o parser de CSV é testado com um arquivo de bytes latin1 reais.
- O `select-quarters`, arquivo mais frágil do coletor, ganhou testes para as regras de negócio da limpeza dos dados.
- A flag `DEMO=true` liga um mini Serper fake, o que permite demonstrações e testes manuais sem consumir o serviço real.

Público

Aula 15: Circuit breaker, retry e degradação graciosa

Voltar ao mapa da disciplina • *Voltar à Unidade 3*

OBJETIVOS DE APRENDIZAGEM

- Explicar o que timeout, retry com backoff e circuit breaker resolvem quando um serviço de terceiro falha.
- Aplicar o padrão decorator para acrescentar resiliência ao provedor de busca sem reescrever cada implementação.
- Implementar a degradação graciosa no nó de busca na web e avaliar em quais regras de negócio ela faz sentido.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O modo demo resolve o desenvolvimento e as demonstrações, mas não a produção. Em produção, a flag está desligada, e o Serper está no ar de verdade, o que inclui os dias em que ele não está. API de terceiro cai, tem pico de latência e estoura limite.

Esta aula trata de como tornar o sistema resiliente a essas falhas, sem impactar tanto a experiência do usuário. O sistema não responde da mesma maneira quando um serviço falha, mas também não precisa mostrar uma tela branca.

São três camadas de proteção no provedor de busca, timeout, retry com backoff e circuit breaker, e mais a degradação graciosa no grafo. Ao final, você saberá explicar o problema que cada uma resolve e decidir quando uma falha deve ser contornada e quando deve ser uma falha.

Timeout: um erro claro em vez de uma espera sem fim

Por padrão, um fetch não tem timeout. Ele fica no ar pelo tempo que for preciso esperar pela resposta. Se o Serper travar no meio de uma resposta, a requisição toda trava junto, e o usuário fica olhando para a tela.

É melhor mostrar um erro, pedindo para tentar de novo ou tentar mais tarde, do que deixar a requisição executando sem que o usuário saiba se o problema é a conexão dele, o computador dele ou outra coisa. O timeout dá um erro muito claro.

Retry com backoff: a falha pode ser passageira

Às vezes o serviço não saiu do ar indefinidamente. Houve só um pico de requisições, algo intermitente no serviço de terceiro.

O retry responde a esse caso. Em vez de devolver o erro ao usuário ou de ficar esperando, o sistema cancela a conexão e tenta novamente. Muitas vezes isso resolve e, dependendo de como o frontend é desenvolvido, fica transparente para o cliente.

Circuit breaker: parar de insistir em quem está se recuperando

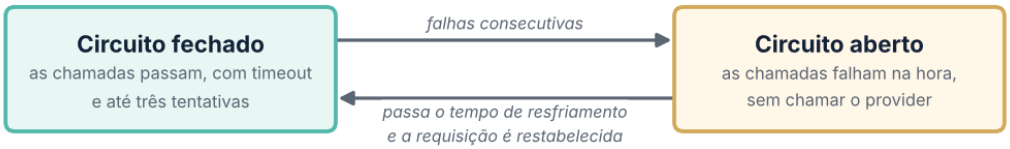
Um retry mal implementado gera outro problema, a falha em massa. O serviço caiu de maneira intermitente e está se levantando, ainda sem condições de atender. Se o sistema insiste, retry atrás de retry, acaba derrubando o serviço de terceiro que tentava se recuperar.

Público

CONCEITO-CHAVE · Circuit breaker

O circuito abre depois de algumas falhas consecutivas. Quando uma nova tentativa, automática ou não, é bem-sucedida, o circuito fecha de novo e volta a operar. O mecanismo protege ao mesmo tempo o serviço externo e a latência do próprio sistema.

Os dois estados do circuit breaker



O circuito protege o serviço externo, que pode estar tentando se recuperar, e a latência da própria aplicação.

Diagrama editorial com os dois estados do circuito descritos na aula e no prompt.

Essas são algumas das técnicas de resiliência de sistemas. Há várias outras.

Os conceitos da engenharia de sistemas clássica continuam funcionando nos sistemas agênticos, cada vez mais. Não há mistério nos sistemas que usam LLM.

Onde esses padrões se aplicam

No projeto, as proteções são aplicadas ao Serper, mas o mesmo tratamento serve ao Vertex AI ou ao banco de dados. É o caso clássico do circuit breaker: não adianta insistir em um terceiro cujo serviço está tentando subir.

Circuit breaker e retry com backoff são muito usados em microsserviços. O projeto não tem microsserviços: o único serviço externo de busca é o Serper. Em um sistema grande, com muitos microsserviços, esses padrões são muito comuns e muito necessários.

O padrão decorator: resiliência sem reescrever os providers

Os padrões de projeto não morreram. Ficaram mais importantes. Uma leitura recomendada é o livro de padrões de projeto do Gang of Four, um clássico.

ATENÇÃO · Primeiro o problema, depois o padrão

A ordem é achar o problema e depois o padrão de projeto que o resolve, e não o inverso. É comum se empolgar com padrão de projeto.

O problema aqui é aplicar o mesmo comportamento de resiliência a duas funções, a do Serper e a do Google Custom Search, sem reescrever as duas.

CONCEITO-CHAVE · Decorator

Um comportamento padronizado é implementado uma vez, em uma função. A função que deve receber esse comportamento é envelopada por ela.

Muitos frameworks já trazem isso pronto. O NestJS tem. No mundo Java, o Spring Boot tem um módulo de resiliência: basta anotar com `@CircuitBreaker`, sem criar o mecanismo.

O decorator não é o arroba em cima da função. O arroba, do Java ou do TypeScript, é syntax sugar, um recurso para deixar o código mais bonito. Na execução, ele faz a mesma coisa: chama uma função de uma função.

No projeto, a função que cria o provider do Serper aparece envelopada pelo `comResiliencia`, sem a sintaxe com arroba. Essa sintaxe não é padrão do JavaScript nem do TypeScript. É uma tentativa que existe há bastante tempo e não é unanimidade. Pelo que conheço, só o NestJS e o Angular usam bastante o decorator do TypeScript. O padrão de projeto pode ser usado com ou sem o syntax sugar.

Público

A resiliência do SearchProvider na prática

As três proteções são criadas com este prompt:

Prompt enviado ao Claude Code

```
O SearchProvider (Serper e Google Custom Search) não tem nenhuma proteção contra falha externa hoje — se a API travar, o fetch espera pra sempre. Adiciona timeout explícito de 10 segundos no fetch dos dois providers. Depois, cria um decorador comResiliencia() que envolve qualquer SearchProvider com retry e backoff exponencial, mais um circuit breaker: falhas consecutivas o bastante abrem o circuito, e chamadas seguintes falham na hora, sem chamar o provider, até um tempo de resfriamento passar. criarSearchProvider() aplica isso por padrão pro Serper e pro Google — a fixture do modo DEMO não passa por essa camada, porque não tem rede ali pra proteger. Tempo de espera e relógio precisam ser injetáveis, pra dar pra testar sem esperar de verdade os segundos de backoff ou o resfriamento do circuito.
```

O prompt define quatro pontos:

- **Timeout**: explícito, de 10 segundos, no fetch dos dois providers.
- **Decorador `comResiliencia`**: envolve qualquer search provider com retry e backoff exponencial, mais o circuit breaker. Com falhas consecutivas em número suficiente, o circuito abre, e as chamadas seguintes falham na hora, sem chamar o provider, até passar o tempo de resfriamento.
- **Aplicação por padrão**: a função que cria o search provider aplica o decorador ao Serper e ao Google. A fixture do modo demo fica de fora, porque ali não há rede para proteger.
- **Tempo injetável**: o tempo de espera e o relógio são injetáveis, para que os testes não precisem esperar de verdade os segundos de backoff ou de resfriamento.

A resiliência fica em um arquivo próprio. O decorador é uma função com algumas constantes, que podem ser alteradas:

- o máximo de tentativas, que por padrão é três;
- o backoff inicial;
- o limite de falhas consecutivas para abrir o circuito;
- o tempo de resfriamento, depois do qual o circuito volta a chamar o serviço envelopado.

O funcionamento não tem nada de sofisticado. As tentativas ficam em um `for`, que chama a busca da função envelopada até três vezes. Depois de três tentativas, o circuito abre. Para saber quando pode chamar de novo, o decorador verifica o tempo restante do circuito.

O timeout não fica nesse arquivo. Ele está nos próprios providers, o do Serper e o do Google: o fetch recebe o parâmetro `signal`, que aborta a requisição depois de 10 segundos. A exceção que sobe é capturada, e o decorador faz uma nova tentativa.

Testes unitários documentam esses tratamentos e garantem a sua qualidade.

Se o framework já dá timeout e as demais capacidades de resiliência por padrão, use o que está nele.

O sistema funciona com ou sem essas proteções. Com elas, um serviço que depende de outros, fora do seu controle, consegue atender ao cliente por mais tempo. Resiliência é isso: fazer de tudo para o serviço não sair do ar.

Público

A tabela a seguir resume as camadas de proteção, incluindo a degradação graciosa, explicada na próxima seção:

Técnica	Problema que resolve	Como foi aplicada
Timeout	O fetch não tem timeout por padrão: se o serviço trava, a requisição trava junto.	Timeout explícito de 10 segundos no fetch dos dois providers, com o parâmetro `signal`.
Retry com backoff	Falhas passageiras, como um pico de requisições no serviço de terceiro.	Até três tentativas, com backoff exponencial entre elas.
Circuit breaker	Retries em excesso podem derrubar o serviço que tenta se recuperar.	Falhas consecutivas abrem o circuito; as chamadas falham na hora até passar o tempo de resfriamento.
Degradação graciosa	O serviço não volta, e o usuário não pode ficar com uma tela branca.	Try/catch por query no nó web search: o grafo termina no answer, que avisa do contexto incompleto.

Degradação graciosa: quando o serviço não volta

Pode acontecer de o circuito estar aberto e o serviço realmente não voltar.

> **CONCEITO-CHAVE · Degradação graciosa**
> O sistema tentou, fez tudo o que podia e, mesmo assim, não deu. Além de registrar o problema em log e em telemetria, ele mostra a falha ao usuário de uma maneira que o conforte.

O pior jeito de mostrar uma falha é a tela branca, porque o usuário não sabe o que aconteceu. O mínimo é um erro, ou um erro com uma alternativa que ele possa seguir, como entrar em contato com o suporte.

Por mais que se tente, o sistema sai do ar. É melhor encarar essa verdade do que a ilusão de que ele nunca vai sair. O ponto é como a equipe, a empresa e o próprio sistema estão preparados para lidar com isso. Em qualquer serviço, na vida real, falhas acontecem, e às vezes o jeito como a falha é tratada faz o cliente gostar até mais da marca.

Quando degradar e quando falhar

A degradação graciosa não serve para qualquer regra de negócio.

> **EXEMPLO · Extrato e saldo**
> Em um sistema com muitos microsserviços, o extrato e o saldo vêm de serviços diferentes. Um PO que conhece bem o cliente pode definir que, sem o detalhe do extrato, o sistema mostra outra tela, só com o saldo, pedindo para aguardar o extrato. Ou o contrário. O usuário não sai totalmente atendido, mas também não sai de mãos abanando.

O caso oposto é um pagamento. Se o pagamento falhou, a pessoa não leva os ingressos do cinema. Aqui a falha tem de ser uma falha: a operação é abortada, o usuário recebe uma mensagem, tudo é registrado na telemetria, e o fluxo não segue até o fim.

Mesmo quando a falha tem de ser uma falha, ela vem com uma mensagem de erro, e não com uma tela em branco. Conhecer o conceito permite proporcionar uma UX melhor ao usuário onde a regra de negócio aceita.

No projeto, uma resposta incompleta não é problema, desde que isso fique claro para o usuário.

Degradação graciosa no nó web search

Sem tratamento, um erro na busca dentro do nó web search, com o Serper fora do ar ou com o circuito aberto, se propaga para fora do nó e derruba o grafo inteiro. Isso acontece mesmo quando o corpus da CVM, sozinho, já daria para responder algo. E a busca na web não é a parte principal do grafo: há também a busca no banco de dados.

Público

Prompt enviado ao Claude Code

```
Hoje, se searchProvider.search() lançar erro dentro do nó web_search — Serper fora do ar, ou o circuito aberto — o erro propaga pra fora do nó e derruba o grafo inteiro, mesmo quando o corpus da CVM sozinho já dava pra responder algo. Quero que cada query tenha seu próprio try/catch em volta da chamada a search(): se uma query falhar, segue pra próxima, sem propagar. Se nenhuma query render documento novo, o nó simplesmente não adiciona nada — o grafo volta pro retrieve de mãos vazias, o guard de iteração eventualmente corta o loop, e o answer já sabe avisar que o contexto pode estar incompleto. Escreve um teste de ponta a ponta no graph.test.ts provando isso: SearchProvider sempre falhando, e o grafo ainda assim termina em answer com uma resposta real.
```

O comportamento pedido é este:

- cada query tem o seu próprio try/catch em volta da chamada de busca. Se uma falha, o nó segue para a próxima, sem propagar o erro;
- se nenhuma query render documento novo, o nó devolve vazio, e o grafo decide o que fazer com o contexto que sobrou: o grade avalia, o guard de iteração corta o loop, e o answer avisa que o contexto pode estar incompleto;
- um teste de ponta a ponta prova isso: com o search provider sempre falhando, o grafo ainda termina no answer, com uma resposta real.

A resposta sai com o que existe, os dados da CVM, e deixa claro que se baseia somente neles. O erro capturado não é ignorado: é nesse ponto que o log passa a ser registrado, quando a observabilidade é adicionada ao projeto.

Esse tratamento não é para tudo. Não saia capturando erro e seguindo em frente em todo lugar, como se nunca fosse haver erro. Há casos em que a operação precisa falhar para não acontecer algo pior, como em um pagamento.

O grafo completo, com o nó answer

O nó answer passa a fazer parte do grafo. Antes, com o contexto suficiente, o processamento simplesmente acabava. Agora o fim vem depois do answer.

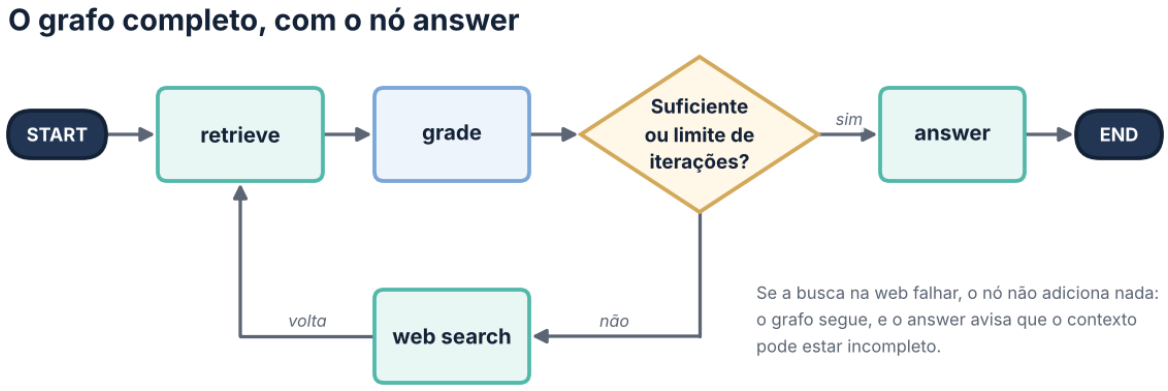


Diagrama editorial do grafo depois da inclusão do nó answer e da degradação graciosa.

A regra continua nas arestas. O grafo começa, faz a busca e faz o grade. Na condicional depois do grade, contexto suficiente leva ao answer, e contexto insuficiente leva ao web search. O web search volta ao retrieve, no máximo duas vezes. Na última, a execução vai para o answer.

Com as proteções no web search, o retry pode funcionar, ou o circuito pode já ter esfriado na segunda volta. Mesmo que o Serper fique fora do ar de vez, o usuário tem uma resposta. Ela não é completa, e o answer deixa isso claro.

Público

O que os testes pegaram durante a implementação

Durante a execução do prompt, um teste unitário acusou um problema no código. Mesmo com um modelo muito bom, erros acontecem, e por isso a exigência de testes está na configuração do projeto desde o início. Foram duas correções:

- **Evento de chat do answer**: o nó answer não tinha um evento de chat. Sem ele, essa etapa não apareceria no painel de raciocínio.
- **Nome do canal de estado**: o LangGraph não deixa um canal de estado ter o mesmo nome de um nó. O nó se chama `answer`, e o estado também tinha um atributo `answer`. O canal passou a se chamar `answerText`. O evento SSE continua com o campo `answer`.

Se a regra dos nomes estivesse na skill do LangGraph, talvez o segundo erro não tivesse acontecido.

> **ATENÇÃO · O agente não deve mudar o teste para ele passar**
> Quando o teste falhou, o agente corrigiu o código, e não o teste. Deixe isso claro nas instruções: o agente não deve ficar alterando os testes.

O desenvolvimento agêntico depende desse harness para garantir a qualidade: testes agora e, mais adiante, telemetria.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar a diferença entre timeout, retry com backoff e circuit breaker e o problema que cada um resolve sem consultar o material?
- Você consegue explicar o que é o padrão decorator e por que ele independe da sintaxe com arroba sem consultar o material?
- Você consegue explicar o que é degradação graciosa e dar um exemplo de regra de negócio em que ela não deve ser usada sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- Em produção, o serviço de busca pode travar, oscilar ou cair, e o sistema precisa continuar respondendo sem mostrar uma tela branca.
- Timeout, retry com backoff e circuit breaker protegem o provedor de busca, aplicados por um decorator que envolve qualquer implementação.
- Com a degradação graciosa, a falha de uma busca não derruba o grafo: a resposta sai com o contexto disponível e avisa que ele pode estar incompleto.
- Nem toda regra de negócio aceita degradação: em um pagamento, a falha tem de ser uma falha, com mensagem clara ao usuário.

Público

Aula 16: Erros padronizados

Voltar ao mapa da disciplina • *Voltar à Unidade 3*

OBJETIVOS DE APRENDIZAGEM

- Explicar por que os erros de uma API devem ser tratados em um ponto central e devolvidos em um contrato padronizado.
- Criar um error handler único, que mapeia erros de validação, de circuito aberto e genéricos sem vazar detalhes internos.
- Avaliar o trade-off entre criar uma dependência entre pacotes e identificar um erro por duck typing.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Em uma aplicação production-ready, é preciso decidir como um erro chega ao cliente, seja o corpo inválido de uma requisição, seja um erro que ninguém previu. Se cada rota trata o erro do seu jeito, sem padrão, a experiência do usuário fica ruim.

Esta aula cria um tratamento único, que captura todos os erros da aplicação em um ponto só, em vez de espalhá-lo pelas rotas. É sobre esse ponto que entram depois a telemetria e os demais controles.

Ao final, você saberá montar um error handler central, explicar o contrato de erro adotado e avaliar a decisão de identificar um erro por duck typing.

Um error handler central no lugar do tratamento por rota

O error handler é uma função de tratamento de erro unificada, no `app.ts`. Ela recebe o app como parâmetro, e qualquer exceção levantada pelos endpoints é tratada ali. O projeto tem só duas rotas por enquanto, mas, com mais rotas, todos os erros continuariam unificados nessa função.

Prompt enviado ao Claude Code

```
Hoje cada rota da API trata erro do seu próprio jeito, sem padrão. Quero um error handler central,
registrado uma única vez com app.setErrorHandler() — nenhuma rota deve tratar erro por conta própria.
Mapeia ZodError pra 400, CircuitoAbertoError (do SearchProvider, já existe em @hero/agent) pra 503, e
qualquer outro erro pra 500 genérico, sem vazar detalhe interno tipo stack trace. O formato da resposta
é RFC 7807 — application/problem+json, com type/title/status/detail — define esse schema em
packages/contracts, porque o frontend também vai precisar dele quando chegar a hora de mostrar erro na
tela.
```

O handler é registrado uma única vez, com o `setErrorHandler` da aplicação, e nenhuma rota trata erro por conta própria. Ele analisa o problema, pega o detalhe e trata três casos:

Tipo de erro	Quando acontece	Como é identificado
`ZodError`	O consumidor envia um parâmetro errado na requisição.	Por `instanceof`.
Erro de circuito aberto	O Serper falha repetidamente e o circuit breaker abre.	Por duck typing: é um `Error` com o nome do erro de circuito aberto.
Qualquer outro erro	Um erro que ninguém previu.	Cai no tratamento genérico, sem vazar stack trace.

Público

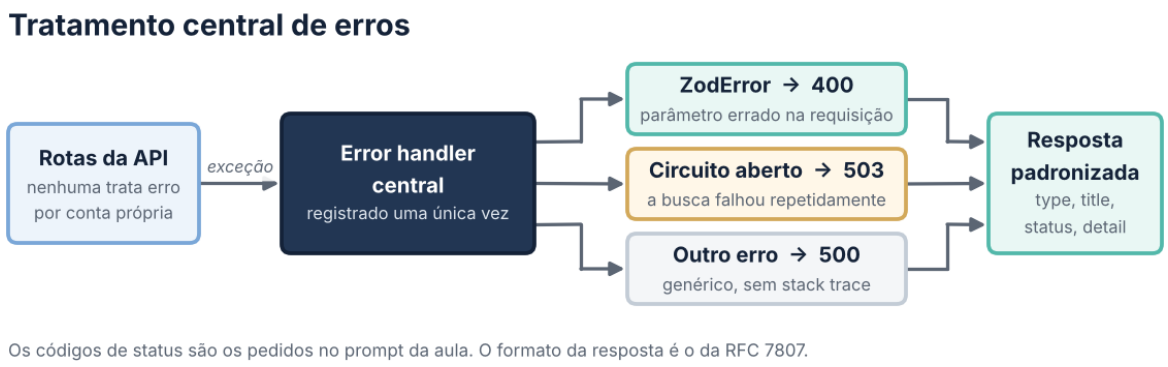


Diagrama editorial do tratamento central de erros pedido no prompt e descrito na aula.

Com o handler pronto, os tratamentos antigos saem das rotas, como o que devolvia o código 400. O código das rotas fica mais simples, principalmente no caso dos erros do Zod, de validação dos parâmetros de entrada da API.

Não vazar detalhes internos

O erro genérico não leva detalhes internos, como o stack trace. É uma questão de segurança.

> **ATENÇÃO · O stack trace não vai para o cliente externo**
> A telemetria fica nos serviços internos, onde quem enxerga é a própria equipe. O cliente externo recebe somente a informação necessária para tratar o problema, por exemplo com uma tela de erro específica. Uma pessoa mal-intencionada pode usar o stack trace para analisar o que acontece nos servidores e explorar essa informação.

O contrato de erro: RFC 7807

A resposta de erro segue o formato da RFC 7807, um formato padrão, com os campos `type`, `title`, `status` e `detail`. O schema fica no pacote de contratos, de onde é exportado.

O contrato atende a dois consumidores:

- **O frontend**: é um acordo. Todo erro devolvido pelo backend vem nesse formato, e o frontend pode mostrá-lo corretamente na UX.
- **Outros agentes**: hoje há muito mais agentes monitorando sistemas e, com MCP, agentes consumindo outros agentes. Um agente consegue entender o contexto de um erro sem padrão, mas, com o erro padronizado, ele o trata gastando muito menos tokens e com muito mais efetividade.

Dependência nova ou duck typing

Durante a execução do prompt, o Claude Code parou para perguntar, porque encontrou um conflito entre o pedido e a arquitetura.

O pacote da API não depende do pacote do agente. Depende só de contratos, do Fastify e do `pg`. Isso é deliberado: a rota de chat recebe o grafo por um tipo estrutural, em vez de importar os tipos reais do grafo, para manter a API testável sem LLM e sem Postgres. As dependências são fracamente acopladas.

Além disso, o erro de circuito aberto só é exportado pelo módulo de busca, e não pelo índice público do pacote do agente. Uma verificação com `instanceof` exigiria duas mudanças: adicionar o pacote do agente como dependência da API e reexportar o erro no índice público. E instalar uma dependência nova é exatamente o que o `CLAUDE.md` manda perguntar antes de fazer.

O agente apresentou duas saídas: refatorar o projeto, criando a dependência, ou usar duck typing pelo nome do erro.

> **CONCEITO-CHAVE · Duck typing**
> Em vez de um tipo exportado e verificável, o erro é reconhecido pelas suas características. Aqui, a verificação é que o objeto é uma instância de `Error` e que o nome dele é o do erro de circuito aberto.

Público

As duas saídas têm custo:

- **Duck typing**: mantém o acoplamento fraco. Em troca, se o nome da exceção mudar, a verificação quebra, e o erro deixa de ser mostrado corretamente.
- **Dependência nova**: permite o `instanceof`, mas acopla a API ao pacote do agente.

Mantive o duck typing, porque a prioridade é conservar as dependências como estão e continuar testando sem LLM e sem Postgres.

> **EM SÍNTESE**
> No dia a dia, vale conversar mais com o agente, no Claude Code ou em outra ferramenta, para verificar as alternativas e os riscos antes de decidir.

Testando com a aplicação no ar

O tratamento pode ser conferido com a aplicação rodando, e não só com teste unitário:

1. Suba só a API local.
2. Faça uma requisição com `curl` a uma rota que não existe. A resposta volta no formato unificado: uma mensagem de rota POST não encontrada, com o status 404.
3. Envie um body vazio à rota de chat e verifique se o erro retornado está no formato do contrato.

Na demonstração, o terceiro passo foi pedido ao Claude Code, que subiu o servidor internamente e fez o teste pela própria aplicação.

A aplicação didática tem só dois endpoints, mas a técnica facilita muito quando o assunto passa a ser volume, observabilidade e telemetria.

CHECKPOINT DE APRENDIZAGEM

• Você consegue explicar por que o tratamento de erros deve ficar em um ponto central e seguir um contrato sem consultar o material?

• Você consegue explicar quais tipos de erro o error handler mapeia e por que o stack trace não deve chegar ao cliente externo sem consultar o material?

• Você consegue explicar o que é duck typing, o que ele preserva no projeto e qual risco ele traz sem consultar o material?

PRÁTICA RECOMENDADA

• Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

• Um error handler central, registrado uma única vez, substitui o tratamento de erro espalhado pelas rotas.

• As respostas de erro seguem o formato da RFC 7807, definido no pacote de contratos, que o frontend e outros agentes conseguem tratar de forma padronizada.

• Detalhes internos, como o stack trace, ficam na telemetria interna e não vão para o cliente externo.

• O erro de circuito aberto é reconhecido por duck typing, o que mantém a API desacoplada do pacote do agente, ao custo de depender do nome do erro.

Público

Aula 17: O servidor MCP

Voltar ao mapa da disciplina • *Voltar à Unidade 3*

OBJETIVOS DE APRENDIZAGEM

- Expor o agente a outros agentes por meio de um servidor MCP com uma única tool, mantendo o guardrail do lado de dentro.
- Explicar o papel da descrição da tool, que é lida pelo cliente MCP para decidir quando chamá-la.
- Diferenciar o transporte stdio, usado localmente, do Streamable HTTP, usado em um servidor, e compartilhar a mesma instância do grafo entre as rotas.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O chat não é a única forma de interagir com o agente. Outra forma, hoje muito comum, é o protocolo MCP, que deixa o sistema disponível para outros agentes utilizarem.

Esta aula expõe o grafo por um servidor MCP, primeiro para uso local, por stdio, e depois para uso em um servidor, por Streamable HTTP. Ao final, você saberá decidir o que expor por MCP, escrever a descrição de uma tool e explicar por que as duas portas de entrada do agente compartilham o mesmo grafo.

Uma única tool: a experiência do agente

Em MCP, como em uma API, pensa-se na experiência de quem consome. Só que aqui é a experiência do agente.

Seria possível expor o grafo inteiro para um agente consumir. O risco é esse agente não aplicar o guardrail e usar o sistema para uma operação para a qual ele não foi feito.

Por isso o servidor MCP abstrai o sistema para o agente externo e oferece uma função só: o agente pergunta, e o sistema responde. O que acontece da porta para dentro não interessa a quem consome.

> **EM SÍNTESE**
> Não se abre todo o sistema por MCP. Abre-se só o que faz sentido, na mesma lógica da API: ninguém transforma todas as funções do backend em APIs.

Público

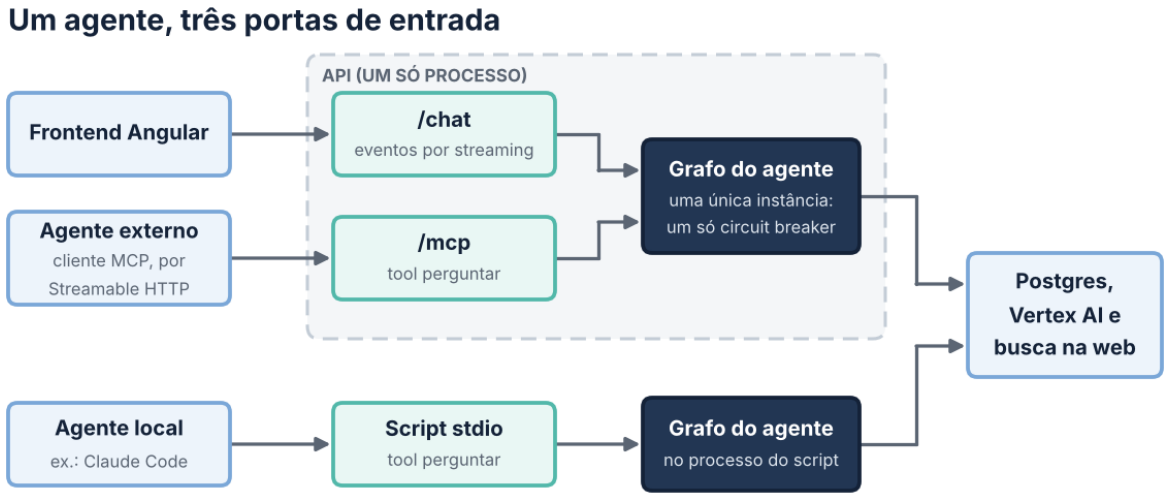


Diagrama editorial com as formas de consumir o agente descritas na aula.

Servidor e tool: os elementos de um MCP

O SDK do Model Context Protocol é uma biblioteca própria para expor um backend no protocolo MCP. No projeto o backend é um agente, mas vale para qualquer backend.

Publicar um MCP exige duas coisas:

- **Um servidor**, criado com a classe de servidor do SDK.
- **Uma tool registrada**. Em MCP não há endpoints, há tools.

> **CONCEITO-CHAVE · Tool**
> Uma tool é um nome, um título, uma descrição, um schema e a função que executa a partir da entrada enviada pelo agente que consome o MCP.

No arquivo `perguntar.ts`, esses elementos aparecem como constantes:

- **Nome**: `perguntar`.
- **Schema de entrada**: só a pergunta, com a indicação de que é sobre os indicadores financeiros.
- **Resultado**: o conteúdo, que é um texto, e o indicador de erro, verdadeiro ou falso.
- **Função**: recebe a pergunta, obtém o grafo, o mesmo usado pelo chat, manda processar e responde. Em caso de erro, responde com a mensagem de erro e com o indicador de erro verdadeiro.

A descrição da tool é lida pelo agente

A descrição da tool não é documentação para humano. Ela é lida pelo cliente MCP, que a usa para decidir quando chamar a tool e quando não chamar.

Quando uma tool é instalada, o agente a coloca no contexto. Diante de um prompt, ele raciocina e, se entender que a tool é importante para a resposta, a aciona. Por isso a descrição precisa deixar claro o domínio e repetir o guardrail.

A descrição da tool `perguntar` informa:

- que ela responde a perguntas sobre indicadores financeiros, como receita líquida e lucro líquido, de 20 empresas listadas na B3, com base em dados públicos da CVM;
- que, quando o contexto do corpus não é suficiente, ela busca na web;
- que toda resposta cita a fonte e o período do dado;
- que ela nunca recomenda comprar, vender ou manter uma posição em um ativo. Só informa números e fatos publicados e nunca dá conselho de investimento;
- que não é uma tool de consultoria. Pedidos de opinião, sugestão ou recomendação são recusados pela própria resposta, não pela ausência da tool.

Público

O servidor MCP com transporte stdio

O servidor é criado com este prompt:

Prompt enviado ao Claude Code

```
Cria packages/mcp-server, um servidor MCP com uma única tool, perguntar: recebe uma pergunta em texto e roda o grafo inteiro que já existe em packages/agent — retrieve → grade → webSearch → answer — devolvendo a resposta final, já citada e com guardrail aplicado. A descrição da tool é lida pelo cliente MCP pra decidir quando chamar — não é documentação pra humano — então precisa deixar claro o domínio (20 empresas da B3, dados da CVM) e repetir o guardrail (nunca recomenda). O grafo é recebido por parâmetro (GetGraph), mesmo padrão de injeção de dependência do resto do projeto, pra testar com fake sem tocar Postgres nem Vertex AI — usa o InMemoryTransport do próprio SDK do MCP pra testar o servidor inteiro. Se o grafo falhar, a tool nunca deixa a exceção crua vazar — devolve isError: true com uma mensagem genérica, mesma filosofia de degradação graciosa da E.6. A construção do grafo real (Postgres + Vertex AI) — incluindo os clients de embeddings, grade e answer, que ainda não existem como módulo reutilizável — deve morar dentro de packages/agent, memoizada por processo: o transporte HTTP desta mesma aula vai precisar exatamente da mesma função. Constrói o grafo real só na primeira chamada, nunca na inicialização do processo. Sobe primeiro o transporte stdio, pro claude mcp add.
```

Além da tool e da sua descrição, o prompt define:

- **Grafo por parâmetro**: o mesmo padrão de injeção de dependência do resto do projeto, para testar com fake, sem tocar no Postgres nem no Vertex AI. O servidor inteiro é testado com o `InMemoryTransport` do próprio SDK do MCP.
- **Falha sem vazamento**: se o grafo falhar, a tool nunca deixa a exceção crua vazar. Devolve `isError: true` com uma mensagem genérica, na mesma filosofia da degradação graciosa.
- **Construção do grafo real no pacote do agente**: com Postgres e Vertex AI, incluindo os clientes de embeddings, grade e answer, que ainda não existiam como módulo reutilizável. O projeto é refatorado para o MCP consumir o agente.
- **Memoização por processo**: o grafo é instanciado na primeira chamada, nunca na inicialização do processo. Se for preciso usá-lo de novo e ele já estiver na memória, usa-se o que está lá. O transporte HTTP precisa exatamente dessa mesma função.
- **Transporte stdio primeiro**.

> **CONCEITO-CHAVE · stdio**
> É a interface local de texto, de entrada e saída de strings: a própria interface do console. Nesse transporte, o que se expõe é basicamente um script, consumido localmente.

Preparando o ambiente local e consumindo por stdio

Com os testes automatizados passando, o servidor pode ser consumido pelo Claude Code local. O procedimento tem seis passos.

1. Suba o banco de dados com o Docker Compose e confirme que o Postgres está no ar.

2. Configure as variáveis de ambiente:

Prompt enviado ao Claude Code

```
Cria (ou completa, se já existir) o .env na raiz do repositório — é o que tsx --env-file-if-exists=../../.env carrega sozinho em todo pnpm dev do monorepo. Precisa ter: DATABASE_URL apontando pro Postgres local do docker-compose (postgresql://postgres:postgres@localhost:5432/ai_researcher); GOOGLE_CLOUD_PROJECT com o id do projeto GCP deste curso; GOOGLE_CLOUD_LOCATION=global — não us-central1, esses modelos só respondem em global; e os três nomes de modelo, nunca hardcoded no código: MODEL_GRADE=gemini-3.5-flash-lite, MODEL_ANSWER=gemini-3.6-flash, MODEL_EMBEDDING=gemini-embedding-001.
```

O `.env` na raiz do repositório é carregado sozinho em todo `pnpm dev` do monorepo. Ele precisa ter:

- a URL do banco apontando para o Postgres local, explícita, para evitar erro caso a porta tenha mudado;
- o ID do projeto do Google Cloud, que no seu caso é o do projeto da sua conta;
- a localização `global`, por causa dos modelos;
- os três modelos: um Gemini Flash no grade, outro no answer e o Gemini Embedding 001 nos embeddings.

Público

Se o arquivo já existir, o agente só o atualiza.

3. Aplique a migration. O banco local é efêmero, e um banco novo sempre precisa do migrate.

4. Carregue os dados com `pnpm seed`.

5. Faça o build do servidor MCP e adicione-o ao agente com o comando `claude mcp add`. Outras ferramentas de linha de comando, como o Gemini CLI, que agora é o Antigravity CLI, têm um comando equivalente para adicionar um MCP.

O build não sobe um servidor HTTP. Ele gera um arquivo de script, que o Claude Code executa localmente quando vai fazer uma pergunta.

6. Em outra aba do Claude Code, faça uma pergunta mandando usar a tool:

Prompt enviado ao Claude Code

```
Usando a tool perguntar do servidor ai-researcher, qual foi a receita líquida da Vale no último trimestre?
```

O prompt força o uso da tool. Sem essa indicação, o agente decidiria pela descrição se a tool cabe naquele contexto.

> **EXEMPLO · A resposta da tool**
> Na demonstração, com o servidor HTTP fora do ar, o agente executou o script e respondeu que a receita líquida da Vale, conforme fonte web complementar, foi de R$ 53 bilhões, com as fontes citadas. A resposta também contou, em texto, o que aconteceu no processamento.

Streamable HTTP: o MCP em um servidor

Com stdio, é possível criar servidores MCP para rodar localmente, o que é muito válido. Para um agente que roda em um servidor consumir o AI Researcher, o MCP precisa estar em um servidor também. Antes dessa mudança, vale fazer o commit do código, para ficar claro o que mudou.

Prompt enviado ao Claude Code

```
Expõe o packages/mcp-server também via Streamable HTTP, além do stdio que já existe — mesma tool perguntar, um plugin Fastify reutilizável (mcpHttpPlugin), registrável em dois lugares: um servidor standalone com porta própria (MCP_HTTP_PORT, default 3100), e também dentro do apps/api, como gateway em /api/mcp pro Angular consumir, sem CORS novo, reaproveitando a mesma porta e o mesmo proxy que o /chat já usa. Modo stateless — sem sessão guardada em memória entre chamadas, GET/DELETE respondem 405. A superfície pública que o apps/api importa (o plugin, os tipos do grafo) precisa ficar separada dos arquivos que têm efeito colateral ao rodar (o bootstrap do stdio, o app.listen() do servidor HTTP standalone) — importar esse pacote como biblioteca nunca pode disparar nenhum dos dois. E dentro do apps/api, /chat e /mcp devem compartilhar a mesma instância do grafo, não duas separadas — senão viram dois SearchProviders com dois circuit breakers independentes pra mesma API.
```

O prompt define:

- **Mesma tool, novo transporte**: a tool `perguntar` passa a ser exposta por Streamable HTTP, além do stdio.
- **Plugin Fastify reutilizável**, registrado em dois lugares: em um servidor standalone, com porta própria, e dentro da API, como gateway. A API passa a ter o `/mcp` além do `/chat`, sem CORS novo, reaproveitando a mesma porta e o mesmo proxy.
- **Modo stateless**: sem sessão guardada em memória entre as chamadas. GET e DELETE respondem 405.
- **Separação entre biblioteca e execução**: a superfície pública que a API importa, o plugin e os tipos, fica separada dos arquivos que têm efeito colateral ao rodar. Importar o pacote como biblioteca nunca dispara o bootstrap do stdio nem o `listen` do servidor HTTP.

No código gerado, o Fastify é instalado no pacote, e o stdio continua existindo ao lado do plugin HTTP. O plugin responde que o método não é permitido para GET e DELETE e aceita só o POST, que faz o mesmo que o stdio fazia: recebe a requisição, chama o agente e retorna, agora via HTTP. A lógica é a mesma.

O arquivo do servidor registra o grafo e faz o `listen`. O arquivo de biblioteca só expõe as funções, para os outros pacotes utilizarem. Na API, que antes tinha só o health check e o chat, entra a rota do MCP.

Público

A tabela a seguir compara os dois transportes:

Critério	stdio	Streamable HTTP
Onde roda	Localmente: o cliente executa um script e conversa pela interface de texto do console.	Em um servidor: standalone, com porta própria, e dentro da API, como gateway em `/mcp`.
Como é consumido	Adicionado ao agente local com o comando `claude mcp add`.	Por requisições POST; GET e DELETE respondem 405.
Estado	O script é executado quando o agente faz uma pergunta.	Stateless: sem sessão guardada em memória entre as chamadas.
O que expõe	A tool `perguntar`.	A mesma tool `perguntar`, com a mesma lógica.

Uma única instância do grafo para /chat e /mcp

Dentro da API, o `/chat` e o `/mcp` compartilham a mesma instância do grafo.

> **ATENÇÃO · Duas instâncias, dois circuit breakers**
> Com duas instâncias separadas, haveria dois search providers, com dois circuit breakers independentes, para a mesma API. O circuit breaker poderia não funcionar corretamente.

As regras de resiliência valem independentemente de como o agente é chamado. Com uma instância só, não importa se o erro ocorre no `/chat` ou no `/mcp`: o circuito abre, e todo o tratamento de resiliência se aplica.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar por que o servidor MCP expõe uma única tool em vez de abrir todo o grafo sem consultar o material?
- Você consegue explicar para que serve a descrição da tool e o que ela precisa deixar claro sem consultar o material?
- Você consegue explicar a diferença entre os transportes stdio e Streamable HTTP e por que `/chat` e `/mcp` compartilham a mesma instância do grafo sem consultar o material?

PRÁTICA RECOMENDADA

- Suba o ambiente local, faça o build do servidor MCP, adicione-o ao seu agente de linha de comando e faça uma pergunta pedindo que ele use a tool `perguntar`.

RESUMO DA AULA

- O agente é exposto por MCP com uma única tool, `perguntar`: o agente externo pergunta e recebe a resposta já citada e com o guardrail aplicado.
- A descrição da tool é lida pelo cliente MCP para decidir quando chamá-la, por isso declara o domínio e repete que a tool nunca recomenda.
- O transporte stdio serve ao uso local, por um script; o Streamable HTTP expõe a mesma tool em um servidor e dentro da API.
- As rotas `/chat` e `/mcp` compartilham a mesma instância do grafo, para que haja um único circuit breaker protegendo a busca.

Público

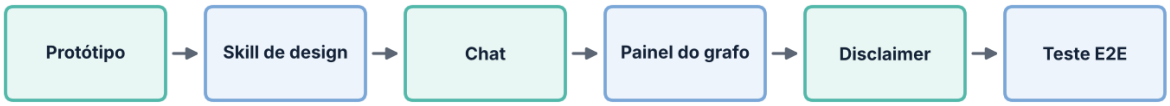
Unidade 4: Frontend

Do protótipo visual ao chat com streaming, painel de raciocínio e teste de ponta a ponta.

NESTA UNIDADE

- Aula 18: Do protótipo no Claude Design à skill frontend-design
- Aula 19: Chat com streaming e citações clicáveis
- Aula 20: Painel de estado do grafo em tempo real
- Aula 21: Disclaimer, guardrail visual e o E2E no CI

Do protótipo à tela



Fluxo conceitual criado para apoiar a leitura da unidade.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público

Aula 18: Do protótipo no Claude Design à skill frontend-design

Voltar ao mapa da disciplina • *Voltar à Unidade 4*

OBJETIVOS DE APRENDIZAGEM

- Prototipar a tela do chat com um prompt no Claude Design e refiná-la por iteração.
- Exportar o protótipo e usá-lo como base para o Claude Code criar a skill `frontend-design`.
- Gerar, com essa skill, os tokens globais de estilo e a casca estática do componente de chat em Angular.

GITHUB E PRÁTICA

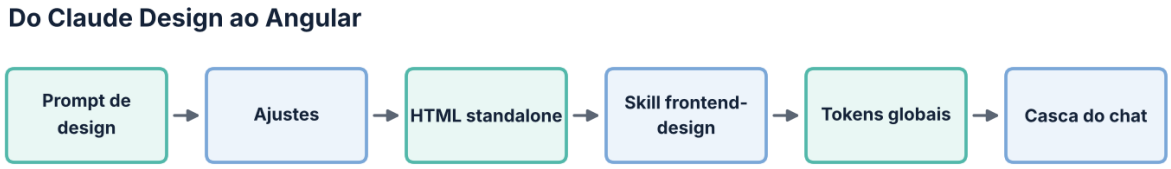
- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Com o backend testado, resiliente e exposto por MCP, falta a tela em que o usuário interage com o agente. O frontend começa pelo design.

Esta aula percorre o caminho do protótipo ao código: desenhar a tela em uma ferramenta de design com IA, exportar o resultado, transformá-lo em uma skill e, com ela, gerar a base visual do chat em Angular. Ao final, você saberá repetir esse processo com a ferramenta de design a que tiver acesso.



Fluxo editorial que organiza as etapas percorridas na aula.

A ferramenta de design: Claude Design ou Google Stitch

O protótipo é feito no Claude Design, que funciona por prompt, como o Google Stitch. As duas ferramentas servem:

- **Google Stitch**: gratuito, com uma cota bem generosa.
- **Claude Design**: disponível a partir da assinatura Pro, no aplicativo instalado ou no site do Claude.

Qualquer ferramenta à sua disposição serve para acompanhar o projeto.

O Claude Design vai além de UX. Ele permite criar um design system inteiro, e já sugere alguns, além de slides, apresentações, documentos, mockups e diagramas, inclusive de arquitetura. O projeto não usa um design system, mas poderia.

> **EM SÍNTESE**
> As ferramentas mudam depressa: já existe a do Google, pode surgir a da Microsoft. O importante é entender os fundamentos e o processo, para fazer o melhor com a ferramenta que a sua empresa oferece ou à qual você tem acesso.

Público

O protótipo: da descrição da tela ao refinamento

O protótipo nasce de um prompt que descreve a tela e é refinado em conversa com a ferramenta.

1. Descreva a tela de que o produto precisa:

Prompt enviado ao Claude Design

```
Desenhe a tela de um chat consultor de investimentos, ChatGPT-like. Paleta extraída da página da UNIPDS (unipds.com.br/gads_pos_ia): fundo quase-preto, accent ciano, um único botão em verde-limão pro envio da mensagem. Preciso de: bolha de mensagem do usuário à direita e do assistente à esquerda com avatar; texto do assistente aparecendo token a token, com um cursor piscando enquanto ainda está 'digitando'; um painel lateral mostrando o pipeline de raciocínio em 4 etapas — Retrieve, Grade, Search, Answer — cada uma com estado pendente/em execução/concluído; e citação clicável abaixo da resposta, abrindo um painel com data-base, origem e link pro documento original.
```

O prompt pede:

- **O estilo**: um chat consultor de investimentos, no estilo do ChatGPT, com a paleta da página da UNIPDS: fundo quase preto, acento ciano e um único botão em verde-limão, o de envio da mensagem.
- **As mensagens**: bolha do usuário à direita e do assistente à esquerda, com avatar.
- **O streaming**: o texto do assistente aparecendo token a token, com o cursor piscando enquanto ele digita.
- **O painel lateral**: o pipeline do raciocínio em quatro etapas, retrieve, grade, search e answer, cada uma com os estados pendente, em execução e concluído.
- **A citação**: clicável, abaixo da resposta, abrindo um painel com data-base, origem e link para o documento original.

A ferramenta cria o design em tempo real e o exibe simulando o funcionamento. É tudo mock. O código que ela gera pouco importa, porque o protótipo serve de exemplo para a aplicação em Angular.

2. Ajuste o que não foi pedido ou ficou diferente do esperado:

Prompt enviado ao Claude Design

```
A pílula de citação deve ter borda ciano parada quando fechada, e preencher em ciano quando aberta. Quando o pipeline cair no fallback de busca web, mostre um banner explicando que o contexto local foi insuficiente — na mesma paleta ciano, não em vermelho ou amarelo de alerta: fallback é um caminho normal do sistema, não um erro.
```

Outros dois ajustes foram pedidos na conversa: tirar a barra lateral de conversas, que não foi pedida, porque a aplicação é mais simples e não guarda as conversas, e destacar melhor, em vermelho, o aviso de conteúdo informativo.

> **CONCEITO-CHAVE · Fallback não é erro**
> O fallback de busca na web é um caminho normal do sistema. Por isso o banner que o anuncia usa a mesma paleta ciano, e não vermelho ou amarelo de alerta.

O Claude Design também testa o próprio design, principalmente o CSS, para ver se há alguma quebra. Na demonstração, ele percebeu que o scroll estava quebrando e adicionou a rolagem automática. Essa é a vantagem de ferramentas como o Claude Design e o Stitch: o design evolui em conversa, com o resultado à vista.

O prompt único é uma simplificação. Também é possível escrever uma especificação do frontend, com spec-driven, e conversar com o próprio Claude sobre mais ideias até chegar a um prompt mais assertivo.

A exportação em HTML standalone

O protótipo pronto é exportado para servir de fonte ao agente de código:

1. Clique em compartilhar e escolha exportar em HTML.
2. Entre as opções, escolha o HTML standalone, que traz em uma única página todo o código necessário, com o JavaScript dentro. A outra opção é um ZIP com tudo.
3. Baixe o arquivo e coloque-o em uma pasta de exemplo de UI dentro do projeto.

Público

> **ATENÇÃO · Não exporte em PDF**
> Com o PDF, o Claude Code teria menos dados para extrair e para pensar o código Angular. O HTML standalone é o formato mais adequado para criar a skill.

A skill frontend-design: o protótipo vira regra

O Claude Code lê o arquivo exportado e cria uma skill para ele mesmo, específica deste projeto. É ela que orienta a escrita do código Angular.

Prompt enviado ao Claude Code

```
Leia docs/design/prototype/ai-researcher-chat.dc.html e design-tokens-used.md. Crie uma skill
em .claude/skills/frontend-design/ que traduza os tokens de cor, tipografia e forma, e os padrões de
componente (bolha de mensagem, citação, painel do pipeline, disclaimer, botão de envio) em regras pro
código Angular. Inclua a regra de nunca usar verde ou vermelho em dado financeiro ou estado do pipeline
– só ciano é cor de estado.
```

O prompt aponta para o HTML exportado, no caminho em que ele foi salvo no projeto, e pede que a skill traduza em regras para o código Angular os tokens de cor, a tipografia, as formas e os padrões de componente. Ele inclui uma regra de produto: nunca usar verde ou vermelho em dado financeiro ou em estado do pipeline. Só o ciano é cor de estado.

A skill criada tem estas partes:

- **Descrição**: usar ao estilizar componentes do frontend Angular do projeto, em cores, tipografia, formas e padrões visuais de bolha de mensagem, citação, painel do pipeline e disclaimer. Ela traduz o mockup em tokens de CSS e em regras de componente e complementa a `angular-component`, que cobre arquitetura, e não aparência.
- **Origem**: de onde as regras vieram, com a instrução de abrir o mockup.
- **Tokens de cor**, com as cores em hexadecimal, **tipografia** e **formas**.

O frontend é escrito com três fontes de regras: a `angular-component`, criada para o projeto, as skills do time do Angular e a `frontend-design`. O protótipo continua valendo como exemplo ao lado da skill.

> **EM SÍNTESE**
> Quanto mais exemplos o agente recebe, menos tokens são gastos e mais assertivo é o resultado.

Tokens globais e a casca estática do chat

O projeto Angular já existe desde o walking skeleton, mas está praticamente vazio. O primeiro passo é a base visual:

Prompt enviado ao Claude Code

```
Usando a skill frontend-design, declare os tokens de cor e tipografia como custom properties globais em
styles.scss, e gere a casca estática do componente Chat – header com o wordmark, faixa de disclaimer,
área de mensagens vazia e composer com input e botão Enviar. Sem lógica de streaming ainda.
```

O prompt pede duas entregas:

- **Tokens globais**: os tokens de cor e de tipografia declarados como custom properties globais na folha de estilos.
- **Casca estática do chat**: header com o wordmark, faixa de disclaimer, área de mensagens vazia e composer com input e botão de enviar, sem lógica de streaming.

Na execução, o Claude Code perguntou sobre a fonte, e mandei adicionar. Ele adicionou as fontes, criou a folha de estilos e criou o chat só com o HTML, sem lógica nenhuma no componente. Depois fez o build e refatorou o primeiro componente, que servia só ao health check.

Nessa etapa ele usa apenas a `frontend-design`. As skills do Angular são acionadas quando o trabalho passa a envolver a lógica dos componentes.

Público

EXEMPLO · Verificação no navegador

Com a extensão do Claude instalada no Chrome, o Claude Code abre o navegador automaticamente e faz ali os testes de interface, além dos testes unitários.

Skills da comunidade: código atual com qualquer modelo

Com Vue ou React, o processo é o mesmo. Em qualquer framework, vale se basear em skills que já existem na comunidade, para criar o código mais otimizado possível.

Essas skills também reduzem a dependência do modelo. Um modelo mais fraco pode criar, por exemplo, um app Angular com padrões do passado. Uma skill força o modelo a seguir as melhores práticas atuais.

O projeto foi construído só com prompts, mais completos, para ser direto. No seu, vale usar spec-driven e outras técnicas, outra nuvem e outro modelo. O Capstone é onde todo o conhecimento do curso é posto à prova.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar como o protótipo da tela foi criado e refinado no Claude Design sem consultar o material?
- Você consegue explicar por que o protótipo foi exportado em HTML e como ele virou a skill `frontend-design` sem consultar o material?
- Você consegue explicar o que a casca estática do chat contém e o que ainda fica de fora dela sem consultar o material?

PRÁTICA RECOMENDADA

- Crie o protótipo da sua tela na ferramenta de design a que você tiver acesso, exporte-o em HTML e peça ao seu agente de código uma skill de design a partir dele. Depois, gere a casca estática do chat usando essa skill.

RESUMO DA AULA

- A tela do chat nasce de um prompt no Claude Design, que descreve os elementos de que o produto precisa, e é refinada por iteração.
- O fallback de busca na web é mostrado em ciano, e não em cor de alerta, porque é um caminho normal do sistema, não um erro.
- O protótipo exportado em HTML standalone serve de fonte para o Claude Code criar a skill `frontend-design`, com tokens de cor, tipografia, formas e regras de componente.
- Com a skill, são gerados os tokens globais e a casca estática do chat, ainda sem lógica de streaming.

Público

Aula 19: Chat com streaming e citações clicáveis

Voltar ao mapa da disciplina • *Voltar à Unidade 4*

OBJETIVOS DE APRENDIZAGEM

- Conectar a casca estática do chat ao backend, com roteamento, um serviço que lê o stream de eventos e um store que guarda as mensagens.
- Explicar o truque de frontend que revela a resposta palavra por palavra para dar fluidez ao streaming.
- Formatar as citações da resposta, diferenciando a base local da CVM das fontes da web.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

A casca do chat existe, mas não faz nada: nenhum componente conversa com o backend. Esta aula conecta a tela à API e começa a moldar a experiência do usuário.

O trabalho é dividido em peças pequenas, cada uma com o seu prompt: as rotas, o serviço que lê o stream, o store que guarda as mensagens, a função que formata as citações e, por fim, a ligação de tudo no componente. Ao final, você saberá explicar o papel de cada peça e por que a resposta é exibida palavra por palavra.

As peças do chat no frontend

A tabela a seguir apresenta as peças e o papel de cada uma:

Peça	Papel
Rotas	A rota vazia abre o chat e a rota de health mostra o status, com o router outlet no lugar do componente fixo.
`ChatService`	Faz o POST para a API de chat, lê o stream da resposta e entrega os eventos do grafo; lança exceção se o evento for de erro.
`ChatStore`	Adiciona a mensagem do usuário e o placeholder do assistente e revela a resposta palavra por palavra, a cada 70 milissegundos.
`parseAnswer`	Separa o corpo da resposta do bloco de fontes e monta as citações: base local da CVM com período, ou o host da URL.
Componente Chat	Liga a textarea e o botão de envio ao store e renderiza as mensagens e as citações.

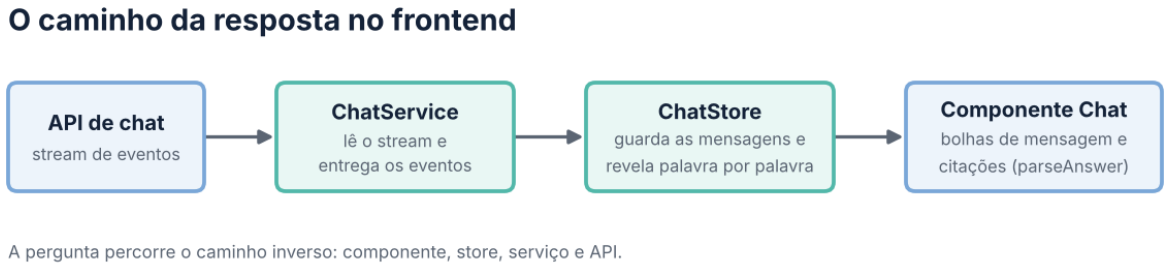


Diagrama editorial com as peças do frontend criadas na aula.

Público

Roteamento: o chat vira a página inicial

Por padrão, a aplicação abria a página de health check, e não o chat. O primeiro prompt corrige isso com uma refatoração simples, só do frontend:

Prompt enviado ao Claude Code

```
No projeto Angular já existente em apps/web, adicione roteamento com @angular/router (já é dependência). Crie src/app/app.routes.ts com duas rotas: '' aponta pro componente Chat, e 'health' aponta pra um componente novo Health em src/app/health/, reaproveitando o HealthService que já existe desde o Bloco C pra mostrar o texto de status que hoje está solto em app.html. Registre as rotas em app.config.ts com provideRouter. Troque app.html pra só <router-outlet />, e app.ts pra importar RouterOutlet em vez do Chat direto.
```

O resultado tem quatro partes:

- o arquivo de rotas, com a rota vazia apontando para o chat e a rota de health para um componente novo;
- o componente de health, que reaproveita o serviço de health já existente para mostrar o texto de status que estava solto no HTML do app, com o seu teste;
- o provider do router registrado na configuração da aplicação;
- o router outlet no lugar em que o componente ficava direto.

A aplicação usa standalone components, e por isso não há módulos Angular.

Um bug escondido na rota /chat

A mudança do MCP deixou um bug que passou despercebido: a rota `/chat` da API passou a responder só pelo grafo falso, que não chega ao Vertex AI nem ao Postgres.

Prompt enviado ao Claude Code

```
apps/api/src/routes/chat.ts ainda recebe graph: GraphLike já resolvido. Troque pra receber getGraph: GetGraph, resolvendo com await getGraph() dentro do handler, do mesmo jeito que packages/mcp-server já faz. Em apps/api/src/app.ts, renomeie as opções de build() pra getChatGraph e getMcpGraph. Em apps/api/src/index.ts, passe getRealGraph (de @hero/agent) pras duas — a mesma função memoizada, senão /api/chat e /api/mcp acabam com Postgres/Vertex AI/circuit breaker duplicados.
```

A rota passa a receber o `getGraph` e a resolvê-lo com `await` dentro do handler, como o servidor MCP já faz. As opções de build separam o grafo do chat do grafo do MCP, e as duas recebem a mesma função memoizada.

> **ATENÇÃO · A mesma função memoizada nas duas rotas**
> Com o agente memoizado, não importa se a chamada vem do MCP ou do `/chat`: se ele já está instanciado, é reutilizado. Os mecanismos de resiliência continuam funcionando e não ficam duplicados.

ChatService: quem conversa com o backend é o serviço

No padrão correto do Angular, quem interage com o backend nunca é o componente gráfico. É um serviço, que concentra a regra de negócio e toda a lógica de integração.

Prompt enviado ao Claude Code

```
No projeto Angular já existente em apps/web, crie um ChatService em src/app/api/chat.service.ts. Um método ask(question) que faz POST /api/chat, lê o ReadableStream do corpo da resposta e produz um AsyncGenerator dos eventos do grafo (retrieve, grade, webSearch, answer). O formato dos frames é o mesmo que apps/api/src/routes/chat.ts escreve: blocos separados por linha em branco, data: com o JSON do evento. Se o evento for error, lance uma exceção com a mensagem.
```

O serviço tem um método, o `ask`, que:

- faz o POST para a API de chat, com um fetch;
- lê o stream do corpo da resposta;
- produz um async generator dos eventos do grafo;
- lança uma exceção com a mensagem quando o evento é de erro.

Público

O formato dos frames é o mesmo que a rota de chat escreve: blocos separados por linha em branco, com `data:` e o JSON do evento.

O Claude Code cria o serviço usando a skill do Angular. Como há um contrato novo, ele o coloca no pacote de contratos, o lugar correto, em vez de deixá-lo no projeto web. Assim o mesmo contrato serve ao Angular e ao backend.

Três detalhes do Angular atual aparecem nesse código:

- `inject`: nas versões mais recentes, a forma antiga de construir e injetar serviços deu lugar à função `inject`, importante com standalone components.
- **Observable**: não é mais obrigatório. Ele não morreu, mas, principalmente com a API de signals, a integração com o backend não depende dele.
- **Resource**: é o recurso próprio do Angular para essa integração, baseado em signal. Ele não foi usado porque, quando o projeto foi desenvolvido, ainda não tratava retorno por stream.

ChatStore: a resposta revelada palavra por palavra

O frontend recebe a resposta do agente por streaming, mas os chunks não chegam de forma regular. Mostrar os dados conforme chegam deixa o texto truncado, diferente do que o ChatGPT e o Gemini fazem.

> **CONCEITO-CHAVE · O truque da digitação fluida**
> O frontend guarda a resposta conforme a recebe, como um buffer, e a solta na tela palavra por palavra, a 70 milissegundos por palavra. O truque não é obrigatório: serve para dar a sensação de UX do modelo respondendo de um jeito fluido.

Prompt enviado ao Claude Code

```
Crie um ChatStore em src/app/state/chat-store.ts. send(question) adiciona a mensagem do usuário e um placeholder do assistente, chama ChatService.ask, e quando chegar o evento do nó answer, revela o texto palavra por palavra (70ms por palavra) — com um helper tokenize separado. Se a chamada falhar, mostra uma mensagem de erro fixa, sem streaming. Limpe qualquer timer pendente ao destruir o store.
```

O store adiciona a mensagem do usuário e um placeholder do assistente, chama o `ask` do serviço e, quando chega o evento do answer, revela o texto palavra por palavra.

Se a chamada falhar, o store mostra uma mensagem de erro fixa, sem streaming. É esse cuidado que evita que um erro simplesmente estoure na tela.

Citações: de onde veio cada informação

As citações aparecem abaixo da resposta e dizem de onde veio a informação. Uma função as extrai do texto e as formata:

Prompt enviado ao Claude Code

```
Em apps/web/src/app/state/citation.ts, crie parseAnswer. Recebe o answerText, separa o corpo do bloco Fontes: no final, e extrai uma Citation por linha (rótulo + URL). Se o rótulo tiver o formato TICKER · período (data-base X), extraia essa parte como o período e marque a origem como 'Base local (CVM)'; senão, use o hostname da URL.
```

A função recebe o texto da resposta, separa o corpo do bloco de fontes que vem no final e extrai uma citação por linha, com rótulo e URL. A origem define a apresentação:

- **Base de dados**: se o rótulo tem o formato de ticker e período, a função extrai o período e marca a origem como base local da CVM. A citação mostra a data-base.
- **Internet**: nos demais casos, a origem é o host da URL, e a citação mostra a URL.

O Claude Code cria os testes dessa função junto com ela.

Público

O componente Chat: ligando tudo

O último prompt faz o wiring, a ligação das peças na casca estática:

Prompt enviado ao Claude Code

```
O componente Chat (src/app/chat/) ainda é a casca estática da 4.1 — nada nele usa ChatStore, ChatService
ou parseAnswer, que já existem e já têm teste. Injete ChatStore, ligue a textarea a um signal local com
(input)/(keydown) (Enter envia, Shift+Enter quebra linha), e o botão .chat__send a store.send(question)
(desabilitado enquanto vazio ou enquanto a última resposta ainda não chegou). Renderize store.messages()
em .chat__messages: mensagem do usuário como bolha à direita, mensagem do assistente passando por
parseAnswer — corpo do texto, e a lista de citações (link + data-base) quando houver.
```

O prompt detalha ponto a ponto:

- injetar o `ChatStore` no componente;
- ligar a textarea a um signal local. Enter envia, e Shift+Enter quebra a linha;
- ligar o botão de envio ao `send` do store, desabilitado enquanto o campo está vazio ou enquanto a última resposta não chegou;
- renderizar as mensagens: a do usuário como bolha à direita e a do assistente passando pelo `parseAnswer`, com o corpo do texto e a lista de citações, com link e data-base, quando houver.

Para saber como escrever, o Claude Code vai ao arquivo da skill e trabalha a partir dele. O resultado usa a sintaxe atual do Angular.

> **EXEMPLO · @if e @for**
> Antes, a lógica de visualização do Angular usava `ngIf` e `ngFor`. Hoje, usa `@if` e `@for`. Com uma skill atualizada, o agente escreve do jeito que o time do Angular definiu.

Antes de testar localmente: Postgres, migrate e seed

Em um teste local, confirme antes que o Postgres está no ar, com o migrate e o seed já executados.

> **ATENÇÃO · O Postgres local é efêmero**
> Do jeito que o projeto está configurado, o banco local apaga tudo a cada vez. Depois de reiniciar a máquina, por exemplo, é preciso subir o ambiente de novo, ou os testes dão erro.

O banco pode ser configurado para usar um volume, uma área de disco da máquina mapeada na configuração do Docker. Nesse caso, a carga não precisa ser refeita, porque o arquivo do banco fica guardado. Deixei o banco efêmero porque a aplicação é muito pequena e o banco para valer é o que sobe no GCP.

O teste no navegador feito pelo agente

Depois dos testes e do build, o Claude Code sobe o servidor de teste, na porta 4200, e verifica a interface no navegador, com o Claude no Chrome. O cursor laranja que aparece na tela é o do agente. Outras ferramentas, como o Gemini e o Codex, têm recursos parecidos.

> **EXEMPLO · O que o teste de interface mostrou**
> O agente digitou a pergunta "Qual foi o lucro líquido da Petrobras?" e deu Enter. A resposta veio com o aviso de contexto recuperado incompleto: como a pergunta não especificou o período, a resposta se baseou exclusivamente nos períodos disponíveis. A citação veio formatada, com a data-base, o que confirma a chamada real ao Vertex AI e ao RAG.

Ao avaliar o resultado, o agente encontrou um erro de interface: enquanto a resposta era processada, o botão ficava desabilitado, mas com a aparência de habilitado, como se fosse possível continuar enviando. Ele corrigiu e testou de novo no navegador.

> **EM SÍNTESE**
> Um teste unitário não pegaria esse erro. Ele só aparece na verificação da interface.

A tela ainda fica parada enquanto o agente pensa, sem animação, o que pode ser ajustado depois.

Público

Fundamentos, especificações e testes seletivos

O conteúdo não é só para quem se especializa em Angular, em backend ou em Java. O objetivo é entender os fundamentos, e com a IA isso ficou mais importante do que nunca. Conhecer a linguagem e o framework é o que permite entender a IA e guiá-la na direção correta.

A IA também serve para aprender a parte mais mecânica, a da sintaxe. O Angular tem um modo tutor, uma skill do próprio time do Angular, em que o agente explica os conceitos e passa exercícios.

O frontend foi construído prompt a prompt, para deixar claro o que cada passo pretende. Um bom desafio é escrever uma spec que amarre tudo isso em detalhe, com spec-driven, e gerar o projeto em ondas maiores.

O monorepo, com o Turborepo, permite escolher quais testes executar: todos de uma vez, que já passam de 40, só os de frontend ou só os de uma parte do backend.

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar o papel do serviço, do store e do componente na conexão do chat com o backend sem consultar o material?
- Você consegue explicar por que a resposta é revelada palavra por palavra, mesmo chegando por streaming, sem consultar o material?
- Você consegue explicar como as citações são formatadas conforme a origem da informação sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O chat vira a rota inicial da aplicação, e a rota `/chat` da API volta a usar o grafo real, com a mesma função memoizada usada pelo MCP.
- O `ChatService` concentra a integração com o backend e lê o stream de eventos; o `ChatStore` guarda as mensagens e revela a resposta palavra por palavra.
- O `parseAnswer` extrai as citações e distingue a base local da CVM, com período e data-base, das fontes da web.
- O teste no navegador, feito pelo próprio agente, encontrou um problema de interface que um teste unitário não pegaria.

Público

Aula 20: Painel de estado do grafo em tempo real

Voltar ao mapa da disciplina • *Voltar à Unidade 4*

OBJETIVOS DE APRENDIZAGEM

- Explicar como o frontend deduz o andamento do grafo a partir dos eventos do stream e prevê a próxima etapa.
- Criar o `GraphStore` sem carregar dependências de servidor no bundle do navegador.
- Construir o componente do painel de raciocínio e verificar o resultado no navegador.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

O painel lateral mostra ao usuário o raciocínio do agente, que são os nós do backend em execução: retrieve, grade, web search e answer.

Esse painel é um requisito do projeto. Ferramentas como o ChatGPT e o Gemini não mostram isso, porque fazem tudo por baixo dos panos. Aqui ele é um jeito didático de mostrar como um agente funciona, com as etapas de processamento à vista.

Esta aula cria a lógica do painel e o componente que a exibe. Ao final, você saberá explicar como o frontend deduz o andamento do grafo e por que ele não importa o código do agente para isso.



Diagrama editorial que organiza como os eventos do stream chegam ao chat e ao painel.

O GraphStore: deduzir o andamento a partir dos eventos

O `GraphStore` recebe os eventos de raciocínio expostos pelo backend, e não os eventos de resposta. Ele é a regra por trás do painel.

O backend não avisa que concluiu um nó e que outro vem a seguir. Ele informa cada nó conforme o processa. O frontend precisa deduzir o resto.

> **CONCEITO-CHAVE · A regra do painel**
> A cada evento do stream, o store marca a última etapa da lista como concluída e prevê a próxima. Se o grade está em andamento, o retrieve já terminou.

Público

Não chega a ser uma regra de negócio. É uma maneira de mostrar o andamento de forma mais fluida. As previsões seguem as transições do grafo:

Evento recebido	Próxima etapa prevista
retrieve	Sempre grade.
grade com `sufficient` verdadeiro	answer.
grade com `sufficient` falso	web search, a não ser que o máximo de iterações já tenha sido atingido: aí vai direto para answer.
web search	Sempre retrieve.

O store é criado com este prompt:

Prompt enviado ao Claude Code

```
Crie GraphStore em apps/web/src/app/state/graph-store.ts. A cada evento do stream, marca a última etapa da lista como concluída e prevê a próxima: depois de retrieve vem sempre grade; depois de webSearch vem sempre retrieve; depois de grade, se sufficient for verdadeiro vai pra answer, senão volta pra webSearch — a não ser que já tenha rodado o número máximo de iterações do grafo, aí vai direto pra answer. Espelhe o limite de iterações do packages/agent/src/graph.ts, mas não importe o pacote @hero/agent aqui — ele carrega LangGraph, Vertex AI e Postgres, dependências de servidor que não pertencem ao bundle do navegador.
```

O código gerado tem os tipos e os estados exibidos, a regra do próximo nó e, no final, a lógica de mostrar os nós.

Espelhar a regra sem importar o pacote do agente

A previsão depende do limite de iterações, que é definido no pacote do agente. O prompt manda espelhar esse limite, sem importar o pacote.

> **ATENÇÃO · Dependência de servidor não entra no bundle do navegador**
> O pacote do agente carrega LangGraph, Vertex AI e Postgres. Como tudo no projeto é JavaScript, o agente de código, dependendo do modelo, pode querer levar o bundle inteiro do backend ao frontend só por causa de uma regra. O prompt precisa proibir isso explicitamente.

O resultado foi a constante repetida no frontend. Essa escolha tem um risco e um atenuante:

- **Risco**: se o máximo de iterações mudar no backend, para 3, por exemplo, o valor do frontend fica errado.
- **Atenuante**: o projeto é um monorepo. Em uma refatoração futura, o agente também lê o frontend e altera o valor.

Outro esquema de compartilhamento é possível dentro do monorepo, desde que nenhuma dependência do backend vá parar no bundle do frontend.

Skills oficiais e carregamento sob demanda

Na execução, o Claude Code carrega a skill oficial `angular-component`. Boas skills, atualizadas, fazem diferença, e isso vale para qualquer ferramenta.

> **EM SÍNTESE**
> Se o framework ou a linguagem oferece uma skill oficial, use-a de preferência a uma skill da comunidade. O time sabe o que é melhor para o seu próprio framework.

A skill também se diferencia do system prompt pelo momento em que entra no contexto.

> **CONCEITO-CHAVE · Carregamento sob demanda**
> O agente avalia quando precisa carregar a skill no contexto. Em um projeto com LangGraph e muitos outros componentes, não faz sentido carregar algo do LangGraph durante um trabalho de Angular. Essa é a grande vantagem da skill.

Público

O componente do painel

O componente exibe as etapas do `GraphStore`:

Prompt enviado ao Claude Code

```
Crie GraphPanelComponent em apps/web/src/app/graph/graph-panel/. Lista as etapas de GraphStore como uma timeline vertical — círculo com spinner girando enquanto running, check quando concluída, com rótulo, descrição e a duração formatada. Quando alguma etapa for webSearch, mostra um banner explicando que o contexto local foi insuficiente. Use a skill frontend-design pra cor e forma. No final verifique no navegador se tudo esta funcionando corretamente
```

O prompt pede:

- **Timeline vertical**: um círculo com spinner girando enquanto a etapa está em execução e um check quando ela é concluída, com rótulo, descrição e duração formatada.
- **Banner de fallback**: quando alguma etapa é web search, um banner explica que o contexto local foi insuficiente.
- **Cor e forma**: pela skill `frontend-design`.
- **Verificação no navegador**: ao final, o agente abre o Claude no Chrome e testa. Se houver problema, corrige.

O Claude Code carrega as skills `angular-component` e `frontend-design` e, no raciocínio, usa como exemplo o HTML do protótipo. Mesmo com a skill, um exemplo concreto em HTML ajuda o agente a manter a consistência.

O componente é criado e colocado na página do chat, à direita, com o layout ajustado.

Testes unitários e verificação no navegador

O componente tem testes unitários, mas os tipos de teste se complementam. O teste unitário não pegou, por exemplo, a aparência do botão desabilitado, tratada na Aula 19. É a pirâmide de testes, um fundamento.

> **EM SÍNTESE**
> Com um agente, nunca foi tão fácil escrever testes. Não há motivo para não ter testes automatizados.

Na verificação no navegador, o agente abriu a página do chat e fez a pergunta sobre o lucro líquido. O painel mostrou o spinner e a etapa de busca no acervo, depois as demais etapas e, no final, a resposta.

Os tempos exibidos são aproximados

O painel mostra quantos segundos cada etapa levou.

> **ATENÇÃO · A duração é calculada pelo frontend**
> O backend não retorna os tempos. O frontend os calcula com base no momento em que as respostas chegam. Não é a medida mais acurada do que acontece no backend, mas serve para o usuário acompanhar o andamento. A medição precisa é assunto da observabilidade, na Aula 22.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar como o painel sabe que uma etapa foi concluída e qual será a próxima sem consultar o material?
- Você consegue explicar por que o `GraphStore` espelha o limite de iterações em vez de importar o pacote do agente sem consultar o material?
- Você consegue explicar o que o painel mostra em cada etapa e por que os tempos exibidos são aproximados sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- O painel lateral mostra ao usuário os nós do grafo em execução, o que torna visível o raciocínio do agente.
- O `GraphStore` marca a última etapa como concluída a cada evento e prevê a próxima, seguindo as mesmas transições do grafo.
- O limite de iterações é espelhado no frontend para não levar dependências de servidor ao bundle do navegador.
- O painel é uma timeline com spinner, check, descrição e duração, e os tempos são calculados pelo próprio frontend, de forma aproximada.

Público

Aula 21: Disclaimer, guardrail visual e o E2E no CI

Voltar ao mapa da disciplina • *Voltar à Unidade 4*

OBJETIVOS DE APRENDIZAGEM

- Adicionar o disclaimer fixo que deixa claro ao usuário que a ferramenta informa e não recomenda.
- Avaliar quando usar um agente para testar a interface e quando usar um framework de teste E2E, considerando custo e tempo.
- Estender o modo demo para o grafo inteiro e criar o teste E2E com Playwright.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Dois pontos fecham o frontend. O primeiro é o disclaimer, que deixa claro ao usuário que a ferramenta é de informação, e não de recomendação, respeitando a diretriz da CVM. O segundo é o teste E2E, que verifica a aplicação de ponta a ponta e roda na esteira.

Ao final, você saberá por que um aviso simples resolve uma obrigação regulatória, quando testar a interface com um agente e quando usar um script, e quais são os trade-offs de testar com mocks.

O disclaimer fixo: um aviso sempre visível

O disclaimer é uma faixa fixa no chat, entre o header e a lista de mensagens:

Prompt enviado ao Claude Code

```
Adicione uma faixa de disclaimer fixa em Chat, entre o header e a lista de mensagens, renderizada
incondicionalmente — sem @if, sem botão de fechar, sem modal. Texto: 'Este assistente apresenta
informações públicas, não recomendação de investimento.'
```

A faixa é renderizada incondicionalmente: sem `@if`, sem botão de fechar e sem modal. O texto é "Este assistente apresenta informações públicas, não recomendação de investimento".

> **EM SÍNTESE**
> Nem toda obrigação precisa de muita engenharia. Às vezes, um banner bem posicionado resolve.

O disclaimer não tem regra de negócio, mas é importante para a questão regulatória. Ele não substitui o resto: a validação continua no backend, com o guardrail. O aviso deixa a regra clara também para o usuário.

Agente ou framework de teste E2E

Um agente consegue testar a interface abrindo o navegador, interagindo e avaliando, como o Claude Code fez nas Aulas 19 e 20. Isso não significa que ele substitui os frameworks de teste, como o Cypress e o Playwright. O motivo principal é o custo.

Público

A tabela a seguir compara as duas formas:

Critério	Agente operando o navegador	Script de Playwright
Custo	Bem mais caro: gasta muitos tokens a cada execução.	Consome só recurso de máquina, sem tokens.
Tempo	Demora bem mais do que o script.	Mais rápido.
Execução	O agente abre o navegador, interage e avalia o que vê.	Determinística: o script é criado uma vez e repetido quantas vezes for preciso.
Onde usar	No desenvolvimento local, para verificar a tela e pegar erros.	Na esteira do GitHub Actions, como guarda em todo PR aprovado para a `main`.

No ambiente de desenvolvimento, o teste com agente é totalmente viável. Na pipeline, executado a cada PR aprovado, ele subiria o agente, faria o teste e gastaria muitos tokens. Por isso não o recomendo, neste momento, para a esteira.

O caminho adotado combina os dois: o Claude Code escreve o script, e o Playwright o executa na esteira. Como o script só consome recurso de máquina, pode ser executado quantas vezes for preciso.

> **ATENÇÃO · Saber Cypress ou Playwright continua necessário**
> A IA não dispensa o framework de teste. Há casos e casos: para o teste local, em tempo de desenvolvimento, o agente serve bem; para a esteira, o script é a escolha. Usar um agente para criar o script é totalmente viável.

O modo demo estendido para o grafo inteiro

Um teste automatizado roda muitas vezes. A cada execução, o sistema usaria o Serper e o Vertex AI, duas dependências caras e com limite. A flag de demo, criada na Aula 14 para o Serper, é estendida para cobrir o grafo inteiro:

Prompt enviado ao Claude Code

```
Estenda DEMO=true pro grafo inteiro. criarEmbeddings() deve devolver um vetor de zeros quando em modo demo — o valor não importa, porque a busca por similaridade também vai ser fixture. Crie criarBuscarDemo() que sempre devolve o mesmo documento real da Vale 3T25, congelado do corpus. Crie criarGradeLlmDemo() que sempre considera o contexto suficiente, e criarAnswerLlmDemo() com uma resposta fixa. Isole o embedDocuments() em lote — usado só pelo seed real — numa função separada, criarEmbeddingsReal(), porque o seed nunca deve rodar em modo demo.
```

Com a flag ligada, o backend responde com mocks, que retornam sempre a informação usada no teste:

- os embeddings devolvem um vetor de zeros, porque a busca por similaridade também é fixture;
- a busca devolve sempre o mesmo documento real da Vale no terceiro trimestre de 2025, congelado do corpus;
- o grade considera sempre o contexto suficiente;
- o answer devolve uma resposta fixa.

O embedding em lote, usado só pelo seed real, fica isolado em uma função separada, porque o seed nunca deve rodar em modo demo.

Tudo isso só vale com a variável de ambiente `DEMO` igual a `true`. Depois da refatoração, o projeto tem 82 testes unitários.

Público

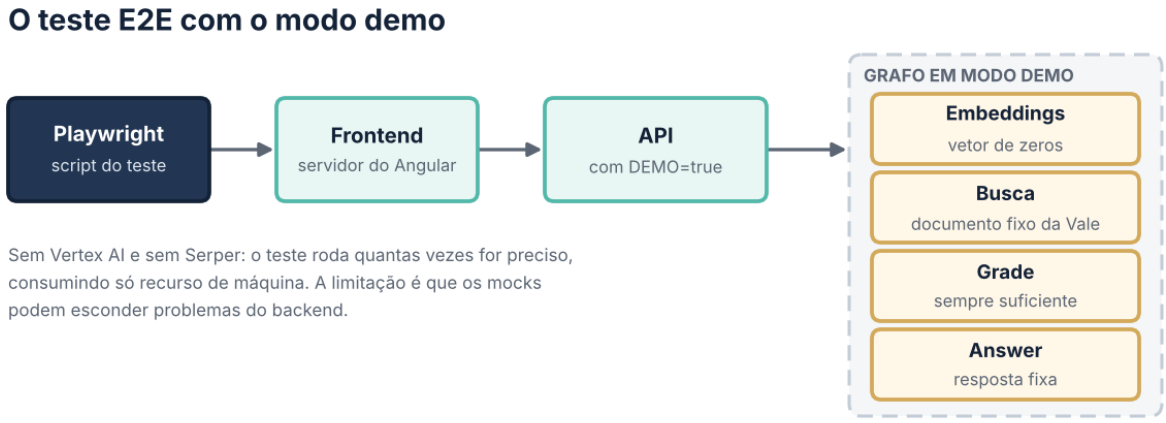


Diagrama editorial do ambiente de teste descrito na aula e nos prompts.

Trade-offs de testar com mocks

Testar com mocks pode maquiar problemas do backend. É uma escolha, e aqui pesaram o custo e a rapidez do teste.

O teste continua sendo um E2E, com limitações. As alternativas também têm custo:

- **Flags mais finas**: se o risco for crítico, dá para criar uma flag que só desligue o Serper e outra que só faça o mock do Vertex AI.
- **Sem mocks**: o teste pode quebrar por uma razão que não depende de você, como o Serper fora do ar. A aplicação trata essa falha, mas o teste não passa por um problema que não veio da sua alteração.

> **EM SÍNTESE**
> Na engenharia de sistemas, tudo é risco e trade-off. Não há certo e errado: depende do contexto e do problema.

O teste E2E com Playwright

O teste é criado e executado com este prompt:

Prompt enviado ao Claude Code

```
Crie apps/web/e2e/chat-happy-path.spec.ts com Playwright. Sobe os dois servidores reais via webServer do playwright.config.ts — apps/api com DEMO=true e apps/web com ng serve. O teste: disclaimer sempre visível, envia a pergunta 'Qual foi a receita líquida da Vale no 3T25?', espera a resposta completa em streaming conter 'R$ 56,7 bilhões', clica na citação e confirma data-base '2025-09-30', origem 'Base local (CVM)' e o link pro ZIP da CVM, e confirma que o painel do grafo terminou em 3 etapas — sem passar por Search.No final execute o teste pelo playwrite
```

O prompt define o ambiente, com os dois servidores reais, a API com `DEMO=true` e o web com o `ng serve`, e um caso de teste simplificado:

1. Verificar que o disclaimer está sempre visível.
2. Enviar a pergunta sobre a receita líquida da Vale.
3. Esperar a resposta completa.
4. Clicar na citação e confirmar a data-base, a origem e o link para o ZIP da CVM.
5. Confirmar que o painel do grafo terminou em três etapas, sem passar por search.

Para rodar, o Claude Code instala o Chromium e configura o Playwright. Cypress, Selenium ou outra ferramenta serviriam igualmente. O teste roda em background: ao contrário do Claude no Chrome, o Playwright não abre o navegador na tela.

Público

EXEMPLO · Testes a partir de BDD

A regra de negócio dos testes pode ser escrita em Cucumber, e o agente cria o teste do Playwright a partir dela. É um caminho válido, porque muitos analistas de teste já conhecem o BDD (Behavior-Driven Development) e sabem escrever nesse padrão.

A inconsistência que o teste revelou

Ao criar o teste, o Claude Code encontrou uma inconsistência e parou para perguntar. O teste pedia a confirmação da origem depois do clique na citação, mas o template só renderizava o link e o período. Nenhum lugar da interface mostrava a origem. A skill de frontend já documentava um painel de detalhes da citação, com uma tabela de chave e valor que inclui a origem, ainda não implementado.

O agente ofereceu dois caminhos:

- **Painel de detalhes completo**, com data-base, origem e trecho. Dá mais trabalho no frontend, mas fecha com o design documentado, e o teste reflete a interface pretendida.
- **Fix mínimo**: exibir a origem inline, com um pequeno elemento ao lado do que já existe, sem painel de detalhes. O escopo é bem menor.

Escolhi o fix mínimo, porque o foco era o teste E2E. Com ele, o teste foi criado e passou.

A aplicação completa e o starter são open source. Você pode fazer o fork, ajustar o código e mudar a regra de negócio como quiser.

CHECKPOINT DE APRENDIZAGEM

• Você consegue explicar, sem consultar o material, por que o disclaimer é renderizado incondicionalmente e sem botão de fechar?

• Você consegue explicar por que o teste E2E da esteira usa um script de Playwright, e não um agente operando o navegador, sem consultar o material?

• Você consegue explicar o que o modo demo estendido substitui e quais riscos e vantagens ele traz para o teste sem consultar o material?

PRÁTICA RECOMENDADA

• Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

• Um disclaimer fixo e sempre visível completa, na interface, a regra de que o sistema informa e não recomenda.

• Testar com um agente operando o navegador é útil no desenvolvimento, mas caro e lento para a esteira; ali, o agente escreve o script e o Playwright o executa.

• O modo demo foi estendido para o grafo inteiro, o que evita o custo do Serper e do Vertex AI nos testes, com a limitação de poder esconder problemas do backend.

• O teste E2E verifica o disclaimer, a resposta, a citação e o painel do grafo, e a sua criação revelou uma inconsistência entre a interface e o que o teste esperava.

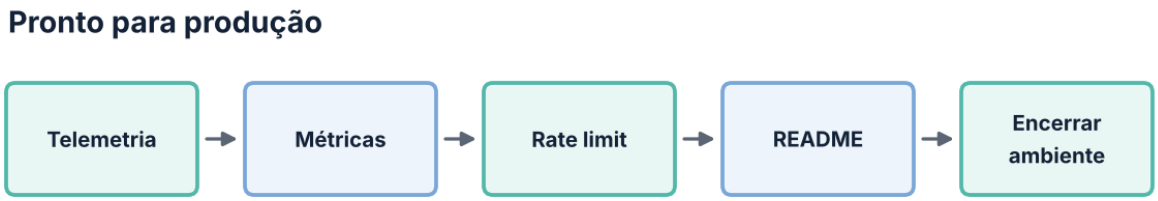
Público

Unidade 5: Observabilidade e produção

Da telemetria por chamada ao controle de custo e ao encerramento do ambiente.

NESTA UNIDADE

- Aula 22: Langfuse: tour e instrumentação por nó
- Aula 23: Métricas, budget de tokens e rate limiting
- Aula 24: README e desligando o ambiente pelo Console do GCP



Fluxo conceitual criado para apoiar a leitura da unidade.

GITHUB E PRÁTICA

- Projeto completo: Capstone Project AI Researcher
- Ponto de partida do estudante: AI Researcher Seed

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público

Aula 22: Langfuse: tour e instrumentação por nó

Voltar ao mapa da disciplina • *Voltar à Unidade 5*

OBJETIVOS DE APRENDIZAGEM

- Explicar por que uma aplicação em produção precisa de observabilidade e o que é telemetria.
- Preparar um projeto no Langfuse e instrumentar o grafo do agente para enviar a telemetria.
- Reconhecer o papel do OpenTelemetry como padrão aberto e do padrão interceptor na instrumentação.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Uma aplicação production-ready não se resolve só com controle de qualidade e testes. Com a aplicação em produção, duas perguntas precisam de resposta: como saber se o usuário está tendo algum problema e como controlar o custo.

Na esteira, no ambiente de desenvolvimento ou no de homologação, é possível parar, abrir o capô e ver o que está acontecendo. Em produção, as respostas têm de vir com o avião voando. É o papel da observabilidade.

Esta aula trata da observabilidade do consumo de modelos. Ao final, você saberá explicar o que é telemetria, instrumentar o grafo do agente para enviá-la a uma ferramenta de observabilidade e conferir o trace e o custo de cada chamada.

Observabilidade, telemetria e instrumentação

Em observabilidade, fala-se muito de telemetria.

> **CONCEITO-CHAVE · Telemetria**
> São os dados puros de log, de quando algo começou e de quando terminou, que o sistema vai enviando ao servidor. É o sistema informando que está tudo bem, que deu erro em tal ponto, que tal operação demorou tanto. Na prática, é um JSON com essas informações.

Sobre a telemetria constroem-se o monitoramento e os alarmes. O ponto fundamental é a ingestão: sem a telemetria chegando, não há o que monitorar.

> **CONCEITO-CHAVE · Instrumentar**
> É usar o SDK da ferramenta de observabilidade nos pontos de chamada do projeto, para que ele envie a telemetria. Dependendo do framework, isso nem é tão invasivo.

No uso de modelos, o custo deve ser considerado desde o começo, sempre pesado contra o valor que se quer entregar ao usuário.

O Langfuse

O projeto usa o Langfuse, um framework de observabilidade de agentes. Como outros frameworks open source, ele também tem um serviço de nuvem.

O plano gratuito atende até dois usuários, com retenção de 30 dias, e não exige cartão de crédito. É o suficiente para o desenvolvimento e permite conhecer uma ferramenta real de observabilidade.

O Langfuse não é a única opção. Há outras ferramentas do gênero, e ferramentas de observabilidade tradicionais também já têm módulo de IA. Ele serve aqui para explicar os conceitos.

Público

EM SÍNTESE

Tecnologias, nuvens e agentes de código são efêmeros: ferramenta se troca como quem troca de roupa. O valor do profissional está nos fundamentos, no pensamento estratégico e analítico e na prática.

Preparando o projeto e as chaves no Langfuse

A preparação tem três passos:

1. Faça o cadastro no Langfuse e o login. Você chega ao dashboard.
2. Crie um projeto e, em Settings, crie a API key, com a secret key.
3. Coloque no ambiente de desenvolvimento as três variáveis de ambiente que a própria tela indica, no arquivo de ambiente ou nas variáveis do sistema operacional.

A instrumentação do grafo

O grafo é instrumentado com este prompt:

Prompt enviado ao Claude Code

```
Instrumente o grafo do agente com Langfuse. Registre um NodeTracerProvider do OpenTelemetry com
LangfuseSpanProcessor (@langfuse/otel) uma vez no boot de cada processo que pode invocar o grafo —
apps/api e os dois transportes do packages/mcp-server. Crie um CallbackHandler (@langfuse/langchain) a
cada chamada de graph.stream()/graph.invoke(). Sem as chaves configuradas, nada disso deve registrar
nada nem impedir o processo de subir.
```

O prompt define três pontos:

- **Provider no boot**: o provider do OpenTelemetry é registrado, com o span processor do SDK do Langfuse, uma vez na inicialização de cada processo que pode invocar o grafo. São três: a API e os dois transportes do servidor MCP.
- **Callback a cada chamada**: um callback handler é criado a cada chamada do grafo, já que o projeto usa o LangChain.
- **Sem chaves, sem efeito**: sem as chaves configuradas, nada é registrado, e o processo sobe normalmente.

No código, a telemetria é inicializada quando a aplicação é instanciada, antes de o backend Node subir, e a função de callback fica preparada para as chamadas.

A documentação do Langfuse oferece uma skill própria para agentes. Em uma aplicação simples como esta, o Claude Code se sai bem sem ela, explorando o Langfuse por conta própria. No seu projeto, instalar a skill deixa o agente mais rápido e mais assertivo nessa atividade. A instrumentação também pode partir de uma spec de fim a fim, escrita em conversa com o agente.

OpenTelemetry: um padrão aberto

Na implementação aparece o termo OTel, de OpenTelemetry.

CONCEITO-CHAVE · OpenTelemetry

É um framework open source e um padrão aberto de telemetria, aceito pela indústria e pela maioria das ferramentas de observabilidade.

A vantagem do padrão é permitir a troca de ferramenta sem grandes traumas. Com um ambiente ELK customizado ou com o Dynatrace, por exemplo, uma aplicação que conversa em OpenTelemetry consegue enviar as informações, porque elas estão em um formato aceito.

O padrão interceptor

Para enviar a telemetria ao Langfuse, a chamada do stream do grafo continua a mesma, com a pergunta. A diferença está no segundo parâmetro, que recebe uma função de callback.

Público

CONCEITO-CHAVE · Interceptor
É uma função no meio do caminho, entre a chamada e o que essa chamada executa. Ela intercepta a chamada e faz o que tem de fazer, aqui o envio da telemetria, sem interferir em nada na execução. É um modo menos invasivo de instrumentar.

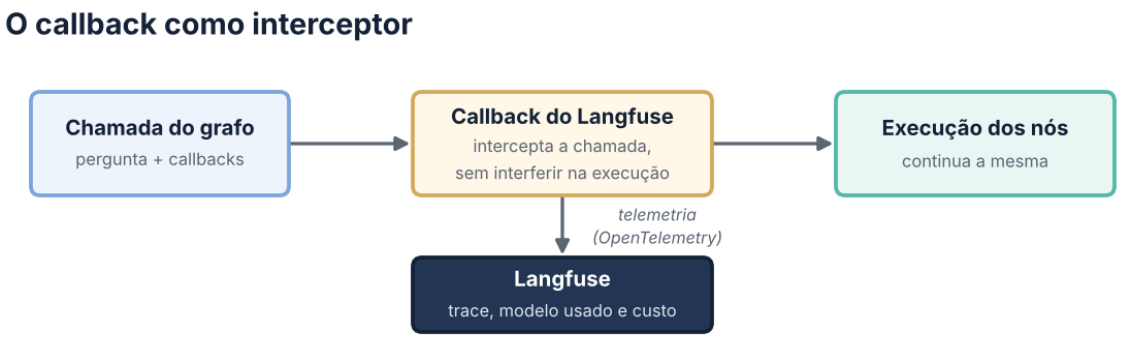


Diagrama editorial do padrão interceptor aplicado ao envio de telemetria.

O interceptor se junta a outros padrões de projeto usados no curso, como o adapter e o strategy. É um conhecimento da década de 90, ainda importantíssimo. Estudar os modelos e as ferramentas mais recentes é necessário, mas estudar fundamentos é mais importante do que nunca. Hoje os agentes programam muito mais do que as pessoas, e conhecer os padrões de projeto é o que permite guiar o agente e identificar o que está acontecendo no código.

Testando com o modo demo desligado

Para conferir o envio da telemetria, as chamadas ao Vertex AI precisam ser reais.

Com a variável `DEMO` igual a `true`, o sistema responde com mocks e não chama os modelos. Para ver a telemetria das chamadas reais, o teste é feito com o demo falso.

Na demonstração, o próprio Claude Code subiu a aplicação, perguntou qual foi o lucro líquido da Vale no último trimestre e verificou, pela API do Langfuse, se a telemetria tinha chegado. Dois contratempos apareceram no caminho:

- **Chaves**: o agente procurou as chaves do Langfuse no arquivo `.env`, mas elas estavam nas variáveis de ambiente do sistema. Ele verificou e passou a usá-las.
- **Porta**: a porta 3000 ainda estava em uso por um teste anterior. Ele liberou a porta e executou de novo.

O teste é manual e local, mas em produção funciona do mesmo jeito. Com os secrets no vault da nuvem, do GCP ou de qualquer outra, a aplicação envia a telemetria para o mesmo lugar.

O que o dashboard do Langfuse mostra

Com a chamada feita, o dashboard mostra:

- **Trace**: em Tracing, cada chamada aparece registrada, para investigação em caso de problema.
- **Custo**: o Langfuse identifica o modelo usado, o Gemini 3.5 Flash, e calcula o custo da chamada. Uma chamada só custa centavos.
- **Alertas**: é possível definir alertas, que enviam e-mail quando um threshold é atingido.

ATENÇÃO · Confira a região
O Langfuse tem regiões: Estados Unidos, Europa, Japão e HIPAA, esta para ambientes nos Estados Unidos com dados médicos, que têm outro tipo de governança. Se o ambiente foi configurado para uma região e o monitor aberto é de outra, os dados não aparecem.

Produção sempre exige observabilidade. Sem ela, um problema em produção obriga a simular a situação em homologação para tentar entender o que houve. Com a aplicação instrumentada, dá para saber o que está acontecendo e monitorá-la em tempo real.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar por que a observabilidade é necessária em produção e o que é telemetria sem consultar o material?
- Você consegue explicar o que é preciso criar no Langfuse e o que o prompt de instrumentação manda fazer sem consultar o material?
- Você consegue explicar, sem consultar o material, o que o OpenTelemetry padroniza e como o padrão interceptor envia a telemetria sem interferir na execução?

PRÁTICA RECOMENDADA

- Crie a sua conta e um projeto no Langfuse, configure as três variáveis de ambiente indicadas na tela de API keys e execute uma pergunta com o modo demo desligado. Depois, localize o trace e o custo da chamada no dashboard.

RESUMO DA AULA

- Em produção, é preciso saber o que está acontecendo com a aplicação e quanto ela custa, e isso depende de observabilidade.
- Instrumentar é colocar o SDK da ferramenta nos pontos de chamada para enviar telemetria, os dados de início, fim e erro de cada operação.
- O OpenTelemetry é um padrão aberto, aceito pela maioria das ferramentas, e o callback do Langfuse entra como interceptor, sem alterar a execução do grafo.
- No dashboard do Langfuse aparecem o trace de cada chamada, o modelo usado e o custo calculado, e é possível configurar alertas.

Público

Aula 23: Métricas, budget de tokens e rate limiting

Voltar ao mapa da disciplina • *Voltar à Unidade 5*

OBJETIVOS DE APRENDIZAGEM

- Diferenciar telemetria, que registra item a item, de métrica, que é o valor agregado ao longo do tempo.
- Criar contadores de tokens e de custo e expô-los em um endpoint de métricas.
- Implementar o rate limiting e um orçamento diário de tokens como mecanismos de controle de custo e de volumetria.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Aplicações agênticas podem ficar bem caras. Em produção, vale ter controles de custo dentro da própria aplicação, para não haver surpresa indesejável quando um usuário abusa ou quando o gasto passa do que a empresa deseja.

Esta aula cria as métricas internas do agente e, sobre elas, dois mecanismos de controle: o rate limit, que limita o volume de chamadas, e o orçamento diário de tokens. Ao final, você saberá diferenciar telemetria de métrica, explicar de onde vêm os números de custo e descrever o que acontece quando cada limite é atingido.

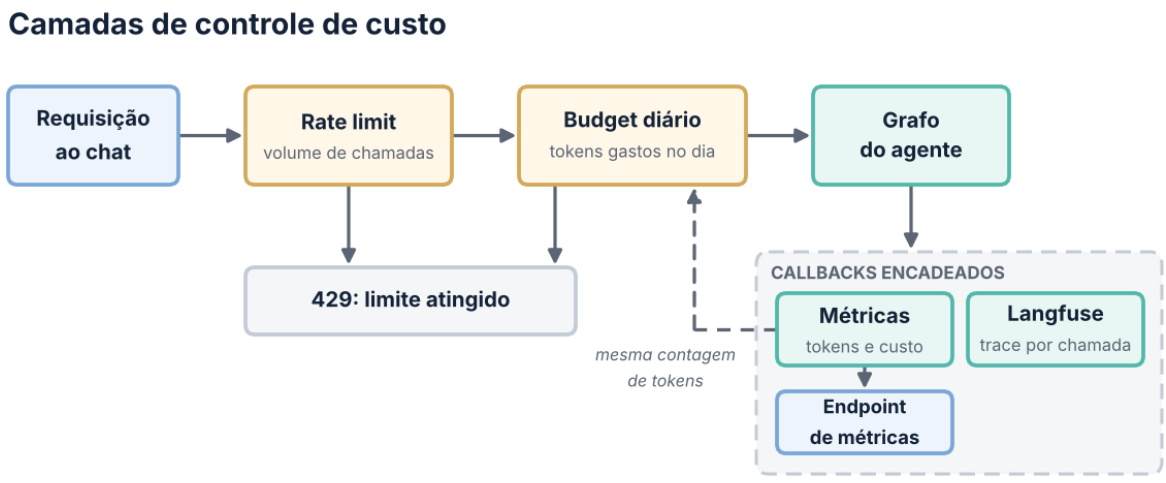


Diagrama editorial com os controles criados na aula e a origem dos dados de cada um.

Telemetria e métrica

Os dois termos descrevem dados diferentes:

- **Telemetria**: o dado analítico, item a item. É o trace de cada chamada.
- **Métrica**: o agregado. Quanto neste mês, quanto nesta semana e assim por diante.

O Langfuse oferece os dois. Mesmo assim, o projeto cria uma métrica própria, local, porque é sobre ela que os controles de custo são implementados no agente.

CONCEITO-CHAVE · FinOps
É saber controlar o custo da aplicação.

Público

Com a métrica na aplicação, um abuso ou um custo acima do desejado pode receber um tratamento de erro ou outro tratamento, como um fallback para um modelo mais barato.

Contadores de tokens e de custo

A métrica interna é criada com este prompt:

Prompt enviado ao Claude Code

```
Crie contadores Prometheus (prom-client) num Registry próprio: llm_tokens_total (labels node, model,
kind) e llm_cost_usd_total (labels node, model). Crie um callback LangChain genérico que correlacione
handleChatModelStart/handleLLMEnd por runId, lendo metadata.langgraph_node e metadata.ls_model_name, e
incremente os contadores a partir do usage_metadata real devolvido pela Vertex AI — nunca uma
estimativa. O custo vem de uma tabela de preço USD por milhão de tokens; modelo fora da tabela loga
aviso e conta como custo zero, nunca quebra a resposta. Esse callback deve entrar somado ao
criarLangfuseCallbacks() já existente — callbacks: [...criarLangfuseCallbacks(), criarMetricsCallback()]
— no mesmo call site do graph.stream()/invoke() em apps/api/src/app.ts, nunca sozinho dentro de llm.ts:
o LangGraph propaga o callback manager por AsyncLocalStorage, e um callbacks explícito dentro de um nó
substituiria esse ambiente em vez de somar a ele.
```

O prompt pede:

- **Dois contadores Prometheus**, em um registro próprio, com uma biblioteca bem utilizada para esse tipo de métrica: o total de tokens e o custo total, em dólar.
- **Um callback genérico do LangChain**, nas chamadas do agente, que lê os metadados, como o nome do modelo, e incrementa os contadores a partir do uso real devolvido pelo Vertex AI, nunca de uma estimativa.
- **Uma tabela de preço**, em dólar por milhão de tokens, de onde vem o custo. Um modelo fora da tabela gera um aviso no log e conta como custo zero, sem nunca quebrar a resposta.
- **O ponto de ligação**: o callback entra somado ao do Langfuse, no mesmo ponto em que o grafo é chamado.

> **CONCEITO-CHAVE · Interceptors encadeados**
> Vários interceptors podem ser colocados na mesma chamada, sempre independentes, cada um fazendo uma coisa, sem influenciar a regra de negócio. Aqui, um captura a telemetria para o Langfuse e outro cria as métricas locais.

O trabalho do Claude Code é parecido com o da instrumentação do Langfuse: ele aproveita a mesma estrutura, cria as funções de métricas, com os contadores, e a tabela com o valor por milhão de tokens dos modelos em uso.

Dois limites dessa implementação:

- **Precisão**: o cálculo não é tão preciso quanto o do Langfuse. Há serviços que retornam em tempo real o custo de vários modelos, mas a opção aqui foi simplificar.
- **Biblioteca depreciada**: na execução, o agente avisou que a biblioteca foi depreciada. Mantive, por fins didáticos. No seu desenvolvimento, use a versão mais nova.

O endpoint de métricas

As métricas são expostas em uma rota própria:

Prompt enviado ao Claude Code

```
Exponha GET /api/metrics no formato Prometheus, usando o Registry já criado. Defina uma interface mínima
pro registry, não o tipo concreto do prom-client, pra apps/api não ganhar essa dependência.
```

A rota entrega as métricas no formato Prometheus, usando o registro já criado. O registro entra por uma interface mínima, e não pelo tipo concreto da biblioteca, para a API não ganhar essa dependência.

Rate limit: o limite de rajadas de chamadas

O rate limit independe de a aplicação ser agêntica. É um conceito de aplicação de backend.

Público

CONCEITO-CHAVE · Rate limit

É o limite de chamadas, das rajadas de chamadas, na aplicação. Quando o limite é atingido, a aplicação devolve um 429, o erro que indica chamadas demais.

Ele existe para prevenir problemas. Toda aplicação tem um padrão de comportamento, conhecido pela regra de negócio ou por observação. Um volume de chamadas muito acima desse padrão pode indicar algo errado:

- uma invasão;
- um usuário cometendo abuso;
- um bot abusando da API, no caso do MCP;
- algo em loop infinito.

As consequências são o custo muito alto ou a queda da aplicação por uma volumetria para a qual ela não está preparada.

EXEMPLO · Um chamador em loop

Um desenvolvedor sobe um código que fica em loop e chama o seu microsserviço sem parar. Sem rate limit, o seu serviço cai porque o chamador está descontrolado. O rate limit breca isso.

Volumetria depende do negócio

A volumetria que a arquitetura precisa suportar depende dos requisitos.

EXEMPLO · Cem usuários ou um milhão

Uma aplicação interna, com no máximo 100 usuários concorrentes, não precisa de servidor, arquitetura e aplicação para um milhão de usuários, porque nunca vai ter um milhão de usuários. Nem toda aplicação vai ser um novo Facebook ou um novo Google.

Mesmo uma aplicação como a do Pix precisa de um rate limit mínimo, ainda que bem grande, porque a infraestrutura e a arquitetura são pensadas para um tipo de volumetria.

Quando o volume cresce com justificativa de negócio, porque a empresa cresceu muito, a premissa do projeto mudou, e muitas vezes é preciso repensar a infraestrutura ou a própria arquitetura. Fora dessas justificativas, um volume acima do limite é sinal de problema.

Rate limit e budget diário na aplicação

Os dois controles são criados com um único prompt:

Prompt enviado ao Claude Code

```
Adicione rate limiting global com @fastify/rate-limit, configurável por
RATE_LIMIT_MAX/RATE_LIMIT_WINDOW, desligado em /health e /metrics, com as opções calculadas dentro de
uma função chamada em build() — não num objeto de módulo lido no import, pra funcionar tanto em produção
quanto em teste que ajusta a env var em runtime. Adicione um budget diário de tokens em memória, que
zera na virada do dia UTC, alimentado pela mesma fonte de tokens do callback de métricas — não uma
segunda contagem. Verifique o orçamento no início da rota /chat, antes de escrever qualquer byte da
resposta SSE.
```

Para o rate limit, o prompt define:

- **Plugin do framework**: o rate limit é global, feito com o plugin do Fastify, configurado pelo máximo de chamadas e pela janela de tempo.
- **Rotas isentas**: as rotas de health e de métricas ficam de fora, porque só leem dados da própria aplicação.
- **Opções calculadas em função**: as opções são calculadas dentro de uma função chamada no build, e não em um objeto de módulo lido no import. Assim funcionam em produção e em testes que ajustam a variável de ambiente em runtime.

Para o orçamento, o prompt define:

- **Budget diário em memória**: um limite de tokens que zera na virada do dia.

Público

- **Mesma fonte das métricas**: o budget é alimentado pela mesma fonte de tokens do callback de métricas, e não por uma segunda contagem. Por isso as métricas foram criadas antes.
- **Verificação antecipada**: o orçamento é verificado no início da rota `/chat`, antes de qualquer byte da resposta ser escrito.

Quando o limite diário é atingido, quem chama recebe um 429, seja o frontend, seja o agente que consome o MCP. Outro tratamento possível é o fallback: se está caro, o sistema passa a usar um modelo local ou um modelo mais barato, que não dá respostas tão boas, mas não estoura o orçamento.

A tabela a seguir compara os dois controles:

Critério	Rate limit	Budget diário de tokens
O que controla	O volume de chamadas, as rajadas de chamadas à aplicação.	O gasto acumulado de tokens no dia.
De onde vem o dado	Da contagem de requisições, feita pelo plugin do Fastify.	Da mesma fonte de tokens do callback de métricas, sem uma segunda contagem.
Onde atua	Em todas as rotas, exceto as de health e de métricas.	No início da rota `/chat`, antes de escrever qualquer byte da resposta.
Quando o limite é atingido	Devolve 429: chamadas demais.	Devolve 429; outra opção seria um fallback para um modelo mais barato.

> **EM SÍNTESE**
> O orçamento só pode ser controlado porque há métricas. Sem métricas, não há como controlar.

O que foi implementado e o que o error handler facilitou

Na implementação, o Claude Code instrumentou o rate limit e o desligou, nas configurações, nas rotas de métricas e de health. Ele não carregou a skill do Fastify para isso, embora ela exista no projeto.

Para o orçamento, criou uma nova exceção, a de orçamento excedido. A verificação é simples: consulta as métricas, pega o total do dia, que já vem acumulado, e compara com o orçamento diário.

Com os erros unificados na Aula 16, criar um novo tipo de exceção exigiu mexer em um lugar só, de forma bem mais organizada.

Os testes unitários pegaram um erro: mesmo com o rate limit lançando o 429, o error handler sobrescrevia o status com 500. O tratamento foi ajustado no error handler, e os testes passaram.

Mais uma camada de proteção

O rate limit se soma aos mecanismos de resiliência da Aula 15. São várias camadas: se o rate limit não existir ou for mal calculado, a aplicação ainda tem a degradação graciosa.

A maioria dos frameworks, de JavaScript e de Java, traz o rate limit de forma praticamente nativa, e não é preciso recriar o mecanismo. O Express, que é muito básico, tem bibliotecas de rate limit que podem ser adicionadas ao projeto.

Os fundamentos valem em qualquer linguagem. O projeto usa JavaScript de fim a fim porque isso facilita explicar o frontend, o backend e o resto, mas o seu Capstone pode ser em Go, em Python ou em outra linguagem, e o frontend pode ser em React no lugar do Angular.

Um pet project muitas vezes não precisa de observabilidade nem de controle de custo. Em aplicações de produção, os requisitos não funcionais de custo e de controle de volumetria são muito importantes.

Público

CHECKPOINT DE APRENDIZAGEM

- Você consegue explicar a diferença entre telemetria e métrica sem consultar o material?
- Você consegue explicar como os contadores de tokens e de custo são alimentados e de onde vem o valor do custo sem consultar o material?
- Você consegue explicar o que o rate limit e o budget diário de tokens protegem e o que acontece quando cada limite é atingido sem consultar o material?

PRÁTICA RECOMENDADA

- Abra o projeto da aula no GitHub, execute o exemplo e faça pelo menos uma alteração relevante. Depois, compare o comportamento antes e depois da mudança.

RESUMO DA AULA

- Telemetria é o registro item a item; métrica é o agregado, e é com ela que se controla o custo dentro da própria aplicação.
- Um callback encadeado ao do Langfuse alimenta contadores de tokens e de custo com o uso real devolvido pelo Vertex AI, expostos em um endpoint de métricas.
- O rate limit protege a aplicação contra volumes anormais de chamadas e é um conceito de backend, não só de aplicações agênticas.
- O budget diário de tokens usa as mesmas métricas e bloqueia a rota de chat quando o orçamento do dia é excedido.

Público

Aula 24: README e desligando o ambiente pelo Console do GCP

Voltar ao mapa da disciplina • *Voltar à Unidade 5*

OBJETIVOS DE APRENDIZAGEM

- Documentar o projeto em um README com visão geral e diagramas Mermaid gerados com apoio do agente.
- Explicar por que o Mermaid é adequado para registrar arquitetura quando se trabalha com IA.
- Encerrar o projeto no console do Google Cloud para não continuar pagando pelos recursos provisionados.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Visão geral

Com o desenvolvimento concluído, faltam duas tarefas. A primeira é a documentação: um bom README, que descreva todo o projeto. A segunda é desligar o ambiente na nuvem, para que ele não continue gerando custo.

Ao final, você saberá gerar a documentação de arquitetura em diagramas escritos como texto e encerrar o projeto no console do Google Cloud com segurança.

O README do projeto

O README fica na raiz do projeto e é gerado pelo Claude Code, depois de todo o desenvolvimento:

Prompt enviado ao Claude Code

```
Crie um README.md na raiz do projeto com visão geral, e três diagramas Mermaid: a arquitetura de infraestrutura completa (cliente, Cloud Run, Google Cloud, serviços externos), o grafo do agente como RAG corretivo, e o pipeline de deploy canary. No diagrama do grafo, nomeie o node principal como agentGraph, não graph — é palavra reservada da gramática do Mermaid mesmo dentro de flowchart TB e quebra o parser com um erro genérico. Inclua o disclaimer regulatório logo no início. Renderize os três com @mermaid-js/mermaid-cli antes de considerar pronto.
```

O prompt pede:

- **Visão geral** do projeto.
- **Disclaimer regulatório** logo no início.
- **Três diagramas Mermaid**: a arquitetura de infraestrutura completa, o grafo do agente como RAG corretivo e o pipeline de deploy canary.
- **Renderização dos três diagramas** com o CLI do Mermaid antes de considerar o trabalho pronto. É um teste.

> **ATENÇÃO · graph é palavra reservada no Mermaid**
> No diagrama do grafo, o nó principal se chama `agentGraph`, e não `graph`. A palavra `graph` é reservada na gramática do Mermaid, mesmo dentro de um flowchart, e quebra o parser com um erro genérico.

Na demonstração, retirei a última frase do prompt, porque a renderização demoraria demais e o GitHub carrega o Mermaid naturalmente.

Público

Mermaid: diagramas escritos como código

> **CONCEITO-CHAVE · Mermaid**
> É uma biblioteca que permite criar diagramas de arquitetura dos mais diferentes tipos com uma linguagem de marcação bem simples. O diagrama é escrito na notação do Mermaid, e a biblioteca renderiza a imagem, que pode ser exportada.

A notação é simples, embora possa ficar bem complicada. O Mermaid tem vários tipos de diagrama, como os de componentes, de sequência, de Gantt, de classes e de Git. É open source e muito utilizado, e hoje também tem a sua IA.

O GitHub renderiza o Mermaid naturalmente: se o Markdown tiver um bloco Mermaid, a página mostra o diagrama.

Por que o Mermaid combina com IA

Os modelos conseguem gerar imagens, mas são muito melhores e muito mais baratos para gerar texto. Com o Mermaid, o modelo gera o diagrama na notação de texto, e a renderização fica por conta do mecanismo do próprio Mermaid, que o GitHub e várias bibliotecas suportam.

Isso faz do Mermaid um bom jeito de registrar e formalizar desenhos de arquitetura:

- **Sem desenhar à mão**: o que mais me irrita em um desenho de arquitetura não é a arquitetura, que é a parte divertida. É ficar no PowerPoint fazendo quadradinhos, errar e ver tudo ficar torto. No Mermaid, o diagrama é escrito, e o desenho é gerado.
- **Da conversa ao diagrama**: ao explorar a arquitetura de uma aplicação com a IA, a conversa termina com um diagrama Mermaid pronto, que serve para explicar a arquitetura à equipe ou para justificá-la.
- **Fácil de alterar**: trabalhar com texto facilita a mudança. Basta alterar o diagrama, e a imagem é renderizada de novo.

Antes, desenhar as caixas tinha valor na hora de discutir a arquitetura. Hoje, a maior parte da discussão acontece com o agente, e o diagrama registra o resultado.

Pouca gente usa o Mermaid, inclusive poucos arquitetos, o que é uma pena. Há outras ferramentas para desenhar, mas ele é a minha preferida, e é mais barato usar um agente de IA para pensar e registrar a arquitetura dessa forma.

Conferindo os diagramas

O README gerado traz um trecho em tabela e, mais adiante, os blocos em Mermaid, com os diagramas da infraestrutura, do grafo e do deploy.

Na demonstração, o preview de Markdown do VS Code não renderizou os diagramas corretamente, mesmo com a extensão de suporte a Mermaid instalada. A saída foi pedir ao agente que renderizasse e testasse e conferir o resultado no próprio GitHub. Talvez a frase de renderização não devesse ter saído do prompt.

No GitHub, o Markdown no padrão Mermaid é renderizado corretamente, e é possível dar zoom, exportar e copiar o diagrama.

Encerrando o projeto na nuvem

O projeto roda em produção, em uma nuvem, e isso tem custo. Depois de subir o serviço e, se quiser, compartilhar o link por um tempo, apague o ambiente.

> **ATENÇÃO · O Cloud SQL cobra mesmo sem uso**
> O Vertex AI e o Cloud Run só geram custo a partir do uso. O Cloud SQL, por causa do armazenamento, cobra de qualquer maneira, mesmo que pouco.

No Google Cloud, o encerramento é feito assim:

1. No console, vá à parte de IAM e administrador e abra as configurações, onde está o projeto.
2. Clique em encerrar. É preciso ser administrador da conta, o que acontece naturalmente em uma conta pessoal.

Público

3. Digite o código do projeto para confirmar a desativação.

Em outra nuvem, o caminho é diferente, mas a providência é a mesma.

Com o encerramento, toda conexão do projeto é fechada automaticamente, e não há mais chamadas.

A janela de 30 dias

O encerramento não destrói o projeto imediatamente.

> **CONCEITO-CHAVE · Projeto arquivado por 30 dias**
> O GCP mantém o projeto encerrado arquivado por 30 dias, para o caso de arrependimento. A cobrança para a partir do pedido de encerramento, e a aplicação deixa de funcionar, mas o projeto pode ser restaurado nesse período. O mecanismo existe para evitar acidentes.

Em um ambiente corporativo, só o dono da conta tem esse acesso: não é qualquer administrador de sistema que consegue apagar o projeto. Em uma conta pessoal, encerrar o projeto é muito recomendado, para não ficar com nenhum custo.

Depois do projeto: expandir por conta própria

O aprendizado continua quando você pega o projeto e o expande:

- com outra linguagem de programação;
- com outro agente. O projeto usou o Claude, mas o código pode ser criado com o Codex, o Kimi ou o Gemini;
- com outra nuvem, como AWS ou Azure, ou com tudo montado na máquina local.

O projeto foi escrito quase todo pelo Claude Code, com pouca edição direta do código, e prompt a prompt. Outra forma de construí-lo é o spec-driven development.

> **EM SÍNTESE**
> O importante é entender os conceitos e os fundamentos e reconhecer o valor da boa engenharia de software, com ou sem IA.

CHECKPOINT DE APRENDIZAGEM

• Você consegue explicar o que o README do projeto deve conter e por que o nó principal do diagrama não pode se chamar `graph` sem consultar o material?

• Você consegue explicar por que o Mermaid combina bem com agentes de IA na documentação de arquitetura sem consultar o material?

• Você consegue explicar como encerrar o projeto no Google Cloud e o que acontece durante os 30 dias seguintes sem consultar o material?

PRÁTICA RECOMENDADA

• Depois de concluir e, se quiser, compartilhar o seu projeto, encerre-o no console da sua nuvem para não gerar custos. Em seguida, expanda o projeto por conta própria: refaça-o com outra linguagem, outro agente ou outra nuvem.

RESUMO DA AULA

• O projeto é documentado em um README com visão geral, disclaimer regulatório e três diagramas Mermaid: infraestrutura, grafo do agente e pipeline de deploy.

• O Mermaid descreve diagramas em texto, o que é mais barato e mais confiável para um modelo gerar, e o GitHub renderiza esses diagramas automaticamente.

• O Cloud SQL cobra mesmo sem uso, por isso o projeto deve ser encerrado no console do Google Cloud ao final dos estudos.

• O projeto encerrado para de cobrar e fica arquivado por 30 dias, período em que ainda pode ser restaurado.

Público

Fechamento da disciplina

Use este material como guia de estudo, não apenas como leitura. A proposta é alternar explicação, reprodução do código, experimentação, validação e adaptação.

CICLO DE REVISÃO FINAL

- Revisite os checkpoints das unidades sem consultar as respostas no texto.
- Abra os projetos do GitHub e identifique quais conceitos aparecem em cada implementação.
- Escolha um projeto e crie uma variação própria para portfólio.
- Registre dúvidas específicas para discutir com professores e comunidade.

GITHUB E PRÁTICA

- **Projeto completo: Capstone Project AI Researcher**
- **Ponto de partida do estudante: AI Researcher Seed**

Sugestão de estudo: tente reproduzir o passo a passo antes de consultar a implementação completa quando houver template e solução.

Público